

COURSE BOOK

Testing with Python

unittest, pytest and automotive software — lectures, labs
and a capstone

17 lecture chapters · Lab 0 + 13 labs · capstone · glossary · self-check answers

Unit · Integration · System · Property-based · Robustness · Regression · Performance · Simulation · Mutation ·
Acceptance

Python 3.10+ · pytest · unittest · Hypothesis · built 08 October 2026

Contents

Start here

About this course

Part I — Foundations

Chapter 1 — Why we test: foundations

Chapter 2 — The testing landscape

Chapter 3 — Automotive primer: the domain you will test

Part II — The tools: unittest and pytest

Lab 0 — Setup and your very first tests

Chapter 4 — unittest: the standard library's test framework

Lab 1 — Unit testing with `unittest`

Chapter 5 — pytest: concise, powerful tests

Lab 2 — pytest fundamentals

Part III — Test design and isolation

Chapter 6 — Designing tests from a specification

Lab 3 — Black-box test design

Chapter 7 — Isolating the unit: testability and test doubles

Lab 4 — Test doubles: isolating the unit

Part IV — Integration and system testing

Chapter 8 — Integration testing: where components meet

Lab 5 — Integration testing

Chapter 9 — State-based and system-level testing

Lab 6 — State-transition and system-level testing

Part V — Finding the unexpected

Chapter 10 — Property-based testing and fuzzing

Lab 7 — Property-based testing with Hypothesis

Chapter 11 — Robustness and fault injection

Lab 8 — Robustness: negative testing, fault injection and fuzzing

Part VI — Change, speed and simulation

Chapter 12 — Regression testing, golden files and replay

Lab 9 — Regression, golden-file and log-replay testing

Chapter 13 — Performance, timing and resource testing

Lab 10 — Performance and real-time budget testing

Chapter 14 — Simulation, closed-loop testing and acceptance criteria

Lab 11 — Software-in-the-loop (SIL) simulation and acceptance criteria

Part VII — Adequacy, evidence and delivery

Chapter 15 — Test adequacy: coverage and mutation testing

Lab 12 — Test adequacy: coverage and mutation testing

Chapter 16 — Requirements, traceability, standards and continuous integration

Lab 13 — Acceptance tests, requirements traceability and SIL/HIL switching

Part VIII — Capstone

Chapter 17 — Putting it all together: building a test strategy

Capstone — Tyre-Pressure Monitoring bug hunt

Appendices

Appendix A — Glossary

Appendix B — Further reading and resources

Appendix C — Cheat sheet

Appendix D — Code under test

Appendix E — Course tooling

Appendix F — Self-check answers

Start here

About this course

A hands-on course in which you learn **every major kind of software testing** by testing realistic automotive code (CAN bus, battery management, cruise control, ABS, diagnostics, tyre-pressure monitoring) with Python's `unittest` and `pytest`.

- **17 lecture chapters** (concepts, techniques, automotive background — every code example is executed by a checker), **13 labs** (plus a Lab 0 setup lab) and a capstone "bug hunt" (≈ 35 contact hours)
- Every lab: concept $\rightarrow$ fully worked, runnable examples $\rightarrow$ exercises $\rightarrow$ debrief questions
- A verified instructor solution for every exercise (`solutions/`), executed in CI
- No hardware needed: ECUs, sensors and vehicle dynamics are simulated; a switch shows how the same tests would run on a real bench (HIL)

Quick start (5 minutes)

```
python -m venv .venv          # a private Python environment for the course
source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt # pytest, hypothesis, coverage + the course library
python course.py doctor       # checks your setup and runs your first tests
```

Then, for every lab (read its lecture chapter first):

```
python course.py labs          # the list of labs and your progress
python course.py lab 3         # run the WORKED examples of Lab 3 - read them, then break them
python course.py exercise 3    # run YOUR exercises of Lab 3 (each starts red with a TODO)
python course.py progress      # progress bars for all labs
```

`course.py` works the same on Windows, macOS and Linux (no `make`, no `PYTHONPATH`). Start with Lab 0. Developers who prefer `make`: `make fast`, `make lab N=06` (see the Makefile).

Course map

#	Lab (read its lecture first)	Testing type / technique	Main tools	Automotive case
0	Setup and first tests	Getting started	<code>pytest</code> , <code>course.py</code>	a speed conversion
1	<code>unittest</code> fundamentals — Ch. 4	Unit testing	<code>unittest.TestCase</code> , <code>subTest</code> , <code>setUp</code> , <code>assertRaises</code>	CAN frames, CRC-8, unit conversions, wheel-speed maths
2	<code>pytest</code> fundamentals — Ch. 5	Unit testing, fixtures, data-driven	<code>assert</code> , <code>parametrize</code> , fixtures, <code>tmp_path</code> , markers	Temperature sensor, BMS fault detection

#	Lab (read its lecture first)	Testing type / technique	Main tools	Automotive case
3	Black-box test design — Ch. 6	Equivalence classes, boundary values, decision tables	<code>parametrize</code> , invariants	Battery charge-current derating
4	Test doubles — Ch. 7	Isolation: stub, fake, spy, mock, dummy	<code>unittest.mock</code> , <code>monkeypatch</code> , <code>FakeClock</code>	ADC hardware, diagnostics, timeouts without sleeping
5	Integration testing — Ch. 8	Integration / interface / contract	virtual bus, fault hooks	Wheel-speed ECU → CAN → instrument cluster
6	State machines & system tests — Ch. 9	State-transition, functional/system, scenario	exhaustive sequences	Cruise-control modes
7	Property-based testing — Ch. 10	Property-based / generative	Hypothesis	Signal codec round-trips, E2E protection, BMS safety invariants
8	Robustness & fault injection — Ch. 11	Negative, fault-injection, fuzz, recovery	seeded RNG, Hypothesis	Stuck sensors, bit errors, burst loss, fuzzing the UDS handler
9	Regression & golden files — Ch. 12	Regression, snapshot/golden, log replay	custom <code>golden</code> fixture	DTC reports, UDS transcripts, recorded drive cycle
10	Performance & real-time budgets — Ch. 13	Performance, timing, memory	<code>perf_counter</code> , <code>tracemalloc</code>	10 ms control cycle, saturated 500 kbit/s bus
11	Simulation (SIL) — Ch. 14	Closed-loop / model-based, acceptance criteria	plant models	Cruise control on hills, ABS vs locked wheels
12	Coverage & mutation — Ch. 15	Test-adequacy: coverage, mutation testing	<code>pytest-cov</code> , <code>tools/mini_mutate.py</code>	"100 % covered" suite that catches nothing
13	Acceptance, traceability, HIL — Ch. 16	Acceptance (Given/When/Then), requirements traceability, SIL↔HIL	custom marker + report	ISO 26262-style requirement → test matrix
★	Capstone: TPMS bug hunt — Ch. 17	Everything	all of the above	Tyre-pressure monitoring; graded against 17 hidden defects

Which test types appear where?

Test type	Labs	Test type	Labs
Unit	1, 2	Fuzz / negative	7, 8
Smoke / sanity	2, 5	Fault injection / recovery	5, 8

Test type	Labs	Test type	Labs
Black-box design	3	Regression / golden / replay	9
White-box (state, internals)	6, 12	Performance / memory	10
Integration / contract	5	Simulation (SIL) / HIL	11, 13
System / functional	6, 11	Test adequacy (coverage, mutation)	12
Property-based	7	Acceptance / traceability	13

Repository layout

```

src/autotest/      the software under test (the "ECU code")
  can.py           frames, signals, CRC-8, E2E
  sensors.py       conversions, checked sensor
  bus.py           virtual CAN bus, fault hooks
  ecu.py           wheel-speed ECU + cluster
  diagnostics.py   DTC debouncing + UDS-lite
  clock.py         SystemClock / FakeClock
course.py          one command for everything: doctor, labs, lab N, exercise N, progress
course/           17 lecture chapters, glossary, reading list (code examples verified)
labs/labNN_*/     README.md · worked tests (test_*.py) · exercise_labNN.py
capstone/         TPMS specification, starter suite, defect-injection grader
solutions/        instructor solutions (they run in CI, so they cannot rot)
tools/            mini_mutate.py (mutation tester), make_drive_log.py
docs/             requirements.csv, cheat sheet, instructor guide
confptest.py      shared fixtures (clock, bus, golden), --hil / --update-golden / --req-report

```

How each lab is organised

0. **Study** the lecture chapter for the lab (30–60 min) — `course/lecture_NN_*.md`, or the PDF.
1. **Read** the lab README (10 min) — the concept and why it matters in a vehicle programme.
2. **Run and read** the worked tests (`test_labNN_*.py`) — they are deliberately commented teaching material.
3. **Break something** — mutate the code under test and watch which test fails (this builds intuition for Lab 12 early).
4. **Do the exercises** in `exercise_labNN.py` (not collected by default; run the file explicitly).
5. **Debrief** with the discussion questions.

Reference

- pdf/ — the whole course as a printable **Course Book** (learners) and **Instructor Edition**; rebuild with `make pdf`
- docs/cheatsheet.md — unittest ↔ pytest side-by-side, useful flags, marker recipes
- docs/requirements.csv — the requirement set used for traceability

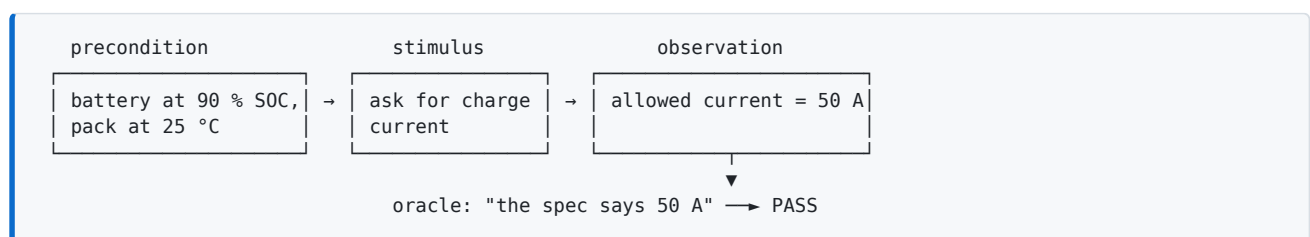
Safety note. The code here is teaching material, not production automotive software. The models are simplified; never use them for real vehicle decisions.

Part I — Foundations

Chapter 1 — Why we test: foundations

1.1 What a test is — and what it is not

A **test** is an *experiment* on software: you set up a situation, provoke a behaviour, and compare what happened with what the specification says *should* happen. The comparison rule is called the **test oracle**. Everything in this course is a variation on that sentence:



Three things distinguish a good test from a script that merely calls the code:

1. **It is automated and repeatable** — anyone, any day, gets the same verdict.
2. **It has an oracle** — it *decides* pass or fail; a human does not need to read a log.
3. **It is independent** of other tests — it can run alone, in any order.

Testing is **not** debugging. Testing *finds* a failure; debugging *locates and removes* its cause. Testing is **not** quality assurance either: QA is the whole process (requirements reviews, coding standards, audits, testing); testing is one activity inside it.

Edsger Dijkstra's famous remark applies to every chapter of this book: *testing can show the presence of defects, never their absence*. A suite that passes tells you "none of these experiments revealed a problem" — nothing more. The rest of the course is about making those experiments as revealing as possible.

1.2 Error, fault, failure

Engineers use three words that everyday language blurs. Keeping them apart makes bug reports precise:

Term	Meaning	Example from this course
Error (mistake)	A human slip	The programmer assumes every comparison with NaN behaves like a number
Fault (defect, bug)	The wrong code or data that results	<code>max_charge_current</code> compares <code>temp_c < 0 or temp_c > 45</code> , which is <code>False</code> for NaN
Failure	Observable wrong behaviour at run time	A sensor glitch reports NaN and the BMS allows 100 A charging at an unknown temperature

A fault is *latent* until an input activates it. Most test design is about choosing inputs that **activate** faults (boundaries, rare combinations, hostile values) and then **propagate** the wrong value to something the test can observe.

1.3 Verification and validation

- **Verification** — "Are we building the product *right*?" Checking against the specification: does the code do what requirement REQ-BMS-001 says?
- **Validation** — "Are we building the *right* product?" Checking against real needs: is that requirement what drivers and the safety case actually need?

Most automated tests in this book are *verification*. Acceptance scenarios (Lab 13) and closed-loop simulation (Lab 11) edge toward validation, because they ask whether the *behaviour* is acceptable, not only whether a function returns a number.

1.4 Why automotive software is different

A modern vehicle contains dozens of electronic control units (ECUs) running tens of millions of lines of code, supplied by many companies and integrated late. Several properties raise the bar for testing:

- **Safety.** A defect can injure people. Standards such as **ISO 26262** require a *systematic* verification argument with evidence — not just "we tested it."
- **Long life, many variants.** A platform lives for years and ships in hundreds of configurations. Re-testing by hand after every change is impossible; **regression automation** is the only option.
- **Distributed and real-time.** Behaviour emerges from ECUs talking over buses with deadlines of milliseconds. Many defects live *between* components.
- **Hardware coupling.** Software reads imperfect sensors, drives actuators and survives noisy power rails. Tests must model faults that rarely occur on a bench.
- **Cost of late defects.** A defect found by a unit test costs minutes; one found in the field can cost a software campaign or a recall. Finding defects *earlier* is the central economic argument for everything in this book.

1.5 The seven principles of testing

These principles, popularised by the ISTQB syllabus, summarise decades of experience. We return to each one in a lab.

1. **Testing shows the presence of defects, not their absence.** (Lab 12: a suite's *adequacy* must itself be measured.)
2. **Exhaustive testing is impossible.** `max_charge_current` has two continuous inputs; you cannot try them all. Hence *test design techniques* (Lab 3).
3. **Test early.** The unit tests of this course run in seconds, long before any hardware exists.
4. **Defects cluster.** A few modules — typically those with complex state or arithmetic at boundaries — contain most defects. Spend effort accordingly.
5. **The pesticide paradox.** Re-running the same tests finds fewer and fewer new defects. Vary and extend them (Labs 7, 8: generated and hostile inputs).
6. **Testing is context-dependent.** A BMS safety function and a radio menu deserve different rigour (ASIL levels, Chapter 3).
7. **Absence-of-errors fallacy.** Software that passes every test but implements the wrong requirement is still a failure (validation).

1.6 The vocabulary you will use daily

Term	Meaning
SUT / CUT	System / component <i>under test</i>
Test case	Inputs + preconditions + expected result (+ postconditions) for one objective
Test suite	A collection of test cases run together
Verdict	Outcome of one test: <i>pass</i> , <i>fail</i> (oracle violated), <i>error</i> (test itself crashed), <i>skip</i> , <i>expected failure</i>
Fixture	The prepared environment a test needs (objects, files, simulated hardware)
Test double	A stand-in for a real collaborator (stub, fake, spy, mock) — Chapter 7
Coverage	How much of the code (or requirements) the tests exercised — Chapter 15
Regression	Previously working behaviour that breaks after a change
Flaky test	A test that sometimes passes and sometimes fails with no code change
Smoke test	A tiny, fast check that the system is basically alive

1.7 How this course works

The book alternates **lecture chapters** (concepts, techniques, worked examples) with **labs** (runnable code and exercises). Every lab tests the *same* automotive library, `src/autotest`, so you spend your effort on technique rather than on learning new code:

```
CAN codec → sensors → BMS → cruise control → ABS → diagnostics
                                     |
ECUs on a virtual bus → tyre pressure (capstone) ←
```

You will use two frameworks: Python's built-in **unittest** (Chapter 4) and the community standard **pytest** (Chapter 5). Neither is "better" in every situation; you will learn both well enough to choose.

A tip that will serve you throughout: **read every test as a specification and every failing test as a question** — *which behaviour did I break, and was this test right to care?*

Check your understanding

1. A temperature sensor driver returns `NaN` after a wiring fault, and a charging function lets 100 A flow. Name the error, the fault and the failure.
2. Is checking that `max_charge_current(50, 25) == 100.0` verification or validation? What question would turn it into validation?
3. Why is "we ran 10 000 tests and all passed" not evidence that the software is correct?
4. Give one reason automotive projects rely more heavily on automated regression tests than a typical web application.

Black box, white box, grey box

- **Black-box** (specification-based): tests derive from the *requirements*, ignoring the code. They survive refactoring. Techniques: equivalence classes, boundary values, decision tables (Chapter 6).
- **White-box** (structure-based): tests derive from *code structure* — statements, branches, conditions. They measure completeness. (Chapter 15.)
- **Grey-box**: mostly black-box, but using knowledge of internals to craft inputs (e.g. knowing the alive counter wraps at 16).

Good practice combines them: design black-box from the spec, then use white-box *coverage* to find what you forgot.

2.3 Static and dynamic verification

Static techniques analyse without running: code review, linting, type checking, formal analysis. **Dynamic** techniques *execute* the software — that is testing proper. They complement each other; this book is dynamic testing, but a real pipeline also runs `ruff` / `mypy` / MISRA-style checkers.

2.4 X-in-the-loop: where the code runs

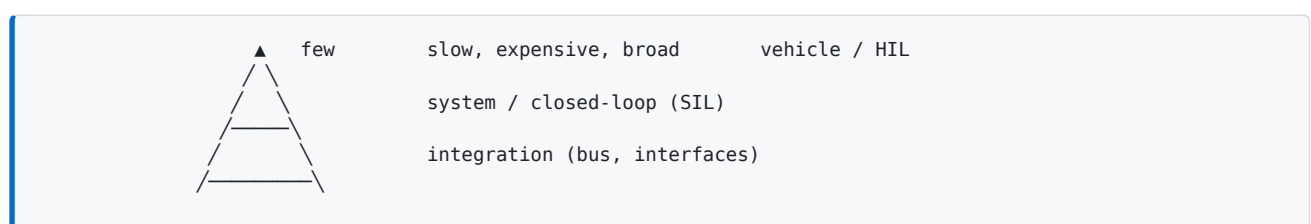
Embedded software is tested in progressively more realistic environments. The *same* test logic is reused as the environment changes:

Stage	Software	Plant (vehicle/hardware)	Used for
MIL model-in-the-loop	Simulink/Python <i>model</i>	simulated	early algorithm design
SIL software-in-the-loop	production <i>source code</i> on a PC	simulated	Labs 11–13 : fast, repeatable, CI-friendly
PIL processor-in-the-loop	compiled code on target CPU	simulated	numeric / timing effects of the real compiler and CPU
HIL hardware-in-the-loop	real ECU	real-time simulator + fault insertion	final ECU verification, safe fault injection
VIL / vehicle	real ECU	real car or proving ground	validation

Each step is slower, costlier and less reproducible — but closer to reality. A practical strategy pushes as many checks as possible *left* (cheap, early) and reserves the right for what only reality can show. Lab 13 demonstrates the technique that makes this work: **one test body, interchangeable back-ends**.

2.5 The test pyramid

Because cost and fragility grow with the scope of a test, healthy suites are shaped like a pyramid:





many fast, cheap, precise unit tests

- **Many unit tests** — milliseconds each, pinpoint the failing function.
- **Fewer integration tests** — seconds, verify interfaces.
- **Few system/HIL tests** — minutes to hours, verify end-to-end behaviour.

The anti-pattern is the **ice-cream cone**: a few unit tests and a mountain of slow manual or end-to-end tests. Failures are then hard to localise, runs are slow, and people stop running them.

2.6 Properties of a good test: F.I.R.S.T.

Letter	Property	In practice
Fast	Milliseconds	no <code>sleep</code> , no network, simulated hardware
Independent	Any order, alone	fresh fixtures, no shared mutable state
Repeatable	Same result every run	inject clocks, seed random generators
Self-validating	Pass/fail without reading logs	assertions, not <code>print</code>
Timely	Written with (or before) the code	bug → failing test → fix

2.7 Anatomy: Arrange – Act – Assert

Nearly every test, in any framework, has the same three phases (also written *Given – When – Then*):

```
from autotest.bms import max_charge_current

def test_charge_current_tapers_at_90_percent():
    # Arrange: a pack at 90 % SOC and 25 °C (here: just the inputs)
    soc, temp = 90, 25
    # Act: call the unit under test once
    current = max_charge_current(soc, temp)
    # Assert: compare with the value the SPECIFICATION gives (not the code's output!)
    assert current == 50.0

test_charge_current_tapers_at_90_percent()
print("ok")
```

Two habits follow: **one reason to fail per test** (so the name tells you what broke), and **derive the expected value from the specification**, never by running the code and pasting its output — that would merely freeze whatever bug exists.

2.8 Choosing a test strategy

A test *strategy* answers: *what will we test, at which level, with which technique, how will we know it is enough, and how often will we run it?* A compact decision guide:

Risk / situation	Reach for
Pure calculation with clear rules	unit tests + boundary values (Labs 2–3)
Logic with modes and history	state-transition tests (Lab 6)

Risk / situation	Reach for
Code that talks to hardware or other ECUs	test doubles, then integration tests (Labs 4–5)
Parsers, codecs, anything with big input spaces	property-based + fuzz (Labs 7–8)
Behaviour under failure	fault injection (Lab 8)
"It worked last release"	regression suite + golden logs (Lab 9)
Real-time or resource limits	budget tests (Lab 10)
Control loops	simulation with acceptance criteria (Lab 11)
"Are the tests any good?"	coverage + mutation (Lab 12)
Audit / safety case	traceability (Lab 13)

Check your understanding

- Place each on the V-model: (a) checking that CRC-8 of "123456789" is 0x4B ; (b) checking that the cluster shows -- after 100 ms of bus silence; (c) braking from 100 km/h in a simulation and measuring distance.
- Why is a suite with 5 unit tests and 200 end-to-end UI tests considered unhealthy?
- What is the difference between SIL and HIL, and why would you still run SIL tests when HIL exists?
- The assertion `assert max_charge_current(90, 25) == max_charge_current(90, 25)` always passes. Which F.I.R.S.T. or oracle principle does it violate?

Chapter 3 — Automotive primer: the domain you will test

You do not need to be a vehicle engineer to complete this course, but you do need a working vocabulary. Each section below introduces a domain concept exactly as the code in `src/autotest` models it.

3.1 ECUs and in-vehicle networks

A vehicle is a network of **electronic control units (ECUs)** — small computers dedicated to engine, brakes, battery, instrument cluster, and so on. They exchange *signals* (vehicle speed, battery temperature, pedal position) over **buses**. The workhorse is the **CAN bus** (Controller Area Network).

CAN in five facts

1. A **frame** carries an **arbitration identifier** (11 bits in standard frames, 29 in extended) and up to **8 data bytes** (classic CAN; CAN FD allows 64).
2. The identifier is *not* an address — it names the **message** and sets its **priority**: when two nodes transmit simultaneously, the lower identifier wins arbitration without data loss.
3. Any node can listen to any frame; receivers *filter* by identifier.
4. Typical speed is 500 kbit/s. A full 8-byte frame is 111 bit-times including intermission before bit stuffing, so a saturated bus carries roughly 4 500 frames/s at most (fewer after stuffing). Lab 10 uses this number as a throughput budget.
5. Hardware checks a 15-bit CRC, but *application-level* protection (below) is still needed for safety-critical signals.

```
from autotest.can import CANFrame

frame = CANFrame(arbitration_id=0x1A0, data=bytes([0x10, 0x27, 0x01]))
print(f"id=0x{frame.arbitration_id:03X} dlc={frame.dlc} data={frame.data.hex(' ')}")
try:
    CANFrame(0x800)          # 12 bits do not fit a standard identifier
except ValueError as e:
    print("rejected:", e)
```

Signals: physical values packed into bits

Engineers do not send "100.0 km/h"; they send an integer **raw value** in a bit field and agree a **scaling**:

```
physical = raw × factor + offset
```

A message definition lists each signal's *start bit*, *length*, *factor* and *offset* (in practice in a "DBC" file). Example from the wheel-speed message: a 16-bit raw value with factor 0.01 represents 0.00...655.35 km/h with 0.01 km/h resolution — and quantisation means a round trip is accurate only *to within one step* (the point of Lab 7's round-trip property).

```
from autotest.can import Signal

speed = Signal("speed_kmh", start_bit=0, length=16, factor=0.01, minimum=0, maximum=300)
raw = speed.to_raw(100.0)
```

```
print(raw, speed.to_physical(raw)) # 10000 100.0
print(speed.to_raw(0.004))       # 0 -> resolution is 0.01
```

End-to-end (E2E) protection

Safety-related signals travel with extra protection so a receiver can detect corruption, loss, repetition, or a stuck sender. Our model uses the common pattern from AUTOSAR E2E:

Mechanism	Detects	In <code>autotest.can</code>
CRC-8 (SAE J1850 polynomial 0x1D)	corrupted bits	<code>crc8</code> , first byte
Alive counter (4 bit, wraps 15 → 0)	lost, repeated or stale frames	second byte
Timeout	sender silent	cluster blanks after 100 ms

On any failure the receiver must not use the data and typically falls back to a safe value (the cluster shows `-`) and records a diagnostic code.

3.2 Diagnostics: DTCs and UDS

When an ECU detects a fault it stores a **diagnostic trouble code (DTC)**, e.g. `U0121` (lost communication with the ABS module; the letter gives the domain: **P**owertrain, **C**hassis, **B**ody, **U** (network / communication)). To avoid false alarms, a fault must usually persist for several monitoring cycles before the DTC is *confirmed* — **debouncing**. After enough healthy cycles the DTC stops being *active* but stays *stored* as history until a technician clears it.

A workshop tester talks to the ECU using **UDS** (Unified Diagnostic Services, ISO 14229). Requests start with a service id; a **positive response** adds `0x40` to it; a **negative response** is `0x7F <service> <code>`:

Service	Request	Positive	Our <code>DiagnosticManager</code>
ReadDTCInformation	<code>19 02 <mask></code>	<code>59 02 ...</code>	stored DTCs
ClearDiagnosticInformation	<code>14 ...</code>	<code>54</code>	clears history
ReadDataByIdentifier	<code>22 F1 90</code>	<code>62 F1 90 <VIN></code>	reads the VIN
anything else	—	<code>7F <svc> 11</code>	<i>service not supported</i>

Because the diagnostic port is reachable by anyone with an OBD dongle, its parser is a classic target for **robustness testing and fuzzing** (Lab 8).

3.3 Battery management (BMS)

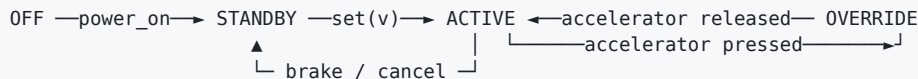
An electric vehicle's traction battery is built from many **cells** (typically 3.0–4.2 V each for lithium-ion). The **battery management system** protects and monitors them:

- **State of charge (SOC)** — percent of capacity remaining. **Coulomb counting** integrates current: $\Delta SOC = I \cdot \Delta t / \text{capacity}$. Charging current is positive.
- **Cell protection** — a cell above ~4.2 V (over-voltage) or below ~3.0 V (under-voltage), or the pack outside a safe temperature, must raise a fault.

- **Charge derating** — allowed charge current depends on temperature and SOC: no charging below 0 °C (lithium plating) or above 45 °C, reduced current when cold, and a taper to zero between 80 % and 100 % SOC. This is the decision table you will design tests for in Lab 3.

3.4 Cruise control

The cruise controller is a **mode state machine** plus a **control law**:



The control law is a **PI controller** (proportional–integral): $\text{throttle} = K_p \cdot \text{error} + K_i \cdot \text{error}$. The proportional term reacts to the present error; the integral term removes the steady-state error (e.g. on a hill). A real implementation needs **anti-windup** — stop integrating while the actuator is saturated — or the controller overshoots badly after a long saturation. Closed-loop quality is expressed with measures such as **overshoot**, **settling time** and **steady-state error** (Lab 11).

3.5 Anti-lock braking (ABS)

When a wheel brakes harder than the tyre can grip it locks, and a locked wheel cannot steer. Tyre force depends on **slip ratio**:

$$\text{slip} = (\text{vehicle_speed} - \text{wheel_speed}) / \text{vehicle_speed} \quad 0 = \text{free rolling}, 1 = \text{locked}$$

Friction rises with slip up to a **peak at roughly 10–30 % slip** and falls off toward lock-up (our simplified model peaks at 20 %). ABS rapidly **applies, holds and releases** brake pressure to keep slip near the peak — shortest stopping distance *and* steering ability. Below a minimum speed ABS stays out of the way.

3.6 Tyre pressure monitoring (TPMS)

Tyre pressure rises with temperature (constant volume: p/T is constant, with T in kelvin). To compare readings fairly the system **normalises to 20 °C**: $p_{20} = p \cdot 293.15 / (273.15 + T)$. Thresholds relative to nominal pressure define OK / LOW / CRITICAL / HIGH, and a rapid drop within a window signals a leak. This is the capstone's system under test.

3.7 Functional safety in one page: ISO 26262

ISO 26262 is the functional-safety standard for road vehicles. The ideas that matter for testers:

- **Hazard analysis and risk assessment** rates each hazardous event by *severity*, *exposure* and *controllability*, producing an **ASIL** — *Automotive Safety Integrity Level* — from **A** (lowest) to **D** (highest), or *QM* (quality management only).
- The **higher the ASIL, the more rigorous the required verification**: more methods, stricter coverage metrics, more independence.
- **Part 6** covers product development at the *software level*: unit design and implementation, **unit verification, integration and verification**, and testing of the embedded software. The methods it names — requirements-based tests, **interface tests**, **fault-injection tests**, **resource-usage evaluation**,

equivalence classes, boundary values, error guessing, structural coverage (statement, branch, MC/DC) and back-to-back comparison — are precisely the techniques of Labs 3–13.

- Evidence matters: the safety case needs **traceability** from safety requirement to test to result (Lab 13).

A related process standard, **Automotive SPICE**, expects the same discipline in its software verification processes (unit verification, component/integration verification, software qualification test).

This course teaches the techniques these standards ask for. It is not a compliance course — consult the edition of the standard your organisation follows, and your safety manager, for obligations on a real programme.

Check your understanding

1. Why does a CAN receiver need an alive counter *in addition to* a CRC?
2. A signal has factor 0.25, offset -40, and raw value 130. What is the physical value? What is the smallest change in physical value the signal can represent?
3. In a DTC with `fail_threshold=3`, a monitor reports fail, fail, pass, fail, fail. Is the code confirmed? Why does debouncing exist?
4. Why does the slip curve make "brake as hard as possible" a poor strategy on a real road?

Part II — The tools: unittest and pytest

Lab 0 — Setup and your very first tests

Duration: 30–45 min · **Goal:** a working environment, and a feel for how every lab works.

No lecture precedes this lab. If you can run `python course.py doctor` successfully, you are ready for Chapter 1.

1. Install (once)

You need **Python 3.10 or newer** (check with `python --version`; on some systems the command is `python3`).

```
# in the repository folder
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python course.py doctor

# a private Python environment for this course
# Windows: .venv\Scripts\activate
# pytest, coverage, hypothesis + the course library
# checks everything and runs your first tests
```

`doctor` prints `[OK]` for each check, or `[!!!]` with the exact fix. Nothing else in this course needs `make`, `PYTHONPATH` or a special IDE.

2. How the course is organised

What	Where	You do
Lectures	the PDF book, or <code>course/lecture_NN_*.md</code>	read first (30–60 min each)
Worked examples	<code>labs/labNN_*/test_*.py</code>	run them, read them, <i>break</i> them
Exercises	<code>labs/labNN_*/exercise_labNN.py</code>	write the missing tests
Code under test	<code>src/autotest/</code>	read it <i>after</i> designing tests from the spec

The commands you will use all the time:

```
python course.py labs
python course.py lab 3
python course.py exercise 3
python course.py progress

# list of labs + your progress
# run the worked examples of Lab 3
# run YOUR exercises of Lab 3
# progress bars for every lab
```

3. Your first test, step by step

A test is a function that calls the code and **asserts** what must be true. Open `test_lab00_first_test.py` — it contains three small tests of a real function, `kmh_to_ms` (km/h → m/s):

```
def test_36_kmh_is_10_metres_per_second():
    assert kmh_to_ms(36) == pytest.approx(10.0)
```

Run it:

```
python course.py lab 0
```

You will see `PASSED` for each test. Now do the most important thing you will do all course — **break it**:

1. Open `src/autotest/sensors.py` and change `kmh / 3.6` to `kmh / 3.7` in `kmh_to_ms`.
2. Run `python course.py lab 0` again. Tests **fail**, and pytest shows the two values it compared (look for the `E` lines).
3. Read the failure message, then change the code back.

A failing test with a clear message is the point of testing: it *tells you what broke*.

4. How exercises work

Every exercise test starts with a `todo(...)` line, so it **fails** until you start:

```
def test_72_kmh_is_20_metres_per_second():
    todo("call kmh_to_ms(72) and assert the result is 20")    # <- delete this line when you start
```

1. Read the docstring (it has the goal and **hints**; read them one at a time).
2. **Delete the `todo(...)` line** and write your test.
3. Run `python course.py exercise 0`. When all tests pass, run `python course.py progress`.

If you get stuck: read the hints, re-read the lecture, run the worked example of the same lab, and only then ask.

5. Exercise (in `exercise_lab00.py`)

Write the two tests described there. They take five minutes and use only `assert` and `pytest.raises`.

Debrief questions

- What does `pytest.approx` do, and why not `==` for floats?
- When a test fails, which two numbers does pytest show you?
- Why is it useful to *break the code on purpose* and watch a test fail?

Worked example — source

labs/lab00_setup/test_lab00_first_test.py

```
"""Lab 0 - three tiny tests. Run: python course.py lab 0

A test is just a function whose name starts with `test_`; `assert` is the check.
"""
import pytest

from autotest.sensors import kmh_to_ms, ms_to_kmh, wheel_speed_kmh
```

```

@pytest.mark.smoke
def test_36_kmh_is_10_metres_per_second():
    # Arrange + Act: call the code under test
    result = kmh_to_ms(36)
    # Assert: compare with the value from the SPECIFICATION (36 km/h = 10 m/s), not from the code
    assert result == pytest.approx(10.0)

def test_converting_there_and_back_gives_the_same_speed():
    assert ms_to_kmh(kmh_to_ms(50)) == pytest.approx(50)

def test_a_wheel_speed_of_zero_pulses_is_standstill():
    # 0 pulses in 1 second -> the wheel is not turning
    assert wheel_speed_kmh(pulses=0, pulses_per_rev=20, circumference_m=2.0, interval_s=1.0) == 0

```

Exercise skeleton — source

labs/lab00_setup/exercise_lab00.py

```

"""Lab 0 exercises - your very first tests.

Run it:      python course.py exercise 0
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
               test; the test then passes only if what you wrote is correct.
Done when:   `python course.py progress` shows every test of Lab 0 passing.
"""
import pytest

from autotest.learn import todo
from autotest.sensors import kmh_to_ms, wheel_speed_kmh

def test_72_kmh_is_20_metres_per_second():
    """Goal: check that kmh_to_ms(72) is 20 m/s.

    Hint 1: call the function and keep the result:  result = kmh_to_ms(72)
    Hint 2: compare it:  assert result == pytest.approx(20.0)
    """
    todo("call kmh_to_ms(72) and assert the result is 20")

def test_an_interval_of_zero_seconds_is_rejected():
    """Goal: wheel_speed_kmh must refuse an interval of 0 seconds (you cannot divide by zero time).

    Hint 1: a function that must raise an error is tested with a `with pytest.raises(...)` block.
    Hint 2: the error type is ValueError.
    Hint 3: with pytest.raises(ValueError):
               wheel_speed_kmh(pulses=10, pulses_per_rev=20, circumference_m=2.0, interval_s=0)
    """
    todo("assert that interval_s=0 raises ValueError")

```

Chapter 4 — unittest: the standard library's test framework

4.1 Where it comes from

`unittest` is Python's built-in member of the **xUnit** family, which began with Kent Beck's *SUnit* for Smalltalk and spread through JUnit (Java), NUnit (.NET) and PyUnit — Python's original name for it. Every xUnit framework shares the same vocabulary: *test case*, *fixture*, *suite*, *runner*, *assertion*. Learn it once and JUnit, GoogleTest and Unity/CppUTest (common in embedded C) will feel familiar.

Why start here?

- It ships with Python — no installation, which matters on locked-down build servers and in certified toolchains.
- It teaches the **concepts** (fixtures, lifecycle, isolation) explicitly, because you must write them out.
- `pytest` runs `unittest` tests unchanged, so nothing you write here is wasted.

4.2 The five core concepts

Concept	Class	Role
Test case	<code>unittest.TestCase</code>	A class whose <code>test_*</code> methods are individual tests
Fixture	<code>setUp</code> / <code>tearDown</code> ...	Code that prepares and cleans the environment
Test suite	<code>unittest.TestSuite</code>	A collection of tests or other suites
Test loader	<code>unittest.TestLoader</code>	Finds tests (by module, class or directory) and builds suites
Test runner	<code>TextTestRunner</code>	Executes a suite and reports a <code>TestResult</code>

Most of the time you only write test cases; the command line does the rest.

4.3 Your first test case

```
import unittest
from autotest.sensors import kmh_to_ms

class TestSpeedConversion(unittest.TestCase):
    def test_36_kmh_is_10_ms(self):
        self.assertAlmostEqual(kmh_to_ms(36.0), 10.0)

    def test_zero_stays_zero(self):
        self.assertEqual(kmh_to_ms(0), 0)

suite = unittest.defaultTestLoader.loadTestsFromTestCase(TestSpeedConversion)
result = unittest.TextTestRunner(verbosity=0).run(suite)
assert result.wasSuccessful() and result.testsRun == 2
```

Rules the framework applies:

- The class must inherit from `unittest.TestCase`.
- Only methods whose names **start with** `test` are run.

- Each test method runs on a **fresh instance** of the class — instance attributes set in one test are *not* visible in another. This is the foundation of test independence.
- A test **passes** if it returns normally, **fails** if an assertion raises `AssertionError`, and is an **error** if any other exception escapes (the test itself is broken or the code crashed unexpectedly).

4.4 The fixture lifecycle

Fixtures are the answer to: "how do I prepare the same starting situation for many tests without copy-pasting?" `unittest` provides hooks at three scopes:

<code>setUpModule()</code>	once per module	
<code>setUpClass()</code>	once per class	(a <code>@classmethod</code>)
<code>setUp()</code>	before EVERY test method	
<code>test_a()</code>		
<code>tearDown()</code>	after EVERY test method (even if the test failed)	
<code>setUp()</code>		
<code>test_b()</code>		
<code>tearDown()</code>		
<code>tearDownClass()</code>		
<code>tearDownModule()</code>		

You can watch the order yourself:

```
import unittest

events = []

def setUpModule():    events.append("setUpModule")
def tearDownModule(): events.append("tearDownModule")

class Lifecycle(unittest.TestCase):
    @classmethod
    def setUpClass(cls):    events.append("setUpClass")
    @classmethod
    def tearDownClass(cls): events.append("tearDownClass")
    def setUp(self):        events.append("setUp")
    def tearDown(self):     events.append("tearDown")
    def test_a(self):       events.append("test_a")
    def test_b(self):       events.append("test_b")

unittest.main(argv=["prog"], exit=False, verbosity=0)
assert events == ["setUpModule", "setUpClass",
                  "setUp", "test_a", "tearDown",
                  "setUp", "test_b", "tearDown",
                  "tearDownClass", "tearDownModule"], events
```

Which hook when?

- `setUp` — default choice. Cheap objects built fresh per test guarantee isolation.
- `setUpClass` — only for *expensive* and *immutable* shared resources (e.g. loading a big signal database). If a test mutates it, you have created a hidden dependency between tests.
- `tearDown` runs even when `setUp` passed and the test failed; it does **not** run if `setUp` itself raised. For cleanup that must happen regardless, register it with `self.addCleanup(fn, *args)` right after you allocate the resource — cleanups run in reverse order and also run if `setUp` fails part-way.

4.5 Assertions: say exactly what you mean

A specific assertion produces a specific failure message. Prefer the most specific one:

Check	Method	Note
equality	<code>assertEqual(a, b)</code>	also compares lists, dicts, sets with a diff
inequality	<code>assertNotEqual(a, b)</code>	
floats	<code>assertAlmostEqual(a, b, places=7) / delta=</code>	never <code>assertEqual</code> on floats
boolean	<code>assertTrue(x) / assertFalse(x)</code>	weaker messages — use only for genuine booleans
identity / None	<code>assertIs</code> , <code>assertIsNone</code> , <code>assertIsNotNone</code>	<code>assertIs(result, BMSFault.NONE)</code> for enums
membership	<code>assertIn(a, container)</code>	
ordering	<code>assertGreater</code> , <code>assertLessEqual</code> , ...	
type	<code>assertIsInstance(obj, cls)</code>	
exceptions	<code>assertRaises(Exc) / assertRaisesRegex(Exc, pattern)</code>	context-manager form preferred
collections	<code>assertCountEqual(a, b)</code>	same elements, any order
text	<code>assertRegex(text, pattern)</code>	
explicit	<code>self.fail("message")</code>	for unreachable branches

Why `assertEqual(a, b)` beats `assertTrue(a == b)` : when it fails, the first prints both values; the second prints only `False is not true`.

Exceptions

```
import unittest
from autotest.sensors import wheel_speed_kmh

class T(unittest.TestCase):
    def test_zero_interval_is_rejected(self):
        with self.assertRaises(ValueError):
            wheel_speed_kmh(10, 20, 2.0, 0)

    def test_message_names_the_problem(self):
        with self.assertRaisesRegex(ValueError, "invalid wheel speed"):
            wheel_speed_kmh(-1, 20, 2.0, 1.0)

r = unittest.TextTestRunner(verbosity=0).run(unittest.defaultTestLoader.loadTestsFromTestCase(T))
assert r.wasSuccessful()
```

Do **not** write `try: ... except ValueError: pass` — if the code raises *nothing*, the test silently passes. `assertRaises` fails in that case.

Floating point

Binary floating point cannot represent most decimals, so `0.1 + 0.2 != 0.3`. Compare with a tolerance: `assertAlmostEqual(a, b, places=7)` rounds the *difference* to `places` decimals; `delta=0.01` states an absolute tolerance directly.

4.6 One test, many inputs: `subTest`

When the same check applies to several inputs, loop inside the test but wrap each iteration in `subTest`. A failing iteration is reported with its parameters and the loop **continues** (a bare loop would stop at the first failure):

```
import unittest
from autotest.sensors import kmh_to_ms, ms_to_kmh

class T(unittest.TestCase):
    def test_round_trip(self):
        for kmh in (0, 1, 50, 130.5):
            with self.subTest(kmh=kmh):
                self.assertAlmostEqual(ms_to_kmh(kmh_to_ms(kmh)), kmh)

r = unittest.TextTestRunner(verbosity=0).run(unittest.defaultTestLoader.loadTestsFromTestCase(T))
assert r.wasSuccessful()
```

4.7 Skipping and known failures

Never delete or comment out a test you cannot make pass — make its status *visible*:

Decorator	Meaning	Shown in the report as
<code>@unittest.skip("reason")</code>	not run	skipped (with reason)
<code>@unittest.skipIf(cond, "reason")</code> / <code>skipUnless</code>	conditional skip (platform, optional dependency)	skipped
<code>@unittest.expectedFailure</code>	a <i>known</i> bug: must fail	"expected failure"; becomes an unexpected success (a failure of the run) when someone fixes the bug, forcing cleanup

4.8 Discovery and the command line

```
python -m unittest           # discover tests in ./ (files test*.py)
python -m unittest discover -s tests -p "test_*.py" # explicit directory and pattern
python -m unittest tests.test_can # a module
python -m unittest tests.test_can.TestCRC # a class
python -m unittest tests.test_can.TestCRC.test_check_value # one method
python -m unittest -v        # verbose, one line per test
python -m unittest -k crc    # only tests whose name contains "crc"
python -m unittest -f        # stop at first failure
python -m unittest -b        # buffer stdout/stderr, show only for failures
```

The **exit code** is 0 only if everything passed — which is what a CI server looks at.

4.9 A first look at `unittest.mock`

`unittest.mock` (also in the standard library) replaces collaborators with controllable objects — central to Chapter 7. The essentials:

```
from unittest import mock
```



```

adc = mock.Mock()
adc.read.return_value = 2048           # a stub: canned answer
assert adc.read() == 2048

adc.read.side_effect = [1, 2, OSError("bus error")] # a sequence, then an exception
assert (adc.read(), adc.read()) == (1, 2)
try:
    adc.read()
except OSError:
    pass
assert adc.read.call_count == 4        # a spy: how often was it called?

with mock.patch("autotest.sensors.kmh_to_ms", return_value=123.0):
    from autotest import sensors
    assert sensors.kmh_to_ms(1) == 123.0 # replaced only inside the with-block

```

4.10 Strengths, limits, and when to choose it

Strengths	Limits
in the standard library; stable for decades	boilerplate: classes, <code>self.assert*</code> zoo
explicit, familiar xUnit lifecycle	no parametrization (only <code>subTest</code>), no fixture injection
good fit for restricted environments and as a teaching tool	plain output for failures; fewer plugins

Choose `unittest` when you must avoid third-party dependencies or integrate with a house xUnit style.

Choose `pytest` (next chapter) for new projects. You can mix: `pytest` happily runs `TestCase` classes, so a codebase can migrate gradually.

Check your understanding

1. A test class creates a battery object in `setUpClass` and one test charges it. What can go wrong in the other tests?
2. Why does `with self.assertRaises(ValueError):` appear in every serious test suite rather than `try/except`?
3. What is the difference between a test **failure** and a test **error**?
4. You have a known bug you cannot fix this sprint. Compare `@unittest.skip` and `@unittest.expectedFailure`. Which is better and why?

Lab 1 — Unit testing with `unittest`

Duration: 2.5 h · **Type:** unit testing · **Code under test:** `can.py` , `sensors.py` , `bms.py`

Why it matters in a vehicle programme

A modern car runs 100M+ lines of code; the cheapest place to find a defect is the unit that contains it. A wrong CAN scale factor or a CRC off-by-one is invisible on the bench but costs a recall in the field. `unittest` ships with Python, so it works in locked-down toolchains where installing packages is hard.

Learning objectives

After this lab you can: write a `TestCase` ; choose the right `assert*` method; test exceptions and floating-point results; share set-up safely (`setUp` , `setUpClass`); run many inputs with `subTest` ; document skipped and known-failing tests; run tests from the CLI, as a hand-built suite, or through `pytest`.

Concepts (10 min)

- **Anatomy:** Arrange → Act → Assert. A unit test fails for exactly one reason.
- **Isolation:** `setUp` runs before *every* test, so tests cannot depend on each other.
- **`subTest`** : reports each failing input separately instead of stopping at the first.
- **Never `assertEqual` floats** — use `assertAlmostEqual` .
- **`expectedFailure` / `skip`** make known problems visible instead of deleting tests.

Run it

```
python -m unittest discover -s labs/lab01_unittest -p "test_lab01*.py" -v      # (needs `pip install`  
-r requirements.txt`; otherwise prefix with PYTHONPATH=src)  
pytest labs/lab01_unittest -v          # same tests, pytest runner
```

Guided tour of `test_lab01_unittest.py`

Class	Look for
<code>TestUnitConversions</code>	<code>subTest</code> , <code>assertAlmostEqual</code> , <code>assertRaisesRegex</code> — also the hand-computed expected value (14.4 km/h)
<code>TestCrc8</code>	A <i>catalogue check value</i> (0x4B) is the best oracle you can get; single-bit-flip loop proves CRC strength
<code>TestCANFrame</code>	<code>setUp</code> / <code>tearDown</code> ; frozen dataclass; boundary 0x7FF/0x800
<code>TestSignalScaling</code>	<code>setUpClass</code> , <code>@skip</code> , <code>@expectedFailure</code> documenting a known resolution defect
<code>load_tests</code>	building a suite by hand

Try this (5 min): in `can.py` change `& 0xFF` in `crc8` to `& 0x7F` . Which tests fail, and which do *not*? What does that tell you about the suite?

Exercises — exercise_lab01.py

Run: `python course.py exercise 1` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

1. Coulomb counting in `BatteryManagementSystem.update_soc`: clamping, discharge, invalid input.
2. Encode/decode a two-signal engine message — work out the expected bytes **by hand first**.
3. E2E protection: round trip, corruption, counter wrap.

Debrief questions

- Why is `assertTrue(a == b)` a worse choice than `assertEqual(a, b)`?
- A test uses `setUpClass` to create a battery object and mutates it. What can go wrong?
- When is an `expectedFailure` better than a skipped test? Than a deleted test?

Common pitfalls

Tests that depend on execution order · asserting on `repr` /exception text that may legitimately change · catching exceptions manually (`try/except` hides "no exception" passing silently) · testing the implementation instead of behaviour.

Worked example — source

labs/lab01_unittest/test_lab01_unittest.py

```
"""Lab 1 - unit testing with the standard library's unittest.

Run:  python -m unittest discover -s labs/lab01_unittest -p "test_lab01*.py" -v
      or:  pytest labs/lab01_unittest -v          (pytest runs unittest tests too)
"""
import dataclasses
import unittest

from autotest.can import CANFrame, Signal, crc8
from autotest.sensors import celsius_to_fahrenheit, kmh_to_ms, ms_to_kmh, wheel_speed_kmh


class TestUnitConversions(unittest.TestCase):
    """Anatomy of a test: Arrange - Act - Assert."""

    def test_kmh_to_ms_known_value(self):
        self.assertAlmostEqual(kmh_to_ms(36.0), 10.0)

    def test_round_trip(self):
        for kmh in (0, 1, 50, 130.5):
            with self.subTest(kmh=kmh): # one failure does not hide the others
                self.assertAlmostEqual(ms_to_kmh(kmh_to_ms(kmh)), kmh)

    def test_freezing_point(self):
        self.assertEqual(celsius_to_fahrenheit(0), 32)

    def test_wheel_speed_formula(self):
        # 40 pulses / 20 per rev = 2 rev in 1 s; 2 m circumference -> 4 m/s = 14.4 km/h
        self.assertAlmostEqual(wheel_speed_kmh(40, 20, 2.0, 1.0), 14.4)

    def test_invalid_interval_raises(self):
        with self.assertRaises(ValueError):
            wheel_speed_kmh(10, 20, 2.0, 0)

    def test_error_message_is_helpful(self):
```

```

        with self.assertRaisesRegex(ValueError, "invalid wheel speed"):
            wheel_speed_kmh(-1, 20, 2.0, 1.0)

class TestCrc8(unittest.TestCase):
    def test_standard_check_value(self):
        # CRC-8/SAE-J1850 catalogue check value for ASCII "123456789"
        self.assertEqual(crc8(b"123456789"), 0x4B)

    def test_detects_single_bit_flip(self):
        good = bytes([0x12, 0x34, 0x56])
        for bit in range(24):
            with self.subTest(bit=bit):
                bad = bytearray(good)
                bad[bit // 8] ^= 1 << (bit % 8)
                self.assertNotEqual(crc8(good), crc8(bytes(bad)))

class TestCANFrame(unittest.TestCase):
    """Fixtures: setUp runs before EVERY test, so tests never share state."""

    def setUp(self):
        self.frame = CANFrame(0x123, b"\x01\x02\x03")

    def tearDown(self):
        pass # release files / sockets here; nothing needed for pure objects

    def test_dlc_is_data_length(self):
        self.assertEqual(self.frame.dlc, 3)

    def test_frame_is_immutable(self):
        with self.assertRaises(dataclasses.FrozenInstanceError): # name the EXACT exception, not
Exception
            self.frame.arbitration_id = 0x7FF

    def test_standard_id_limit(self):
        CANFrame(0x7FF) # last legal id: must not raise
        with self.assertRaises(ValueError):
            CANFrame(0x800)

    def test_extended_id_accepts_29_bits(self):
        self.assertEqual(CANFrame(0x1FFFFFFF, is_extended=True).arbitration_id, 0x1FFFFFFF)

    def test_too_many_bytes(self):
        with self.assertRaises(ValueError):
            CANFrame(0x1, bytes(9))

class TestSignalScaling(unittest.TestCase):
    @classmethod
    def setUpClass(cls): # expensive shared setup, done once per class
        cls.rpm = Signal("rpm", 0, 16, factor=0.25, minimum=0, maximum=16000)

    def test_scaling(self):
        self.assertEqual(self.rpm.to_raw(3000), 12000)
        self.assertEqual(self.rpm.to_physical(12000), 3000)

    def test_out_of_range_rejected(self):
        with self.assertRaises(ValueError):
            self.rpm.to_raw(16001)

    @unittest.skip("demonstration: skipped tests are reported, never silently lost")
    def test_skipped(self):
        self.fail("never runs")

    @unittest.expectedFailure
    def test_known_bug_documented(self):
        # An expected failure documents a KNOWN defect; it turns red when the bug is fixed.
        self.assertEqual(self.rpm.to_raw(0.1), 1) # rounds to 0 - resolution is 0.25 rpm

    def load_tests(loader, tests, pattern):
        """Optional hook: build a suite by hand (see README, section 'Suites and runners')."""

```

```

suite = unittest.TestSuite()
for case in (TestUnitConversions, TestCrc8, TestCANFrame, TestSignalScaling):
    suite.addTests(loader.loadTestsFromTestCase(case))
return suite

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

Exercise skeleton — source

labs/lab01_unittest/exercise_lab01.py

```

"""Lab 1 exercises - unit tests with unittest.

Run it:      python course.py exercise 1
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
               test; the test then passes only if what you wrote is correct.
Done when:   `python course.py progress` shows every test of Lab 1 passing.
Stuck?       Read the hints in the docstring ONE AT A TIME; the last hint of some tests gives the
answer.
"""
import unittest

from autotest.bms import BatteryManagementSystem
from autotest.can import CANMessage, E2EStatus, Signal, e2e_check, e2e_protect
from autotest.learn import todo

class TestCoulombCounting(unittest.TestCase):
    """Exercise 1 - BatteryManagementSystem.update_soc(current_a, dt_s) integrates current into SOC.

    SOC change in percentage points = current * time / 3600 / capacity * 100    (capacity in Ah)
    """

    def setUp(self):
        self.bms = BatteryManagementSystem(capacity_ah=50.0, soc=50.0)    # a 50 Ah pack at 50 %

    def test_charging_is_clamped_at_100(self):
        """50 A for one hour would add +100 points (50 * 100 = 150) but SOC can never exceed 100.

        Hint 1: update_soc returns the new SOC:  new_soc = self.bms.update_soc(50, 3600)
        Hint 2: use self.assertEqual(new_soc, ...) with the value the SPECIFICATION demands.
        """
        todo("charge 50 A for 3600 s and expect SOC 100")

    def test_half_capacity_for_one_hour_adds_exactly_50_points(self):
        """Start from 0 %: 25 A for one hour on a 50 Ah pack is half the capacity.

        Hint 1: set up the start state first:  self.bms.soc = 0
        Hint 2: floats! use self.assertAlmostEqual(actual, expected)
        """
        todo("expect 50.0 after 25 A for 3600 s starting at 0 %")

    def test_discharge_never_goes_below_zero(self):
        """Hint: a large negative current (discharging), e.g. -50 A for 7200 s, must stop at 0 %."""
        todo("expect 0.0")

    def test_negative_dt_is_rejected(self):
        """Time cannot run backwards: update_soc(10, -1) must raise ValueError.

        Hint: with self.assertRaises(ValueError): <call goes here>
        """
        todo("use assertRaises")

class TestCanMessageCodec(unittest.TestCase):
    """Exercise 2 - a two-signal message. physical = raw * factor + offset; signals are packed
    little-endian.
    """

```

```

def setUp(self):
    self.msg = CANMessage(0x2F0, "Engine", 4, (
        Signal("rpm", 0, 16, factor=0.25),      # bits 0..15, 0.25 rpm per step
        Signal("coolant", 16, 8, offset=-40),    # bits 16..23, -40 degC offset
    ))

def test_encode_known_bytes(self):
    """Encode rpm=3000, coolant=90 and compare with the bytes you computed BY HAND first.

    Hint 1: raw = (physical - offset) / factor -> rpm raw = ? coolant raw = ?
    Hint 2: write the raw numbers in hex; little-endian puts the LOW byte first; the 4th byte is
padding (0).
    Hint 3 (spoiler): rpm 12000 = 0x2EE0, coolant 130 = 0x82 -> bytes([0xE0, 0x2E, 0x82,
0x00])

    Call: self.msg.encode({"rpm": 3000, "coolant": 90})
    """
    todo("compare encode(...) with your hand-computed bytes")

def test_round_trip_for_several_operating_points(self):
    """decode(encode(x)) must give x back. Use subTest so one failing point does not hide the
others.

    Hint 1: for rpm, coolant in [(0, -40), (800, 20), (6500.25, 90)]:
        with self.subTest(rpm=rpm, coolant=coolant): ...
    Hint 2: self.assertEqual(self.msg.decode(self.msg.encode(values)), values)
    """
    todo("round trip with subTest")

def test_unknown_signal_name(self):
    """encode() with a signal that is not in the message must raise an error.

    Hint: find out WHICH exception type by reading CANMessage.encode in src/autotest/can.py
(or try it in a Python shell), then use assertRaises with exactly that type.
    """
    todo("assertRaises for an unknown signal")

def test_decode_wrong_length(self):
    """decode() of a 2-byte payload for a 4-byte message must raise ValueError."""
    todo("assertRaises(ValueError)")

class TestE2E(unittest.TestCase):
    """Exercise 3 - end-to-end protection: e2e_protect(payload, counter) / e2e_check(data,
expected_counter)."""

    def test_protect_then_check_is_ok(self):
        """Hint: status, payload = e2e_check(e2e_protect(b"\x01\x02", 5), 5) -> status is
E2EStatus.OK, payload is b"\x01\x02"."""
        todo("round trip, check status AND payload")

    def test_corrupted_payload_is_detected(self):
        """Flip one bit of the last byte after protecting; the status must be E2EStatus.CRC_ERROR.

        Hint: protected = bytearray(e2e_protect(b"\x01\x02", 5)); protected[-1] ^= 0x01;
e2e_check(bytes(protected))
        """
        todo("flip a bit, expect CRC_ERROR")

    def test_counter_wraps_at_16(self):
        """The alive counter is 4 bits wide, so counter 16 behaves like counter 0.

        Hint: compare e2e_protect(b"x", 16) with e2e_protect(b"x", 0).
        """
        todo("counter 16 == counter 0")

if __name__ == "__main__":
    unittest.main()

```

Chapter 5 — pytest: concise, powerful tests

5.1 Why pytest

pytest is the de-facto standard for Python testing. It keeps the xUnit *ideas* (fixtures, isolation, discovery) but removes the ceremony:

- **plain `assert`** statements — pytest rewrites them to show exactly which values differed;
- **tests are functions** — classes are optional and need no base class;
- **fixtures** are injected by *name* instead of inherited, and compose freely;
- **parametrization** turns one test into a table of cases;
- **markers** categorise tests; **plugins** add coverage, property-based testing, parallel runs, reports.

It also runs `unittest.TestCase` suites unchanged, so adoption can be gradual.

5.2 Discovery and naming

By default pytest collects:

- files named `test_*.py` or `*_test.py`,
- inside them: functions named `test_*`, and methods named `test_*` of classes named `Test*` (no `__init__`).

```
pytest                # everything under the current directory (or configured testpaths)
pytest tests/test_can.py  # one file (also works for files that don't match the pattern!)
pytest tests/test_can.py::test_crc[check-value]  # one test, by node id
pytest -k "crc and not slow" # name expression
```

5.3 Assertions with introspection

```
import pytest
from autotest.bms import max_charge_current

def test_taper_at_90_percent():
    assert max_charge_current(90, 25) == pytest.approx(50.0)
```

When an assertion fails pytest shows the values it computed:

```
def test_taper_at_90_percent():
>     assert max_charge_current(90, 25) == 60.0
E       assert 50.0 == 60.0
E         + where 50.0 = max_charge_current(90, 25)
```

Floats: `pytest.approx`

`assert x == pytest.approx(y)` compares with a tolerance (default relative $1e-6$). Tune with `rel=` / `abs=`; it also works on lists and dicts.

Exceptions: `pytest.raises`

```
import pytest
from autotest.can import Signal

def test_value_above_maximum_is_rejected():
    sig = Signal("v", 0, 8, maximum=10)
    with pytest.raises(ValueError, match="above maximum") as info:
        sig.to_raw(11)
    assert "v" in str(info.value)      # inspect the exception afterwards
```

5.4 Fixtures: dependency injection for test set-up

A **fixture** is a function decorated with `@pytest.fixture`. A test (or another fixture) that lists its name as a parameter *receives its return value*:

```
import pytest
from autotest.bms import BatteryManagementSystem

@pytest.fixture
def battery():
    bms = BatteryManagementSystem(capacity_ah=50, soc=50)
    yield bms                                # --- test runs here ---
    assert 0 <= bms.soc <= 100                # teardown: runs after EVERY test, even a failed one

def test_charging_is_clamped(battery):
    battery.update_soc(1000, 3600)
    assert battery.soc == 100

def test_discharging_is_clamped(battery):
    battery.update_soc(-1000, 3600)
    assert battery.soc == 0
```

Key mechanics:

- **Setup / teardown in one function:** code before `yield` is setup; after `yield` is teardown (it runs even when the test fails).
- **Composition:** a fixture may request other fixtures — pytest builds the dependency graph.
- **Fresh by default:** a function-scoped fixture is created per test, giving isolation without a base class.

Scopes

`@pytest.fixture(scope=...)` controls how often the fixture is created:

Scope	Created	Use for
function (default)	per test	anything mutable
class / module	once per class / file	expensive but immutable objects
session	once per run	e.g. a loaded signal database

Wider scope = faster but riskier: if a test mutates a shared fixture, later tests see the mutation.

Factories, autouse, conftest, parameters

```
import pytest
from autotest.sensors import TemperatureSensor

class ScriptedADC:
    def __init__(self, *values): self.values = list(values)
    def read(self): return self.values.pop(0)

@pytest.fixture
def sensor_factory():
    # "factory as fixture": the test chooses the data
    def make(*counts, filter_len=4):
        return TemperatureSensor(ScriptedADC(*counts), filter_len=filter_len)
    return make

def test_filter_averages(sensor_factory):
    s = sensor_factory(1024, 2048, 3072)
    for _ in range(3):
        last = s.read_filtered()
    assert last == pytest.approx(TemperatureSensor.T_MIN + 190 * 2048 / 4095)

@pytest.fixture(params=[0, 4095], ids=["short-to-ground", "open-circuit"])
def rail_value(request):
    # a parametrized fixture: every test using it runs twice
    return request.param

def test_rails_are_faults(rail_value, sensor_factory):
    from autotest.sensors import SensorFault
    with pytest.raises(SensorFault):
        sensor_factory(rail_value).read()
```

- `autouse=True` makes a fixture run for every test in its scope without being requested (use sparingly: it is invisible at the call site).
- Fixtures placed in a file called `conftest.py` are available to all tests in that directory tree without imports — our course's shared `clock` and `bus` fixtures live in the root `conftest.py`.
- `pytest --setup-show` prints exactly when each fixture is set up and torn down — the quickest way to understand scopes.

5.5 Parametrization: tables of cases

```
import pytest
from autotest.bms import BatteryManagementSystem, BMSFault

@pytest.mark.parametrize(
    "cells, temp, expected",
    [
        ([3.7, 3.7], 25, BMSFault.NONE),
        ([3.7, 4.21], 25, BMSFault.OVERVOLTAGE),
        ([3.7, 2.99], 25, BMSFault.UNDERVOLTAGE),
        ([3.7, 3.7], 61, BMSFault.OVERTEMPERATURE),
    ],
    ids=["healthy", "overvolt", "undervolt", "overtemp"],
)
def test_fault_detection(cells, temp, expected):
    assert BatteryManagementSystem().evaluate(cells, temp) is expected
```

Each row is reported as its own test (`test_fault_detection[overvolt]`), so a failure names the case. Additional features:

- **Stacking** two `parametrize` decorators produces the *cartesian product* ($3 \times 4 = 12$ tests).
- `pytest.param(value, id="...", marks=pytest.mark.xfail(...))` attaches a marker or id to a single row.

- `indirect=True` routes a parameter through a fixture of the same name (useful to build objects from parameters).

Parametrization is how this course writes boundary-value and decision-table tests: **the table is the test design**, readable by a reviewer who has never seen Python.

5.6 Markers: classifying tests

```
import pytest

@pytest.mark.smoke
def test_fast_sanity():
    assert 1 + 1 == 2

@pytest.mark.skipif(not hasattr(pytest, "approx"), reason="needs approx")
def test_conditional():
    assert True

@pytest.mark.xfail(reason="BUG-114: known rounding limitation", strict=True)
def test_known_bug():
    assert round(0.1 / 0.25) == 1      # rounds to 0 -> fails -> "expected failure"
```

- Select with `pytest -m smoke` or `pytest -m "not slow and not performance"`.
- **Register markers** (in `pyproject.toml`) and run with `--strict-markers` so a typo like `@pytest.mark.smok` is an error instead of a silently ignored label.
- `xfail(strict=True)` means "this *must* fail": if the bug gets fixed the run goes red, forcing someone to delete the marker. Non-strict `xfail` quietly turns into a pass.

5.7 Built-in fixtures you will use constantly

Fixture	Gives you
<code>tmp_path</code>	a unique empty <code>pathlib.Path</code> directory per test
<code>monkeypatch</code>	safe, auto-undone <code>setattr</code> , <code>setenv</code> , <code>chdir</code> , <code>delattr</code>
<code>capsys</code> / <code>capfd</code>	captured stdout/stderr (<code>capsys.readouterr()</code>)
<code>caplog</code>	captured log records
<code>request</code>	introspect the requesting test (params, markers, config)
<code>pytestconfig</code>	access command-line options

```
import os

def test_tmp_path(tmp_path):
    f = tmp_path / "trace.asc"
    f.write_text("0.001 1 123 Rx d 3 01 02 03\n")
    assert f.read_text().startswith("0.001")

def test_monkeypatch(monkeypatch):
    monkeypatch.setenv("VEHICLE_VARIANT", "EV")
    assert os.environ["VEHICLE_VARIANT"] == "EV"      # restored automatically afterwards

def test_capsys(capsys):
    print("DTC U0121 set")
    assert "U0121" in capsys.readouterr().out
```

5.8 Configuration and options

Project-wide settings live in `pyproject.toml` (or `pytest.ini`). This course uses:

```
[tool.pytest.ini_options]
pythonpath = ["src", "."]
testpaths = ["labs", "solutions"]
addopts = "-ra --strict-markers --import-mode=importlib"
markers = ["smoke: fast sanity checks", "integration: ...", "slow: ..."]
```

Daily commands:

Command	Purpose
<code>pytest -x / --lf / --ff</code>	stop at first failure / rerun last failures / failures first
<code>pytest -vv --tb=short</code>	verbose output, compact tracebacks
<code>pytest -rs -rx</code>	show reasons for skips and expected failures
<code>pytest --durations=10</code>	the ten slowest tests
<code>pytest --pdb</code>	open the debugger at a failure
<code>pytest --junitxml=report.xml</code>	machine-readable report for CI
<code>pytest --cov=pkg --cov-branch</code>	coverage (plugin <code>pytest-cov</code>)

Useful plugins: `pytest-cov` (coverage), `hypothesis` (Chapter 10), `pytest-xdist` (parallel), `pytest-timeout`, `pytest-mock` (mock fixture), `pytest-bdd` (Gherkin scenarios), `pytest-html` (reports).

5.9 unittest or pytest?

	unittest	pytest
Assertion style	<code>self.assertEqual(a, b)</code>	<code>assert a == b</code>
Fixtures	<code>setUp</code> / <code>tearDown</code>	injected, composable, scoped
Many inputs	<code>subTest</code>	<code>parametrize</code> with ids
Failure output	basic	introspected, with diffs
Ecosystem	standard library	large plugin ecosystem
Dependencies	none	one package

A migration path that works: leave existing `TestCase` classes alone (pytest runs them), write *new* tests as plain functions, and convert old ones opportunistically.

5.10 Anti-patterns

- **Fixtures that hide the interesting setup.** If understanding a test requires opening three fixture files, inline the arrange step. Fixtures are for *shared* and *boring* setup.
- **Wide-scoped mutable fixtures** — hidden coupling between tests.

- **Logic in tests** (loops, `if` s computing the expected value) — the test can have its own bugs. Prefer tables of literal expected values.
- **Over-use of `autouse`** .
- **Asserting too much** in one test — the name stops describing the failure.

Check your understanding

1. What does the code after `yield` in a fixture do, and when does it run?
2. How many tests are generated by two stacked `@pytest.mark.parametrize` decorators with 3 and 4 values?
3. Why register markers and use `--strict-markers` ?
4. What is the difference between `xfail` and `xfail(strict=True)` when the bug gets fixed?

Lab 2 — pytest fundamentals

Duration: 3 h · **Type:** unit testing, data-driven testing · **Code under test:** `sensors.py` , `bms.py` , `can.py`

Why pytest

Less boilerplate (plain `assert`), far better failure output, fixtures with dependency injection, parametrization, a plugin ecosystem (coverage, Hypothesis, xdist, JUnit XML for CI) — and it runs `unittest` suites unchanged, so migration is gradual.

Learning objectives

Use plain asserts and `pytest.approx`; assert exceptions with `pytest.raises(match=)`; build fixtures (yield teardown, fixtures using fixtures, scopes); parametrize with ids; use `tmp_path`, `monkeypatch`, `capsys`; select tests with markers.

Concepts

- **Fixture** = named, injected, reusable arrange step; teardown after `yield` always runs.
- **Scope:** `function` (default, safest) → `module` → `session` (only for immutable/expensive objects).
- **parametrize**: one test body × many data rows. Give rows `ids` so a failure names the case.
- **Markers:** `smoke`, `xfail(strict=True)`, `skipif` — `--strict-markers` (enabled here) rejects typos.

Run it

```
pytest labs/lab02_pytest -v
pytest labs/lab02_pytest -v -m smoke
pytest labs/lab02_pytest -k "bms and not undervolt" -v
pytest labs/lab02_pytest --setup-show -k soc_clamped # see fixture set-up/teardown
```

Guided tour

Sections 1-5 of `test_lab02_pytest.py`: asserts → parametrization → fixtures → built-ins → markers. Note `battery` fixture: its teardown asserts an **invariant** (SOC in 0..100) after *every* test using it — one fixture, free protection.

Try this: make one parametrized row wrong. Compare pytest's failure output to Lab 1's.

Exercises — `exercise_lab02.py`

Run: `python course.py exercise 2` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

1. Collapse two copy-pasted tests into one parametrized test.
2. Write a *factory fixture* (`sensor_factory`) and test the moving-average filter.
3. `parametrize` with `pytest.param(..., id=..., marks=xfail)` for charge current by SOC.

4. `tmp_path` + error cases for a calibration-file reader.

Debrief

- When does a module-scoped fixture become a hidden coupling between tests?
- `xfail(strict=True)` vs `skip` — what does each tell the next engineer?
- Why is `pytest.approx` needed, and how do `rel` and `abs` differ?

Worked example — source

labs/lab02_pytest/test_lab02_pytest.py

```
"""Lab 2 - pytest: plain asserts, fixtures, parametrization, markers, built-ins.

Run: pytest labs/lab02_pytest -v
"""
import pytest

from autotest.bms import BatteryManagementSystem, BMSFault
from autotest.can import CANFrame, CANMessage, Signal
from autotest.sensors import SensorFault, TemperatureSensor, kmh_to_ms

# ---- 1. plain asserts: pytest rewrites them to show values -----
def test_plain_assert():
    assert kmh_to_ms(72) == pytest.approx(20.0)

def test_approx_for_floats():
    assert 0.1 + 0.2 == pytest.approx(0.3) # never compare floats with ==

def test_exceptions_with_context():
    with pytest.raises(ValueError, match="above maximum") as info:
        Signal("v", 0, 8, maximum=10).to_raw(11)
    assert "v" in str(info.value)

# ---- 2. parametrization: one test body, many cases -----
@pytest.mark.parametrize(
    "cells, temp, expected",
    [
        ([3.7, 3.7, 3.7], 25, BMSFault.NONE),
        ([3.7, 4.21, 3.7], 25, BMSFault.OVERVOLTAGE),
        ([3.7, 2.99, 3.7], 25, BMSFault.UNDERVOLTAGE),
        ([3.7, 3.7, 3.7], 61, BMSFault.OVERTEMPERATURE),
        ([3.7, 3.7, 3.7], -21, BMSFault.UNDERTEMPERATURE),
    ],
    ids=["healthy", "overvolt", "undervolt", "overtemp", "undertemp"],
)
def test_bms_fault_detection(cells, temp, expected):
    assert BatteryManagementSystem().evaluate(cells, temp) is expected

@pytest.mark.parametrize("dlc", range(0, 9))
def test_every_legal_dlc(dlc):
    assert CANFrame(0x100, bytes(dlc)).dlc == dlc

# ---- 3. fixtures: setup/teardown with dependency injection -----
class FakeADC:
    def __init__(self, *values):
        self.values = list(values)

    def read(self):
        return self.values.pop(0) if len(self.values) > 1 else self.values[0]
```

```

@pytest.fixture
def adc():
    return FakeADC(2048)

@pytest.fixture
def sensor(adc): # fixtures can use other fixtures
    return TemperatureSensor(adc)

def test_mid_scale_is_about_55_degrees(sensor):
    assert sensor.read() == pytest.approx(55.0, abs=0.05)

@pytest.fixture
def battery():
    bms = BatteryManagementSystem(capacity_ah=50, soc=50)
    yield bms # everything after yield is teardown
    assert 0 <= bms.soc <= 100, "invariant must hold after EVERY test"

def test_soc_clamped(battery):
    battery.update_soc(1000, 3600)
    assert battery.soc == 100

@pytest.fixture(scope="module")
def engine_message():
    """Module scope: built once, shared (only for immutable objects!)."""
    return CANMessage(0x2F0, "Engine", 4, (Signal("rpm", 0, 16, factor=0.25),))

def test_module_scoped_fixture(engine_message):
    assert engine_message.decode(engine_message.encode({"rpm": 1000}))["rpm"] == 1000

# ---- 4. built-in fixtures -----
def test_tmp_path_gives_each_test_a_private_directory(tmp_path):
    log = tmp_path / "trace.asc"
    log.write_text("0.001 1 123 Rx d 3 01 02 03\n")
    assert log.read_text().startswith("0.001")

def test_monkeypatch_environment(monkeypatch):
    import os
    monkeypatch.setenv("VEHICLE_VARIANT", "EV")
    assert os.environ["VEHICLE_VARIANT"] == "EV" # restored automatically afterwards

def test_capsys_captures_output(capsys):
    print("DTC U0121 set")
    assert "U0121" in capsys.readouterr().out

# ---- 5. markers: select what to run -----
@pytest.mark.smoke
def test_smoke_sensor_rails(sensor, adc):
    adc.values = [0]
    with pytest.raises(SensorFault):
        sensor.read()

@pytest.mark.skipif(not hasattr(pytest, "approx"), reason="needs pytest.approx")
def test_conditional_skip():
    assert True

@pytest.mark.xfail(reason="BUG-114: resolution 0.25 rpm rounds 0.1 rpm to 0", strict=True)
def test_known_bug_is_tracked():

```

```
sig = Signal("rpm", 0, 16, factor=0.25)
assert sig.to_raw(0.1) == 1
```

Exercise skeleton — source

labs/lab02_pytest/exercise_lab02.py

```
"""Lab 2 exercises - pytest: parametrize, fixtures, tmp_path.

Run it:      python course.py exercise 2
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
               test; the test then passes only if what you wrote is correct.
Done when:   `python course.py progress` shows every test of Lab 2 passing.
"""
import pytest

from autotest.bms import max_charge_current
from autotest.learn import todo
from autotest.sensors import SensorFault, TemperatureSensor

class ScriptedADC:
    """A fake ADC that returns the values you give it, one per read()."""

    def __init__(self, *values):
        self.values = list(values)

    def read(self):
        return self.values.pop(0)

# -----
# Exercise 1 - parametrize. Two copy-pasted tests would look like this:
#
#     def test_rails_a(): with pytest.raises(SensorFault): TemperatureSensor(ScriptedADC(0)).read()
#     def test_rails_b(): with pytest.raises(SensorFault):
TemperatureSensor(ScriptedADC(4095)).read()
#
# Write ONE parametrized test instead.
# -----
def test_adc_rail_values_are_sensor_faults():
    """Hint 1: @pytest.mark.parametrize("counts", [0, 4095], ids=["short-to-ground", "open-
circuit"])

    Hint 2: the decorated test takes `counts` as an argument and uses it in ScriptedADC(counts).
    Hint 3: remember to turn this into a function with a parameter: def test...(counts):
    """
    todo("parametrize over the two rail values")

# -----
# Exercise 2 - a factory fixture: a fixture that returns a FUNCTION so each test chooses its data.
# -----
@pytest.fixture
def sensor_factory():
    """Return a function make(*counts, filter_len=4) that builds a TemperatureSensor
    reading the given ADC counts (use ScriptedADC)."""
    todo("write the fixture body: define make(...) and return it")

def test_filter_averages_three_samples(sensor_factory):
    """Counts 1024, 2048, 3072 -> after three read_filtered() calls the result is the average of the
three
    temperatures, which equals the temperature of the MIDDLE count (the mapping is linear).

    Hint 1: sensor = sensor_factory(1024, 2048, 3072); call sensor.read_filtered() three times,
    keep the last value.
    Hint 2: expected = TemperatureSensor.T_MIN + (TemperatureSensor.T_MAX - TemperatureSensor.T_MIN)
    * 2048 / 4095
    Hint 3: assert last == pytest.approx(expected)
```



```

"""
    todo("use the factory and check the moving average")

# -----
# Exercise 3 - parametrize with ids. Charge current by SOC at 25 degC (rated current 100 A).
# Spec: no taper up to 80 %, linear taper to 0 A at 100 %.
# -----
@pytest.mark.parametrize("soc, expected", [
    pytest.param(0, 100.0, id="empty"),
    # TODO: add rows for 80, 90 and 100 (work the expected values out from the spec, with an id
each)
])
def test_charge_current_by_soc(soc, expected):
    """Hint: max_charge_current(soc, 25) == pytest.approx(expected).
    Hint 2: at 90 % the taper is (100 - 90) / 20 = 0.5 of the full current.
    """
    todo("complete the table (4 rows) and the assertion")

# -----
# Exercise 4 - tmp_path and error cases. A calibration file contains one line: circumference=1.95
# -----
def load_calibration(path):
    """Return the wheel circumference (float) from a file containing 'circumference=1.95'.

    Raises FileNotFoundError if the file does not exist, ValueError if the key is not
'circumference'.
    Hint: text = path.read_text() (a missing file raises FileNotFoundError by itself); key, _,
value = text.strip().partition("=")
    """
    todo("implement load_calibration")

def test_calibration_file_is_read(tmp_path):
    """Hint: create a file in the private tmp_path directory: f = tmp_path / "cal.txt";
f.write_text("circumference=1.95\\n")"""
    todo("write a good file, call load_calibration, expect 1.95")

def test_missing_calibration_file_raises(tmp_path):
    """Hint: use pytest.raises(FileNotFoundError) around load_calibration(tmp_path / "nope.txt")."""
    todo("expect FileNotFoundError")

def test_malformed_calibration_file_raises(tmp_path):
    """Hint: write 'radius=0.3' and expect ValueError; add match="unexpected key" to be specific."""
    todo("expect ValueError for a wrong key")

```

Part III — Test design and isolation

Chapter 6 — Designing tests from a specification

6.1 The problem: you cannot test everything

`max_charge_current(soc, temp_c)` takes two real numbers. Trying every pair of 64-bit floats would take longer than the age of the universe. Yet a handful of well-chosen tests can reveal almost all defects of a certain kind. **Test design techniques** are systematic ways to choose that handful — and to be able to *defend* why it is enough.

All techniques in this chapter are **black-box**: you derive them from the *specification*, without looking at the code. That has two big benefits: the tests stay valid when the implementation is rewritten, and they can expose faults of *omission* (behaviour the code forgot) that no amount of code-based testing can find.

The specification we use throughout is the docstring of `max_charge_current` :

```
temp < 0 or temp > 45    -> 0 A
0 <= temp < 10          -> 25 % of rated
10 <= temp <= 45        -> 100 % of rated
soc >= 100              -> 0 A
soc > 80                -> taper linearly from the temperature limit to 0 at 100 %
```

6.2 Requirements-based testing

The first rule: **every requirement gets at least one test**, and each test says which requirement it verifies (Lab 13 automates this). Read each sentence of the spec and ask "what observable behaviour does this promise?" If you cannot state an expected result, the *requirement* is untestable and must be clarified before coding — a valuable finding in itself.

6.3 Equivalence partitioning

Inputs that the specification treats **identically** form an **equivalence class**. If one member of the class exposes a fault, others probably would too; if one passes, the others probably pass. So test **one representative per class**.

Partition the temperature input:

$(-\infty, 0)$	$[0, 10)$	$[10, 45]$	$(45, +\infty)$
too cold	cold	normal	too hot
0 A	25 %	100 %	0 A
representative: -20	5	25	60

Rules of thumb:

- Include **valid and invalid** classes. Invalid classes (negative SOC, $\text{SOC} > 100$) test the error handling.

- Partition **each input**, then combine (carefully — see §6.6).
- Classes can be *derived from the output* too: "results where the current is zero" and "results where it is positive" are classes of the output.

```
import pytest
from autotest.bms import max_charge_current

@pytest.mark.parametrize(
    "temp, expected",
    [(-20, 0.0), (5, 25.0), (25, 100.0), (60, 0.0)],
    ids=["too-cold", "cold", "normal", "too-hot"],
)
def test_temperature_partitions(temp, expected):
    assert max_charge_current(soc=50, temp_c=temp) == pytest.approx(expected)
```

6.4 Boundary value analysis (BVA)

Programmers make **off-by-one** mistakes (`<` vs `<=`), so defects *cluster at the edges* of classes. **BVA** adds tests *on and next to every boundary*.

For the boundary "temperature ≥ 0 is allowed", with a resolution of 0.1:

```
-0.1      0.0      0.1
(still too cold) (boundary) (just inside)
```

- **Two-value BVA**: the boundary value and its closest neighbour on the *other* side.
- **Three-value BVA** (used in this course): below, on, and above — it also catches mistakes that flip the direction of the comparison.

Boundary	Values tested	What a defect would look like
0 °C	-0.1, 0.0, 0.1	<code>temp <= 0</code> instead of <code>temp < 0</code>
10 °C	9.9, 10.0, 10.1	cold limit applied at exactly 10 °C
45 °C	44.9, 45.0, 45.1	<code>temp >= 45</code> blocks charging at the allowed limit
80 % SOC	79.9, 80.0, 80.1	taper starts a step too early or late

For floats, "next to" means the smallest meaningful step of the specification (0.1 °C here), not the smallest representable float. When the spec says nothing about resolution, ask.

```
import pytest
from autotest.bms import max_charge_current

@pytest.mark.parametrize(
    "temp, expected",
    [(-0.1, 0.0), (0.0, 25.0), (0.1, 25.0),
     (9.9, 25.0), (10.0, 100.0), (10.1, 100.0),
     (44.9, 100.0), (45.0, 100.0), (45.1, 0.0)],
)
def test_temperature_boundaries(temp, expected):
    assert max_charge_current(soc=50, temp_c=temp) == pytest.approx(expected)
```

6.5 Decision tables

When behaviour depends on a **combination of conditions**, list every combination (or every *rule*) and its expected action. Single-condition tests miss **interactions**.

Rule	cold (0–10 °C)	soc > 80	hot/too cold	soc ≥ 100	Expected
R1	no	no	no	no	full current
R2	yes	no	no	no	25 %
R3	no	yes	no	no	tapered
R4	yes	yes	no	no	25 % and tapered
R5	—	—	yes	—	0 A
R6	—	—	no	yes	0 A

Rule R4 is the one a developer forgets: it combines two rules that each work alone. Each row becomes one parametrized case:

```
import pytest
from autotest.bms import max_charge_current

@pytest.mark.parametrize("soc, temp, expected", [
    (50, 25, 100.0), # R1 normal
    (50, 5, 25.0),   # R2 cold only
    (90, 25, 50.0),  # R3 taper only
    (90, 5, 12.5),   # R4 cold AND taper: the interaction
    (50, 50, 0.0),   # R5 hot
    (100, 25, 0.0),  # R6 full
])
def test_decision_table(soc, temp, expected):
    assert max_charge_current(soc, temp) == pytest.approx(expected)
```

Decision tables also expose **specification gaps**: a combination for which no action is defined (e.g. "hot and soc ≥ 100": which rule wins?). Log such questions; do not guess.

6.6 Combinatorial explosion and pairwise testing

With n parameters of k values each, all combinations number k^n . Empirical studies of defect data repeatedly find that most failures are triggered by the interaction of **one or two** parameters, rarely more. **Pairwise (all-pairs) testing** therefore chooses a small set of tests in which *every pair of values of every pair of parameters* appears at least once.

For three on/off parameters A, B, C: 8 combinations, but 4 tests cover all pairs.

```
from itertools import combinations, product

tests = [(0, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 0)] # 4 instead of 8

def pairs_covered(rows):
    return {(i, a), (j, b)} for r in rows for (i, a), (j, b) in combinations(enumerate(r), 2)

all_pairs = pairs_covered(product((0, 1), repeat=3))
assert pairs_covered(tests) == all_pairs
print(f"{len(tests)} tests cover all {len(all_pairs)} value pairs of the full {2**3}")
```

Tools such as `allpairspy` (Python) generate such sets automatically. Pairwise is excellent for **configuration** testing (variants × markets × options) and a poor substitute for a decision table when you need *specific* rule combinations.

6.7 Error guessing and checklists

Experience beats theory for the stubborn cases. Keep a checklist of inputs that break software, and apply it to every function:

Category	Probe
Numbers	0, -1, 1, max, max+1, -0.0, NaN , ±inf, tiny differences
Sizes	empty, one element, exactly full, one too many
Text/bytes	empty, non-ASCII, very long, embedded newline, wrong length
Time	wrap-around of counters, equal timestamps, going backwards
State	repeated call, call before initialisation, call after failure
Environment	missing file, unreadable device, interrupted transfer

The NaN probe is not hypothetical: it uncovered two real safety-relevant defects in this very course's BMS (Lab 8).

6.8 Invariants: tests without a precise oracle

Sometimes you cannot compute the exact expected value, but you know a **property** that must hold:

- output stays within physical limits: $0 \leq \text{current} \leq \text{rated}$ for every input;
- **monotonic**: more SOC never allows *more* current;
- **safety**: whenever temperature is outside 0–45 °C, the result is exactly zero.

Invariant checks over a grid (or, in Chapter 10, over *generated* inputs) are a cheap way to cover enormous input spaces.

6.9 Putting it together: a design recipe

1. List the **requirements**; mark any that are ambiguous.
2. For each input, draw the **partitions** (valid and invalid).
3. Mark every **boundary**; add three-value tests.
4. If outcomes depend on **combinations**, write the **decision table**; one row = one test.
5. Add **error-guessing** probes from the checklist.
6. Add **invariants** for properties that must always hold.
7. Write expected values **from the spec**, never by running the code.
8. Review: could a plausible fault (wrong operator, off-by-one, swapped constant) pass all these tests? (Chapter 15 measures this.)

Check your understanding

1. A function accepts SOC from 0 to 100 inclusive. List the boundary-value test inputs (three-value BVA) with resolution 1.
2. Why does a decision table often reveal more than a set of individual boundary tests?
3. Three parameters have 4, 3 and 2 values. How many exhaustive combinations exist, and why might pairwise testing need far fewer?
4. Why must the expected value in a test come from the specification and not from running the code?

Lab 3 — Black-box test design

Duration: 2.5 h · **Type:** specification-based testing · **Code under test:** `bms.max_charge_current`, `BatteryManagementSystem.evaluate`

Why it matters

You can never test all inputs (SOC × temperature is continuous). Test-design techniques choose the *few* inputs most likely to expose defects. ISO 26262-6 explicitly recommends **equivalence classes** and **boundary value analysis** for unit tests of safety-related software.

Learning objectives

Derive tests from a specification **without reading the code**: equivalence partitioning, boundary value analysis (BVA), decision tables, invariants/oracle-free checks.

Techniques

Technique	Idea	In this lab
Equivalence partitioning	inputs that the spec treats alike need one representative	temperature classes <code><0</code> , <code>0..<10</code> , <code>10..45</code> , <code>>45</code>
Boundary value analysis	defects cluster at edges → test <i>on</i> and <i>either side</i>	<code>-0.1/0/0.1</code> , <code>9.9/10/10.1</code> , <code>44.9/45/45.1</code>
Decision table	every combination of conditions → action	<code>cold</code> × <code>taper</code> × <code>hot</code> × <code>full</code> (note the <i>cold+taper interaction</i>)
Invariants	properties true for all inputs	<code>0 ≤ I ≤ rated</code> ; non-increasing in SOC

Run it

```
pytest labs/lab03_black_box -v
python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current \
    --tests labs/lab03_black_box/test_lab03_black_box.py # preview of Lab 12: 22/25 = 88 % (3
equivalent mutants)
```

Exercises — `exercise_lab03.py`

Run: `python course.py exercise 3` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

BVA for the six fault limits of `evaluate`, a priority decision table for simultaneous faults, and the "spec is silent" case (empty list) — decide, justify, and log the question.

Debrief

- Why test *both* sides of a boundary rather than only the boundary?
- The decision table has a row (cold + taper) that two single-condition tests would never reach. How did you find it?
- Which of your tests would still be valid if the implementation were rewritten completely? (That is the point of black-box.)

Worked example — source

labs/lab03_black_box/test_lab03_black_box.py

```
"""Lab 3 - test-design techniques (black box) on the BMS charge-current rule.

Spec under test: autotest.bms.max_charge_current (read its docstring FIRST - that is
the 'specification'; we design tests from it without looking at the implementation).
"""

import pytest

from autotest.bms import max_charge_current

RATED = 100.0

# ---- Equivalence partitioning: one representative per class -----
# Temperature classes: <0 | 0..<10 | 10..45 | >45          (SOC fixed at a neutral 50 %)
@pytest.mark.requirement("REQ-BMS-001")
@pytest.mark.parametrize(
    "temp, expected",
    [(-20, 0.0), (5, 25.0), (25, 100.0), (60, 0.0)],
    ids=["too-cold", "cold", "normal", "too-hot"],
)
def test_temperature_partitions(temp, expected):
    assert max_charge_current(soc=50, temp_c=temp) == pytest.approx(expected)

# ---- Boundary value analysis: values ON and either side of every boundary -----
@pytest.mark.requirement("REQ-BMS-001")
@pytest.mark.parametrize(
    "temp, expected",
    [
        (-0.1, 0.0), (0.0, 25.0), (0.1, 25.0),          # 0 degC boundary
        (9.9, 25.0), (10.0, 100.0), (10.1, 100.0),     # 10 degC boundary
        (44.9, 100.0), (45.0, 100.0), (45.1, 0.0),     # 45 degC boundary
    ],
)
def test_temperature_boundaries(temp, expected):
    assert max_charge_current(soc=50, temp_c=temp) == pytest.approx(expected)

@pytest.mark.requirement("REQ-BMS-002")
@pytest.mark.parametrize(
    "soc, expected",
    [(0, 100.0), (79.9, 100.0), (80.0, 100.0), (80.1, 99.5), (90, 50.0), (99, 5.0), (100, 0.0)],
)
def test_soc_taper_boundaries(soc, expected):
    assert max_charge_current(soc=soc, temp_c=25) == pytest.approx(expected)

@pytest.mark.parametrize("soc", [-0.1, 100.1])
def test_invalid_soc_is_rejected(soc):
    with pytest.raises(ValueError):
        max_charge_current(soc=soc, temp_c=25)

# ---- Decision table: every combination of conditions -> expected action -----
#   cold   | hot   | soc>80 | soc>=100 | result
```



```

# -----+-----+-----+-----+-----
# no      | no      | no      | no      | full current
# cold    |         |         |         | 25 % current
# no      | no      | yes     | no      | tapered
# cold    | no      | yes     | no      | 25 % AND tapered (rules combine!)
# any     | hot     | any     | any     | 0
DECISION_TABLE = [
    # soc, temp, expected
    (50, 25, 100.0), # normal
    (50, 5, 25.0),   # cold only
    (90, 25, 50.0),  # taper only
    (90, 5, 12.5),   # cold + taper (interaction - the classic missed case)
    (50, 50, 0.0),   # hot
    (90, 50, 0.0),   # hot beats taper
    (100, 25, 0.0),  # full
]

@pytest.mark.parametrize("soc, temp, expected", DECISION_TABLE)
def test_decision_table(soc, temp, expected):
    assert max_charge_current(soc, temp) == pytest.approx(expected)

# ---- Invariants / oracle-free checks -----
def test_current_never_negative_or_above_rated_over_whole_grid():
    for soc in range(0, 101, 5):
        for temp in range(-30, 71, 5):
            i = max_charge_current(soc, temp)
            assert 0.0 <= i <= RATED, (soc, temp, i)

def test_current_is_monotonic_non_increasing_in_soc():
    values = [max_charge_current(s / 2, 25) for s in range(0, 201)]
    assert values == sorted(values, reverse=True)

```

Exercise skeleton — source

labs/lab03_black_box/exercise_lab03.py

```

"""Lab 3 exercises - test design from a specification (no peeking at the code!).

Run it:      python course.py exercise 3
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
               test; the test then passes only if what you wrote is correct.
Done when:   `python course.py progress` shows every test of Lab 3 passing.

SPECIFICATION - BatteryManagementSystem().evaluate(cell_voltages, temp_c) returns:
    OVERVOLTAGE      if any cell > 4.20 V
    UNDERVOLTAGE     if any cell < 3.00 V
    OVERTEMPERATURE  if temp_c > 60
    UNDERTEMPERATURE if temp_c < -20
    NONE             otherwise
    When several apply, the FIRST in this list wins (priority order).
"""
import pytest

from autotest.bms import BatteryManagementSystem, BMSFault
from autotest.learn import todo

bms = BatteryManagementSystem()
HEALTHY = [3.7, 3.7]

@pytest.mark.parametrize("cells, temp, expected", [
    ([4.20, 3.7], 25, BMSFault.NONE), # ON the limit: still healthy (given, as an
example)
    ([4.21, 3.7], 25, BMSFault.OVERVOLTAGE), # just beyond the limit: a fault (given, as an
example)
    # TODO: add the other 6 boundary cases: under-voltage (3.00 / 2.99), over-temperature (60 /

```

```

60.1),
    #         under-temperature (-20 / -20.1). Use HEALTHY for the cells when you test a temperature.
    ])
def test_boundaries(cells, temp, expected):
    """Boundary value analysis: test ON each limit (still OK) and JUST BEYOND it (fault).

    Hint: the body is one line: assert bms.evaluate(cells, temp) is expected (enums are compared
    with `is`)
    """
    todo("complete the table with 6 more rows, then write the assertion")

def test_priority_between_faults():
    """Design a small decision table for SIMULTANEOUS faults, write it as a comment, then test it as
    a parametrized test.

    Hint 1: e.g. cells [4.3, 2.9] are over- AND under-voltage; which fault does the spec say wins?
    Hint 2: also try over-voltage + over-temperature, and under-voltage + over-temperature.
    Hint 3: turn it into @pytest.mark.parametrize("cells, temp, expected", [...]) and fill 4 rows.
    """
    todo("write the table and the parametrized test (replace this function)")

def test_empty_cell_list():
    """The spec says NOTHING about an empty list of cells. Decide what SHOULD happen, test it, and
    explain why in a comment.

    Hint 1: is 'healthy' a safe answer when there is no data at all?
    Hint 2: look at what the code does now (pytest.raises(ValueError) is one defensible choice) and
    write down
    """
    the question you would ask the product owner.
    """
    todo("choose a behaviour, justify it in a comment, assert it")

```

Chapter 7 — Isolating the unit: testability and test doubles

7.1 Why isolate?

Production code rarely lives alone. `TemperatureSensor` reads an **ADC** (hardware), `InstrumentCluster` listens to a **bus** and reads the **clock**, and both call a **diagnostic manager**. If a test uses the real collaborators, then:

- it may be **impossible** (no hardware on the CI server);
- it is **slow** (real time passes, real I/O happens);
- it is **non-deterministic** (clock, noise, random);
- it cannot **force rare situations** (a short-to-ground; 100 ms of bus silence);
- when it fails you do not know **whose fault** it is.

The remedy is to replace collaborators by controllable stand-ins — **test doubles** — for the duration of the test.

7.2 Designing for testability: seams

You can only substitute what the code lets you substitute. A **seam** is a place where behaviour can be swapped without editing the code. The simplest and best seam is **dependency injection (DI)**: pass collaborators in instead of creating them inside.

```
# Hard to test: the dependency is created inside, hidden.
import time
class ClusterBad:
    def is_stale(self, last_rx):
        return time.monotonic() - last_rx > 0.1    # real time -> must really wait

# Easy to test: the dependency is a parameter.
class ClusterGood:
    def __init__(self, clock):
        self.clock = clock
    def is_stale(self, last_rx):
        return self.clock.now() - last_rx > 0.1

from autotest.clock import FakeClock
clock = FakeClock()
cluster = ClusterGood(clock)
assert cluster.is_stale(0.0) is False
clock.advance(0.101)                                # "wait" 101 ms instantly
assert cluster.is_stale(0.0) is True
```

The recurring seams in this course:

Non-determinism / hardware	Injected as	Test replacement
Time	<code>clock.now()</code>	<code>FakeClock.advance()</code>
Hardware input	<code>ADC.read()</code> (a <code>Protocol</code>)	scripted / faulty ADC
Communication	bus object with <code>send</code> / <code>subscribe</code>	<code>VirtualCANBus</code> with fault hooks
Randomness	a seeded <code>random.Random(seed)</code>	fixed seed

A `Protocol` (structural interface) such as `ADC` documents what the code *needs* from its collaborator; *anything* with a `read()` method — real driver, simulator, mock — qualifies. This is also the key to **SIL ↔ HIL** switching (Lab 13).

7.3 The taxonomy of test doubles

(Terminology follows Gerard Meszaros, *xUnit Test Patterns*.)

Double	What it does	Verifies	Example
Dummy	fills a parameter, never used	nothing	a listener registered for an id that never occurs
Stub	returns canned answers	indirect <i>inputs</i>	<code>StubADC(0)</code> : forces "short to ground"
Fake	simplified <i>working</i> implementation	nothing itself	<code>FakeClock</code> , in-memory bus, a data-logger playing back samples
Spy	a real or fake object that records how it was used	afterwards, by inspection	<code>SpyADC.calls</code> counts conversions
Mock	pre-programmed with <i>expectations</i> and verifies them	indirect <i>outputs</i> (interactions)	<code>Mock(spec=DiagnosticManager).report.assert_called_with(...)</code>

```
import pytest
from autotest.sensors import SensorFault, TemperatureSensor

class StubADC:
    # STUB: a canned reading
    def __init__(self, counts): self.counts = counts
    def read(self): return self.counts

class SpyADC:
    # SPY: counts how it was used
    def __init__(self, counts): self.counts, self.calls = counts, 0
    def read(self):
        self.calls += 1
        return self.counts

def test_stub_forces_a_rare_fault():
    with pytest.raises(SensorFault, match="short to ground"):
        TemperatureSensor(StubADC(0)).read()

def test_spy_shows_one_conversion_per_call():
    spy = SpyADC(2000)
    sensor = TemperatureSensor(spy, filter_len=3)
    for _ in range(3):
        sensor.read_filtered()
    assert spy.calls == 3
```

7.4 State verification vs. interaction verification

There are two ways to judge a test:

- **State verification** — run the code, then check the *outcome*: a return value, the object's resulting state, the data on the bus. Resilient to refactoring.
- **Interaction (behaviour) verification** — check *which calls were made*: "diag.report('U0121', failed=True) was called". Necessary when the only observable effect **is** the call, but it ties the test to the implementation.

Two schools of thought exist ("classicist" vs "mockist"). A pragmatic rule that works well:

Prefer state verification with real, cheap, deterministic collaborators. Use stubs/fakes for slow or non-deterministic ones. Use mocks only when the interaction itself is the requirement.

7.5 Using `unittest.mock` well

```
from unittest import mock
from autotest.bus import VirtualCANBus
from autotest.diagnostics import DiagnosticManager
from autotest.ecu import DTC_COMM_LOST, InstrumentCluster
from autotest.clock import FakeClock

clock = FakeClock()
diag = mock.Mock(spec=DiagnosticManager) # spec=: misspelled methods raise AttributeError
cluster = InstrumentCluster(VirtualCANBus(), clock, diag)

clock.advance(0.5) # 500 ms without a frame
assert cluster.displayed_speed() == "--"
diag.report.assert_called_with(DTC_COMM_LOST, failed=True)
print("interaction verified")
```

- **Always pass `spec=`** (or `autospec=True`). A bare `Mock()` accepts any attribute, so a typo in `assert_called_wiht` silently *passes* (newer Python versions reject names starting with `assert`, but typos in other names still slip through).
- `side_effect=[a, b, Exc()]` scripts a *sequence* of results and exceptions — perfect for "works twice, fails on the third read".
- `call_count`, `assert_called_once_with`, `assert_not_called` verify interactions.

Patch where the name is *looked up*

`mock.patch("a.b.name")` replaces the attribute `name` in module `a.b`. A function imported with `from x import f` creates a *separate* reference in the importing module, so you must patch it *there*:

```
from unittest import mock
from autotest.bus import VirtualCANBus
from autotest.ecu import WheelSpeedECU

# ecu.py does: from .can import e2e_protect -> the name now lives in autotest.ecu
with mock.patch("autotest.ecu.e2e_protect", return_value=b"\x00" * 5) as fake:
    bus = VirtualCANBus()
    WheelSpeedECU(bus).send_speed(50)
    fake.assert_called_once()
    assert bus.log[0].data == b"\x00" * 5 # the ECU used OUR function

with mock.patch("autotest.can.e2e_protect", return_value=b"\x00" * 5):
    bus = VirtualCANBus()
```

```

WheelSpeedECU(bus).send_speed(50)
assert bus.log[0].data != b"\x00" * 5  # patched the wrong place: ECU still used the real
one

```

In pytest the equivalent is `monkeypatch.setattr("autotest.ecu.e2e_protect", fake)`, undone automatically.

7.6 Faking time and randomness

- **Time:** inject a clock (as above). `time.sleep()` in tests makes them slow *and* flaky: too short and the test fails on a loaded machine, too long and the suite crawls.
- **Randomness:** inject a `random.Random(seed)` (or seed once and **print** the seed on failure) so a failing run can be replayed exactly. Hypothesis (Chapter 10) does this for you.

7.7 Fakes that grow into simulators

A fake bus that records frames and can **corrupt, drop or delay** them is both a test double *and* a fault-injection tool (Chapters 8 and 11). A fake ADC that plays back a recorded signal becomes a **replay** tool (Chapter 12). The same abstraction supports tests at several levels — a major payoff of designing seams once.

7.8 When test doubles go wrong

- **Over-mocking:** every collaborator mocked, assertions only on calls. The test re-states the implementation; any refactoring breaks it, and it proves only that the mocks were configured.
- **Doubles that diverge from reality:** the stub returns `None` while the real object raises. Prevent with `spec / autospec` and with *integration tests* (Chapter 8) that use the real thing.
- **Mocking what you don't own:** wrap third-party APIs in your own thin adapter and fake *that*.
- **Testing the mock:** if removing the code under test would still leave the test green, it tests nothing.

Check your understanding

1. Classify: (a) a class returning `2048` for every `read()`; (b) `Mock()` asserting `report` was called once; (c) an in-memory bus; (d) an ADC object that counts reads.
2. Why does `mock.patch("autotest.can.e2e_protect")` not affect `autotest.ecu`'s calls to `e2e_protect`?
3. When is it appropriate to assert *interactions* instead of state?
4. How does dependency injection of a clock make a test both faster and *less* flaky than `time.sleep(0.11)`?

Lab 4 — Test doubles: isolating the unit

Duration: 2.5 h · **Type:** isolation techniques · **Code under test:** `TemperatureSensor`, `InstrumentCluster`, `WheelSpeedECU`

Why it matters

Real ECU software talks to ADCs, buses, NVM and other ECUs. On a CI server none of those exist, and some situations (short-to-ground, 100 ms bus silence) cannot be produced safely or repeatably on hardware. Doubles make the world scriptable.

The five doubles (Meszaros)

Double	Role	Example here
Dummy	placeholder	a listener on an id that never fires
Stub	canned answers	<code>StubADC(0)</code>
Fake	simplified real implementation	<code>SineWaveADC</code> , <code>FakeClock</code> , <code>VirtualCANBus</code>
Spy	records usage	<code>SpyADC.calls</code>
Mock	verifies expected interactions	<code>Mock(spec=DiagnosticManager)</code>

Key skills

- `mock.Mock(spec=Class)` — typos in method names raise instead of silently passing.
- `side_effect=[...]` for sequences and exceptions; `call_count`, `assert_called_with`.
- **Patch where the name is looked up** (`autotest.ecu.e2e_protect`, not `autotest.can.e2e_protect`).
- **Inject time:** `FakeClock.advance(0.5)` replaces `time.sleep(0.5)` — fast and deterministic.

Run it

```
pytest labs/lab04_test_doubles -v
```

Exercises — `exercise_lab04.py`

Run: `python course.py exercise 4` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

Spy on the alive counter · simulate an `OSError` from the ADC · timeout boundary at 99/101 ms with a fake clock · write 3 sentences on when a mock-based test is *bad*.

Debrief

- The last worked test prefers the **real** `DiagnosticManager` over a mock. Why? What does the mock version lock in?

- What would break if `ecu.py` did `from .can import e2e_protect` and you patched `autotest.can.e2e_protect`?
- Which doubles would you need to test a *real* `BenchADC` driver?

Worked example — source

labs/lab04_test_doubles/test_lab04_test_doubles.py

```

"""Lab 4 - test doubles: dummy, stub, fake, spy and mock.

The SUT depends on hardware (an ADC) and on a diagnostic manager. We replace those
collaborators so the test is fast, deterministic, and can force rare situations.
"""

from unittest import mock

import pytest

from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.diagnostics import DiagnosticManager
from autotest.ecu import DTC_COMM_LOST, InstrumentCluster, WheelSpeedECU
from autotest.sensors import SensorFault, TemperatureSensor

# ---- STUB: returns canned answers -----
class StubADC:
    def __init__(self, counts):
        self.counts = counts

    def read(self):
        return self.counts

def test_stub_forces_a_wiring_fault_that_real_hardware_rarely_produces():
    with pytest.raises(SensorFault, match="short to ground"):
        TemperatureSensor(StubADC(0)).read()

# ---- FAKE: a working but simplified implementation -----
class SineWaveADC:
    """Plays back a sequence like a data logger would."""

    def __init__(self, samples):
        self._it = iter(samples)

    def read(self):
        return next(self._it)

def test_fake_playback_feeds_the_moving_average():
    sensor = TemperatureSensor(SineWaveADC([1000, 2000, 3000, 2000]), filter_len=2)
    readings = [sensor.read_filtered() for _ in range(4)]
    expected_second = (TemperatureSensor.T_MIN + 190 * 1000 / 4095 + TemperatureSensor.T_MIN + 190 *
2000 / 4095) / 2
    assert readings[1] == pytest.approx(expected_second)
    assert readings[0] < readings[2] # trend follows the input

# ---- SPY: real object that records how it was used -----
class SpyADC:
    def __init__(self, counts):
        self.counts, self.calls = counts, 0

    def read(self):
        self.calls += 1
        return self.counts

def test_spy_counts_hardware_accesses():

```



```

spy = SpyADC(2000)
sensor = TemperatureSensor(spy, filter_len=3)
for _ in range(3):
    sensor.read_filtered()
assert spy.calls == 3 # exactly one ADC conversion per call: no hidden re-reads

# ---- MOCK (unittest.mock): verify INTERACTIONS -----
def test_mock_verifies_the_cluster_reports_to_diagnostics(clock):
    bus = VirtualCANBus()
    diag = mock.Mock(spec=DiagnosticManager) # spec= makes typos in method names fail
    cluster = InstrumentCluster(bus, clock, diag)

    clock.advance(0.5) # no frames for 500 ms
    assert cluster.displayed_speed() == "--"

    diag.report.assert_called_with(DTC_COMM_LOST, failed=True)

def test_mock_side_effect_simulates_intermittent_failure():
    adc = mock.Mock()
    adc.read.side_effect = [2048, 2048, 0, 2048] # third read: fault
    sensor = TemperatureSensor(adc)
    assert sensor.read() == pytest.approx(55.0, abs=0.05)
    sensor.read()
    with pytest.raises(SensorFault):
        sensor.read()
    assert adc.read.call_count == 3

# ---- patching: replace a name where it is LOOKED UP -----
def test_patch_replaces_collaborator_for_the_duration_of_the_test():
    with mock.patch("autotest.ecu.e2e_protect", return_value=b"\x00\x00\x00\x00") as fake:
        bus = VirtualCANBus()
        WheelSpeedECU(bus).send_speed(50)
        fake.assert_called_once()
    assert bus.log[0].data == b"\x00\x00\x00\x00\x00"

def test_monkeypatch_equivalent_in_pytest(monkeypatch):
    monkeypatch.setattr("autotest.ecu.e2e_protect", lambda payload, counter: b"\xAA" + payload)
    bus = VirtualCANBus()
    WheelSpeedECU(bus).send_speed(10)
    assert bus.log[0].data[0] == 0xAA

# ---- DUMMY: fills a parameter that is never used -----
def test_dummy_listener_is_never_called_for_other_ids():
    bus = VirtualCANBus()
    listener = mock.Mock()
    bus.subscribe(listener, ids={0x100})
    bus.send(CANFrame(0x200, b"\x01"))
    listener.assert_not_called()

# ---- Anti-pattern warning -----
def test_do_not_over_mock_real_collaborator_is_cheaper_here(clock):
    """The real DiagnosticManager is pure and fast, so using it beats a mock:
    we assert on OUTCOMES (state), not on call choreography."""
    diag = DiagnosticManager(fail_threshold=2)
    cluster = InstrumentCluster(VirtualCANBus(), clock, diag)
    clock.advance(1)
    cluster.displayed_speed()
    cluster.displayed_speed()
    assert diag.active_codes() == [DTC_COMM_LOST]

```

Exercise skeleton — source

labs/lab04_test_doubles/exercise_lab04.py

```

"""Lab 4 exercises - test doubles: spy, mock, fake clock.

Run it:      python course.py exercise 4
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
               test; the test then passes only if what you wrote is correct.
Done when:   `python course.py progress` shows every test of Lab 4 passing.
"""

from unittest import mock

import pytest

from autotest.ecu import InstrumentCluster, WheelSpeedECU
from autotest.learn import todo
from autotest.sensors import TemperatureSensor

# The fixtures `bus` (a VirtualCANBus) and `clock` (a FakeClock) come from the course's confest.py:
# just list them as test arguments.

class SpyListener:
    """A SPY records what happened to it. Complete it: a callable that stores every frame it
    receives."""

    def __init__(self):
        self.frames = []

    def __call__(self, frame):
        todo("store the frame: self.frames.append(frame)")

def test_alive_counter_increments(bus):
    """Exercise 1 (SPY). Subscribe a SpyListener to the bus, let a WheelSpeedECU send 3 frames, and
    check that the
    alive counter (byte 1 of each frame's data) goes 0, 1, 2.

    Hint 1: spy = SpyListener(); bus.subscribe(spy); ecu = WheelSpeedECU(bus); ecu.send_speed(10)
    ... (three times)
    Hint 2: assert [f.data[1] for f in spy.frames] == [0, 1, 2]
    """
    todo("spy on the bus and check the counters")

def test_adc_hardware_error_propagates():
    """Exercise 2 (MOCK with a side effect). A real ADC can raise OSError. What does
    TemperatureSensor do with it?
    Document the behaviour with a test.

    Hint 1: adc = mock.Mock(); adc.read.side_effect = OSError("bus error")
    Hint 2: build the sensor with the mock and call read() inside with pytest.raises(OSError):
    Hint 3: adc.read.assert_called_once_with() # verify the interaction too
    """
    todo("mock an ADC that raises OSError")

@pytest.mark.parametrize("age, shown", [
    (0.09, "60"),
    # TODO: add rows for the boundary: just inside the 100 ms timeout (0.099) and just outside
    (0.101), and 0.11
])
def test_cluster_timeout_boundary(bus, clock, age, shown):
    """Exercise 3 (FAKE clock). Send speed 60, advance the fake clock by `age` seconds, ask the
    cluster what it shows.

    Hint 1: cluster = InstrumentCluster(bus, clock); WheelSpeedECU(bus).send_speed(60)
    Hint 2: clock.advance(age); assert cluster.displayed_speed() == shown
    Think: why is time.sleep(0.11) the WRONG tool here? (Two reasons - write them as a comment.)
    """
    todo("complete the table and the test body")

```

Exercise 4 - reflection (no code): in a comment, write 3 sentences on WHEN a mock-based test is a BAD test.

Part IV — Integration and system testing

Chapter 8 — Integration testing: where components meet

8.1 What goes wrong between components

Unit tests prove each part works **in isolation against its own assumptions**. Integration defects are mismatches between those assumptions:

Mismatch	Example	Visible only when connected
Data layout	sender: speed in bytes 2–3 little-endian; receiver reads big-endian	✓
Scaling / units	factor 0.01 vs 0.1; m/s vs km/h	✓
Protocol state	alive counter wraps at 16; receiver expects 255	✓
Timing	sender cycle 20 ms, receiver timeout 15 ms	✓
Error handling	sender sends <code>valid=0</code> , receiver ignores the flag	✓
Sequencing	one corrupted frame disturbs two receptions (BUG-130, Lab 9)	✓

Each side's unit tests can be green while the system is wrong. Integration tests exist to catch exactly these.

8.2 Integration strategies

Strategy	Idea	Pros	Cons
Big bang	connect everything, then test	no scaffolding	failures cannot be localised
Bottom-up	integrate leaves first (codec → ECU → bus → cluster) using <i>drivers</i>	early confidence in basics	UI/top logic tested late
Top-down	start with the top component, <i>stub</i> the lower ones	early view of overall flow	many stubs
Sandwich	both directions, meeting in the middle	balanced	more planning
Risk-based	integrate the riskiest interface first	effort goes where defects are likely	needs a risk analysis

Choose by **risk**: interfaces with safety relevance, many teams, or complex timing deserve the earliest and heaviest testing.

8.3 A virtual bus as the integration environment

Our `VirtualCANBus` connects nodes the way a real bus does — `send(frame)` delivers to subscribers filtered by identifier — and adds two testing features:

- a **log** of every frame (like a bus logger);
- **fault hooks**: functions that may return a *modified frame* (corruption) or `None` (loss).

```
from autotest.bus import VirtualCANBus
from autotest.clock import FakeClock
from autotest.ecu import InstrumentCluster, WheelSpeedECU

bus, clock = VirtualCANBus(), FakeClock()
ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)

ecu.send_speed(87.4)
assert cluster.displayed_speed() == "87"           # the smoke test: it works end to end

clock.advance(0.099); assert cluster.displayed_speed() == "87"    # just inside the 100 ms timeout
clock.advance(0.002); assert cluster.displayed_speed() == "--"    # just outside
print("timing boundary verified without sleeping")
```

8.4 Interface and contract tests

An **interface specification** is a contract: *which frame, which bytes, which scaling, how often*. A **contract test** pins the contract itself by asserting the exact bytes on the wire:

```
from autotest.bus import VirtualCANBus
from autotest.ecu import WHEEL_SPEED_MSG, WheelSpeedECU

bus = VirtualCANBus()
WheelSpeedECU(bus).send_speed(100.0)
frame = bus.log[0]

assert frame.arbitration_id == 0x1A0
assert frame.data.hex() == "d000102701"    # crc=d0 counter=00 speed=0x2710 (10000 × 0.01) valid=1
assert WHEEL_SPEED_MSG.signals[0].factor == 0.01
print("contract holds:", frame.data.hex())
```

Why is this better than a round-trip test (encode then decode)? A round trip passes even if *both* sides change the scaling the same wrong way. Only a test against **independent, written-down bytes** detects an accidental contract change before the other team does. In industry, the independent source is the signal database (DBC/ARXML) and recorded traces.

8.5 Fault injection at the interface

The bus is the natural place to inject faults, because real buses *do* corrupt, lose and duplicate frames:

```
from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.clock import FakeClock
from autotest.ecu import InstrumentCluster, WheelSpeedECU

bus, clock = VirtualCANBus(), FakeClock()
ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)

ecu.send_speed(50)                                     # a good frame
bus.add_fault(lambda f: CANFrame(f.arbitration_id, bytes([f.data[0] ^ 0xFF]) + f.data[1:]))
ecu.send_speed(120)                                     # CRC byte corrupted "in flight"
```

```

value  assert cluster.displayed_speed() == "50"           # the cluster kept the last GOOD
        bus.clear_faults()
        ecu.send_speed(130)
        assert cluster.displayed_speed() == "130"       # and recovered immediately

```

This verifies a **safety mechanism end to end**: the receiver *rejects* corrupted data and does not display an invented value. Lab 8 repeats this with random, seeded bit errors; the chapter on robustness (Chapter 11) generalises it.

8.6 Smoke tests

A **smoke test** is a tiny end-to-end check that the system is fundamentally alive — one frame in, one number out. It runs first (`pytest -m smoke`), takes milliseconds, and answers "is it worth running the rest?" Name them and keep them few.

8.7 Timing in integration tests

Timing behaviour (timeouts, cycle times, debounce) is a major source of integration defects and a major source of **flaky** tests if done with real sleeps. Inject the clock and test **both sides of every limit**: just under the timeout (still valid) and just over it (invalid). The same method scales to supervision timers, watchdogs and E2E timeouts.

8.8 Practical advice

- Keep integration tests **few and meaningful**; each runs more code, so each failure is harder to localise. When one fails, **write a unit test that reproduces it** at the lowest level you can.
- Assert on **observable behaviour** at the interface (frames, displayed value, DTCs), not on private state.
- Share the environment (virtual bus, recorded traces) with later levels — the same scenarios run on a **rest-bus simulation** against a real ECU on the bench.
- Test **negative interface behaviour** too: wrong length, wrong identifier, `valid=0`, stale data.

Check your understanding

1. Give two integration defects that cannot be found by unit-testing each side separately.
2. Why is a contract test (exact bytes) stronger than an encode/decode round-trip test for catching a scaling change?
3. A sender's cycle time is 20 ms and the receiver's timeout is 15 ms. Which test finds this, and why is `time.sleep` unnecessary?
4. When an integration test fails, what should you do besides fixing the bug?

Lab 5 — Integration testing

Duration: 3 h · **Type:** integration / interface / contract testing · **Code under test:** WheelSpeedECU → VirtualCANBus → InstrumentCluster

Why it matters

Most field failures sit *between* components: one team scales by 0.01, another by 0.1; the alive counter wraps and the receiver rejects it; a CRC covers the wrong bytes. Unit tests of each side pass; the system is wrong.

Learning objectives

Test an interface end-to-end through a virtual bus; inject faults (corruption, loss, noise) with bus hooks; verify E2E protection and timeouts; write a **contract test** against the exact bytes on the wire; recognise smoke tests.

Strategy: integration order

Bottom-up (codec → ECU → bus → cluster) was used to build the SUT; *top-down* with stubs is the alternative. Choose by risk — integrate the riskiest interface first.

Run it

```
pytest labs/lab05_integration -v
pytest labs/lab05_integration -m smoke # a single, fastest end-to-end check
```

Guided tour

- `test_speed_travels_from_sensor_ecu_to_display` — the **smoke** test.
- `test_frame_on_the_wire_matches_the_interface_specification` — byte-level contract.
- `test_cluster_blanks_after_100ms...` — timing via `FakeClock`, both sides of 100 ms.
- `test_corrupted_frame...`, `test_lost_frame...` — `bus.add_fault(hook)`; a hook returns a modified frame or `None`.

Real finding: writing the Lab 9 replay later exposed that one corrupted frame cost two frames (BUG-130). Integration tests are where such cross-component sequencing defects hide.

Exercises — `exercise_lab05.py`

Run: `python course.py exercise 5` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

Add a third node (data logger) · seeded random bit-flips on 1 in 5 frames · a contract test that fails if the scale factor changes.

Debrief

- What does a contract test protect that a round-trip test cannot?
- Why must the fault-injection RNG be seeded?
- Which failures can only be found at integration level? Which would you rather find at unit level?

Worked example — source

labs/lab05_integration/test_lab05_integration.py

```
"""Lab 5 - integration testing: WheelSpeedECU -> VirtualCANBus -> InstrumentCluster.

Unit tests prove each part works alone; these prove the parts agree on the INTERFACE:
byte layout, scaling, E2E protection, alive counter and timing.
"""

import pytest

from autotest.can import CANFrame
from autotest.ecu import (
    DTC_COMM_LOST, DTC_E2E_FAIL, InstrumentCluster, WheelSpeedECU,
)

pytestmark = pytest.mark.integration

@pytest.fixture
def ecu(bus):
    return WheelSpeedECU(bus)

@pytest.fixture
def cluster(bus, clock):
    return InstrumentCluster(bus, clock)

@pytest.mark.smoke
def test_speed_travels_from_sensor_ecu_to_display(ecu, cluster):
    ecu.send_speed(87.4)
    assert cluster.displayed_speed() == "87"

@pytest.mark.parametrize("speed", [0, 0.01, 50, 130.55, 299.99])
def test_interface_scaling_agrees_end_to_end(ecu, cluster, speed):
    ecu.send_speed(speed)
    assert cluster.displayed_speed() == f"{speed:.0f}"

def test_frame_on_the_wire_matches_the_interface_specification(ecu, bus):
    ecu.send_speed(100.0)
    frame = bus.log[0]
    assert frame.arbitration_id == 0x1A0
    assert frame.dlc == 5 # crc + counter + 3 payload bytes
    assert frame.data[2:4] == (10000).to_bytes(2, "little") # 100.00 km/h, factor 0.01

@pytest.mark.requirement("REQ-CLU-001")
def test_cluster_blanks_after_100ms_without_frames(ecu, cluster, clock):
    ecu.send_speed(60)
    clock.advance(0.099)
    assert cluster.displayed_speed() == "60"
    clock.advance(0.002) # total 101 ms
    assert cluster.displayed_speed() == "--"

@pytest.mark.requirement("REQ-CLU-001")
def test_cluster_recovers_when_frames_return(ecu, cluster, clock):
    ecu.send_speed(60)
    clock.advance(1)
```



```

    assert cluster.displayed_speed() == "--"
    ecu.send_speed(70)
    assert cluster.displayed_speed() == "70"

@pytest.mark.requirement("REQ-CAN-003")
def test_corrupted_frame_is_rejected_and_last_good_value_kept(ecu, cluster, bus):
    ecu.send_speed(50)
    bus.add_fault(lambda f: CANFrame(f.arbitration_id, bytes([f.data[0] ^ 0xFF]) + f.data[1:]))
    ecu.send_speed(120) # CRC byte corrupted in flight
    assert cluster.displayed_speed() == "50"
    assert cluster.diag.stored_codes() == [] # one failure is only debounced...
    ecu.send_speed(121)
    assert cluster.diag.stored_codes() == [DTC_E2E_FAIL] # ...two confirm the DTC

@pytest.mark.requirement("REQ-CAN-003")
def test_lost_frame_is_detected_by_alive_counter_and_cluster_resyncs(ecu, cluster, bus):
    ecu.send_speed(10)
    bus.add_fault(lambda f: None) # bus drops one frame
    ecu.send_speed(20)
    bus.clear_faults()
    ecu.send_speed(30) # counter jumped by 2 -> rejected once
    assert cluster.displayed_speed() == "10"
    ecu.send_speed(40) # resynchronised -> accepted
    assert cluster.displayed_speed() == "40"

def test_cluster_ignores_other_message_ids(ecu, cluster, bus):
    ecu.send_speed(33)
    bus.send(CANFrame(0x7FF, b"\xFF" * 8)) # unrelated traffic
    assert cluster.displayed_speed() == "33"

def test_comm_lost_dtc_is_confirmed_by_debounce(cluster, clock):
    clock.advance(1)
    cluster.displayed_speed()
    assert cluster.diag.active_codes() == [] # first miss: only counted
    cluster.displayed_speed()
    assert cluster.diag.active_codes() == [DTC_COMM_LOST]

def test_bus_log_contains_every_frame_even_when_dropped(ecu, bus):
    bus.add_fault(lambda f: None)
    assert ecu.send_speed(10) is False
    assert len(bus.log) == 1

```

Exercise skeleton — source

labs/lab05_integration/exercise_lab05.py

```

"""Lab 5 exercises - integration, fault injection, contract tests.

Run it:      python course.py exercise 5
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
               test; the test then passes only if what you wrote is correct.
Done when:   `python course.py progress` shows every test of Lab 5 passing.

Fixtures from conftest.py: `bus` (VirtualCANBus) and `clock` (FakeClock).
Useful: bus.add_fault(hook) installs a function hook(frame) -> frame | None (None = drop the
frame);
               bus.clear_faults(); bus.log = list of every frame sent.
"""
import random

from autotest.bus import VirtualCANBus
from autotest.can import CANFrame, E2EStatus, e2e_check
from autotest.ecu import WHEEL_SPEED_MSG, InstrumentCluster, WheelSpeedECU
from autotest.learn import todo

```

```

class DataLogger:
    """A third node on the bus: records the speed of every frame that passes the CRC check."""

    def __init__(self, bus):
        self.speeds = []
        # TODO: subscribe self._rx to the bus, only for frame id WHEEL_SPEED_MSG.frame_id
        (bus.subscribe(listener, ids={...}))

    def _rx(self, frame):
        # TODO: status, payload = e2e_check(frame.data); if OK: decode with
        WHEEL_SPEED_MSG.decode(payload)["speed_kmh"]
        pass

    def test_logger_sees_what_cluster_sees(bus, clock):
        """Exercise 1. First complete DataLogger above. Then: send 10, send 20 with a corrupted CRC
        byte, send 30.
        The logger must record exactly [10.0, 30.0] and the cluster must show "30".

        Hint 1: corrupt like this: bus.add_fault(lambda f: CANFrame(f.arbitration_id, bytes([f.data[0]
        ^ 0xFF]) + f.data[1:]))
        Hint 2: remove the fault again with bus.clear_faults() before sending 30.
        """
        todo("complete DataLogger, then write the scenario")

    def test_cluster_never_displays_invented_speed(bus, clock):
        """Exercise 2. A fault hook flips ONE random bit in 1 of every 5 frames (use random.Random(42)
        so it is repeatable).
        Send 100 frames. Whatever the cluster shows must be a speed that was actually SENT (or "--").

        Hint 1: rng = random.Random(42); in the hook: if rng.randrange(5) == 0: copy data to a
        bytearray, flip one bit, return a new CANFrame
        Hint 2: keep a set `sent` of the strings you sent (str(speed)); after each send assert shown
        in sent | {"--"}
        """
        todo("write the random-corruption test")

    def test_wheel_speed_contract():
        """Exercise 3 - a CONTRACT test: pin the exact bytes on the wire so nobody changes the scale
        factor unnoticed.

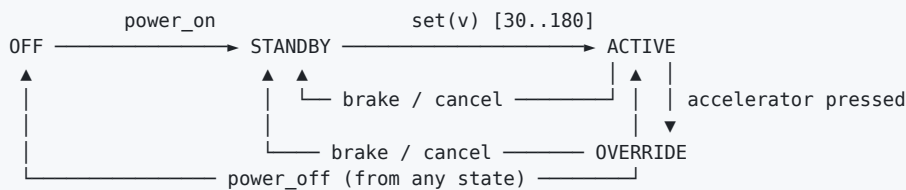
        Hint 1: bus = VirtualCANBus(); WheelSpeedECU(bus).send_speed(100.0); look at bus.log[0].data
        Hint 2: print(bus.log[0].data.hex()) once and CHECK it by hand against the spec (CRC, counter,
        speed 100.00 km/h = raw 10000
        little-endian, valid flag) BEFORE you paste it as the expected value - otherwise you
        only freeze whatever the code does.
        Hint 3: also assert the frame id is 0x1A0.
        """
        todo("assert the exact bytes of a 100.0 km/h frame")

```

Chapter 9 — State-based and system-level testing

9.1 Behaviour that depends on history

Some behaviour cannot be understood from the *current input* alone. The cruise controller reacts to "brake pressed" differently depending on whether it is OFF, STANDBY, ACTIVE or OVERRIDE. Such systems are described by **state machines**: a set of **states**, **events** (inputs), **transitions** (state × event → next state, possibly with an action), and an initial state.



State-based defects are typical safety hazards: "*cruise stays active after braking*", "*re-engages unexpectedly*", "*throttle non-zero in STANDBY*".

9.2 From diagram to transition table

A diagram shows the *valid* arrows; a **state-transition table** also shows what happens for **every** event in every state, including events that should be *ignored*. This completeness is what finds missing handling:

state \ event	power_on	power_off	set	resume	cancel	accel_on	accel_off
OFF	STANDBY	OFF	OFF (refused)	OFF	OFF	OFF	OFF
STANDBY	STANDBY	OFF	ACTIVE	STANDBY ¹	STANDBY	STANDBY	STANDBY
ACTIVE	ACTIVE	OFF	ACTIVE	ACTIVE	STANDBY	OVERRIDE	ACTIVE
OVERRIDE	OVERRIDE	OFF	ACTIVE	OVERRIDE	STANDBY	OVERRIDE	ACTIVE

¹ `resume` from STANDBY becomes ACTIVE only if a saved target exists and the speed is ≥ 30 km/h.

This 4 × 7 table is a complete test specification: 28 rows → 28 tests, and *every* cell is a decision someone made — or forgot to make.

9.3 Coverage levels for state machines

Level	Requirement	Strength
All-states	visit every state	weakest
All-transitions (0-switch)	take every <i>valid</i> transition once	the usual minimum
All invalid events	every event in every state, including "should be ignored"	finds missing handling
n-switch	every <i>sequence</i> of <i>n+1</i> consecutive transitions	finds history-dependent faults

Level	Requirement	Strength
Exhaustive sequences up to length n	all event sequences of length n	strongest, grows as e^n

For 7 events, all sequences of length 3 number $7^3 = 343$ — still trivial for a computer, impossible by hand. That is the power of generating them.

9.3.1 Testing a table in code

```
import pytest
from autotest.cruise import CruiseController, CruiseState as S

def drive_into(state):
    cc = CruiseController()
    if state in ("standby", "active", "override"):
        cc.power_on()
    if state in ("active", "override"):
        cc.set(100)
    if state == "override":
        cc.accelerator(True)
    return cc

EVENTS = {"power_on": lambda c: c.power_on(), "power_off": lambda c: c.power_off(),
          "set": lambda c: c.set(100), "cancel": lambda c: c.cancel(),
          "accel_on": lambda c: c.accelerator(True), "accel_off": lambda c: c.accelerator(False)}

@pytest.mark.parametrize("state, event, expected", [
    ("off", "power_on", S.STANDBY), ("off", "set", S.OFF),
    ("standby", "set", S.ACTIVE), ("standby", "accel_on", S.STANDBY),
    ("active", "cancel", S.STANDBY), ("active", "accel_on", S.OVERRIDE),
    ("override", "accel_off", S.ACTIVE), ("override", "power_off", S.OFF),
])
def test_transitions(state, event, expected):
    cc = drive_into(state)
    EVENTS[event](cc)
    assert cc.state is expected
```

9.4 Invariants over exhaustive sequences

Writing the expected *next state* for 343 sequences is pointless; instead assert **invariants** — facts that must hold in *every* reachable state:

- throttle is exactly 0 unless the state is ACTIVE;
- the controller has a target if and only if it is ACTIVE (or OVERRIDE, where the target is remembered);
- nothing ever raises an exception.

```
import itertools
from autotest.cruise import CruiseController, CruiseState as S

EVENTS = {
    "power_on": lambda c: c.power_on(), "power_off": lambda c: c.power_off(),
    "set": lambda c: c.set(100), "resume": lambda c: c.resume(100), "cancel": lambda c: c.cancel(),
    "accel_on": lambda c: c.accelerator(True), "accel_off": lambda c: c.accelerator(False),
}

checked = 0
for sequence in itertools.product(EVENTS, repeat=3):
    cc = CruiseController()
    for name in sequence:
        EVENTS[name](cc)
```

```

    assert (cc.state is S.ACTIVE) == (cc.target is not None) or cc.state is S.OVERRIDE
    if cc.state is not S.ACTIVE:
        assert cc.control(50, 0.1) == 0.0          # SAFETY: no throttle outside ACTIVE
    checked += 1
    print(f"{checked} sequences keep every invariant")

```

This is a **model-free oracle**: we do not need to know the exact state after each sequence, only that the *rules of the system* are never violated.

9.5 Testing the control law

The cruise controller is also a numerical algorithm. Verify it at two levels:

1. **Unit level**: exact values from a hand calculation. With $K_p = 0.05$, $K_i = 0.01$, target 100, speed 90, $dt = 0.1$: $u_1 = 0.05 \cdot 10 + 0.01 \cdot (0 + 10 \cdot 0.1) = 0.51$. Then 0.52, 0.53.
2. **Property level**: output is always within $[0, 1]$ (saturation), and **anti-windup** holds — while the output is saturated the integral must not keep growing.

```

import pytest
from autotest.cruise import CruiseController

def test_pi_law_matches_hand_calculation():
    cc = CruiseController(kp=0.05, ki=0.01)
    cc.power_on(); cc.set(100)
    assert [cc.control(90, 0.1) for _ in range(3)] == pytest.approx([0.51, 0.52, 0.53])

def test_no_integral_windup_while_saturated():
    cc = CruiseController()
    cc.power_on(); cc.set(100)
    for _ in range(1000):
        assert cc.control(30, 0.1) == 1.0          # far below target: saturated at full throttle
        assert cc._integral == 0.0                 # white-box: the integral did not wind up

```

The second test peeks at private state — a deliberate **white-box** check, acceptable here because the *behavioural* consequence of windup (huge overshoot later) would be slow and indirect to demonstrate. Closed-loop effects belong to Chapter 14.

9.6 System tests: scenarios

A **system test** exercises the assembled behaviour against *requirements*. A good format is a **scenario** in *Given / When / Then* form:

```

Scenario: Driver overtakes while cruising
    Given cruise control is ACTIVE at 100 km/h
    When the driver presses the accelerator for 5 s and then releases it
    Then cruise control is ACTIVE again with the SAME target of 100 km/h
    And the throttle request after release stays below 0.5

```

Each clause maps to code: *Given* = fixture, *When* = stimulus, *Then* = assertions. Writing scenarios in customer language makes them reviewable by requirement owners and test engineers who do not read Python.

9.7 Oracles for system tests

Because the full system has no single "expected value", system tests lean on:

- **Requirement oracles** — concrete limits (`dip < 4 km/h`).
- **Invariants / safety properties** — "never", "always".
- **Reference models** — a simple simulation or earlier release to compare against (**back-to-back testing**).
- **Metamorphic relations** — e.g. a higher target never produces a lower steady-state speed.

9.8 Practical advice

- **Draw the diagram first**; if you cannot, the behaviour is under-specified.
- Test **invalid events** — they are where state machines misbehave.
- Keep the *model* (table) next to the tests; when the design changes, change the table first.
- If states explode, partition into **hierarchical** machines or restrict exhaustive testing to the safety-relevant subset.

Check your understanding

1. State machine with 5 states and 6 events: how many cells has the complete transition table, and what does covering only the valid arrows miss?
2. Why assert invariants over all 3-event sequences instead of explicit expected states?
3. Write the *Then* clause for: "Given cruise ACTIVE, When the driver presses the brake".
4. What is anti-windup and which symptom reveals its absence in a closed-loop test?

Lab 6 — State-transition and system-level testing

Duration: 3 h · **Type:** state-based testing, functional/system testing · **Code under test:** `CruiseController`

Why it matters

Mode logic (OFF / STANDBY / ACTIVE / OVERRIDE) is a classic place for hazardous bugs: cruise that stays active after braking, or re-engages unexpectedly. Behaviour depends on *history*, so single-input tests are not enough.

Learning objectives

Model behaviour as states × events; cover **all states, all valid transitions, all invalid events**; use an exhaustive short-sequence test with invariants; test the control law (saturation, anti-windup) at the same time.

Coverage levels

Level	Meaning	Here
All-states	every state visited	covered by transition tests
0-switch (all transitions)	every valid arrow taken	<code>test_off_to_standby ... test_accelerator_overrides...</code>
Invalid-event coverage	every event in every state	exhaustive 3-event sequences (343 cases) + exercise 1's 28-row table
n-switch	sequences of n transitions	<code>itertools.product(EVENTS, repeat=3)</code>

Run it

```
pytest labs/lab06_state_machine -v -k "not sequences"
pytest labs/lab06_state_machine -q           # includes 343 generated sequence tests
```

Guided tour

The state diagram is in the module docstring — **draw it before reading the tests**. Observe: the sequence test asserts *invariants* (`ACTIVE ⇔ target is set`, throttle is 0 outside ACTIVE) instead of exact outputs, so it can run over every sequence with no oracle for each one. `test_integral_does_not_wind_up...` is a deliberate **white-box** check.

Exercises — `exercise_lab06.py`

Run: `python course.py exercise 6` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

28-row transition table · "overtaking" scenario in Given/When/Then with the plant model · a design question (resume above max).

Debrief

- Which invalid transitions are *safety relevant* here? Would you want a DTC on them?
- What is the difference between asserting `state` and asserting the externally visible *behaviour* (throttle)?
- How many states/events before exhaustive sequence testing becomes impractical — and what do you do then?

Worked example — source

labs/lab06_state_machine/test_lab06_state_machine.py

```
"""Lab 6 - state-transition and system-level testing of the cruise controller.

Technique: draw the state machine, then cover (a) every state, (b) every VALID transition,
(c) every INVALID event in every state ("0-switch" and invalid-transition coverage).

      power_on          set(v)          brake/cancel
OFF -----> STANDBY -----> ACTIVE -----> STANDBY
 ^  <--- power_off (any state) --- | ^
                                accel  v | release
                                OVERRIDE

"""
import itertools

import pytest

from autotest.cruise import CruiseController, CruiseState as S

pytestmark = pytest.mark.system

@pytest.fixture
def cc():
    return CruiseController()

@pytest.fixture
def active(cc):
    cc.power_on()
    assert cc.set(100)
    return cc

# ---- valid transitions -----
def test_initial_state_is_off(cc):
    assert cc.state is S.OFF and cc.target is None

def test_off_to_standby(cc):
    cc.power_on()
    assert cc.state is S.STANDBY

@pytest.mark.requirement("REQ-CC-001")
def test_standby_to_active_with_set(cc):
    cc.power_on()
    assert cc.set(100) is True
    assert (cc.state, cc.target) == (S.ACTIVE, 100)

@pytest.mark.requirement("REQ-CC-002")
```



```

def test_brake_cancels_and_remembers_target(active):
    active.brake()
    assert active.state is S.STANDBY and active.target is None
    assert active.saved_target == 100

def test_resume_restores_saved_target(active):
    active.cancel()
    assert active.resume(current_speed_kmh=90) is True
    assert (active.state, active.target) == (S.ACTIVE, 100)

def test_accelerator_overrides_then_returns_to_active(active):
    active.accelerator(True)
    assert active.state is S.OVERRIDE
    active.accelerator(False)
    assert active.state is S.ACTIVE

def test_set_while_active_changes_target(active):
    assert active.set(120)
    assert active.target == 120

@pytest.mark.parametrize("state_setup", ["off", "standby", "active", "override"])
def test_power_off_from_every_state_clears_everything(cc, state_setup):
    if state_setup != "off":
        cc.power_on()
    if state_setup in ("active", "override"):
        cc.set(100)
    if state_setup == "override":
        cc.accelerator(True)
    cc.power_off()
    assert (cc.state, cc.target, cc.saved_target) == (S.OFF, None, None)

# ---- invalid events (negative transitions) -----
def test_set_is_refused_when_off(cc):
    assert cc.set(100) is False and cc.state is S.OFF

@pytest.mark.requirement("REQ-CC-001")
@pytest.mark.parametrize("speed, accepted", [
    (29.9, False), (30.0, True), (30.1, True), (179.9, True), (180.0, True), (180.1, False),
])
def test_set_speed_window_boundaries(cc, speed, accepted):
    cc.power_on()
    assert cc.set(speed) is accepted
    assert cc.state is (S.ACTIVE if accepted else S.STANDBY)

def test_resume_without_history_is_refused(cc):
    cc.power_on()
    assert cc.resume(100) is False

def test_resume_below_minimum_speed_is_refused(active):
    active.cancel()
    assert active.resume(20) is False and active.state is S.STANDBY

def test_accelerator_is_ignored_in_standby(cc):
    cc.power_on()
    cc.accelerator(True)
    assert cc.state is S.STANDBY

def test_cancel_in_off_is_a_noop(cc):
    cc.cancel()
    assert cc.state is S.OFF

# ---- exhaustive: every event in every state never crashes and keeps invariants -----

```

```

EVENTS = {
    "power_on": lambda c: c.power_on(),
    "power_off": lambda c: c.power_off(),
    "set": lambda c: c.set(100),
    "resume": lambda c: c.resume(100),
    "cancel": lambda c: c.cancel(),
    "accel_on": lambda c: c.accelerator(True),
    "accel_off": lambda c: c.accelerator(False),
}

@pytest.mark.parametrize("sequence", list(itertools.product(EVENTS, repeat=3)), ids="-".join)
def test_all_3_event_sequences_keep_invariants(sequence):
    cc = CruiseController()
    for name in sequence:
        EVENTS[name](cc)
        assert (cc.state is S.ACTIVE) == (cc.target is not None) or cc.state is S.OVERRIDE
        assert cc.control(100, 0.1) >= 0
        if cc.state is not S.ACTIVE:
            assert cc.control(50, 0.1) == 0.0

# ---- control law: system behaviour with a simple vehicle model -----
@pytest.mark.requirement("REQ-CC-003")
def test_throttle_is_zero_unless_active(cc):
    assert cc.control(50, 0.1) == 0.0
    cc.power_on()
    assert cc.control(50, 0.1) == 0.0

def test_throttle_is_saturated_between_0_and_1(active):
    assert active.control(30, 0.1) == 1.0      # far below target -> full throttle
    assert active.control(150, 0.1) == 0.0     # above target -> no throttle

def test_integral_does_not_wind_up_during_saturation(active):
    for _ in range(1000):
        active.control(30, 0.1)
    assert active._integral == 0.0              # white-box check of the anti-windup rule

```

Exercise skeleton — source

labs/lab06_state_machine/exercise_lab06.py

```

"""Lab 6 exercises - state-transition and scenario testing of the cruise controller.

Run it:      python course.py exercise 6
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
               test; the test then passes only if what you wrote is correct.
Done when:   `python course.py progress` shows every test of Lab 6 passing.

Draw the state diagram first (see the Chapter 9 lecture): OFF -> STANDBY -> ACTIVE <-> OVERRIDE.
"""
import pytest

from autotest.cruise import CruiseController, CruiseState as S
from autotest.learn import todo
from autotest.vehicle import run_cruise

def drive_into(state):
    """Helper (given): return a controller that is in the named state."""
    cc = CruiseController()
    if state in ("standby", "active", "override"):
        cc.power_on()
    if state in ("active", "override"):
        cc.set(100)
    if state == "override":
        cc.accelerator(True)

```

```

    return cc

EVENTS = {
    # helper (given): the 7 driver events
    "power_on": lambda c: c.power_on(),
    "power_off": lambda c: c.power_off(),
    "set": lambda c: c.set(100),
    "resume": lambda c: c.resume(100),
    "cancel": lambda c: c.cancel(),
    "accel_on": lambda c: c.accelerator(True),
    "accel_off": lambda c: c.accelerator(False),
}

# Exercise 1 - the COMPLETE transition table: 4 states x 7 events = 28 cells.
# Fill it from the specification (Chapter 9, section 9.2) - NOT by running the code and copying the
output.
#
#          power_on  power_off  set      resume    cancel    accel_on  accel_off
TABLE = {
    "off": (S.STANDBY, S.OFF, S.OFF, S.OFF, S.OFF, S.OFF, S.OFF), #
(given)
    "standby": (), # TODO: 7 states
    "active": (), # TODO
    "override": (), # TODO
}
# (zip without strict=: the rows are empty until you fill them in; see Hint 3 for a row-length
check)
CASES = [(state, event, nxt) for state, row in TABLE.items() for event, nxt in zip(EVENTS, row)] #
noqa: B905

@pytest.mark.parametrize("state, event, expected", CASES)
def test_full_transition_table(state, event, expected):
    """Hint 1: cc = drive_into(state); EVENTS[event](cc); assert cc.state is expected
    Hint 2: remember `resume` only works in STANDBY with a saved target - drive_into("standby") has
    none.
    Hint 3: after you filled the table there must be 28 cases (4 rows x 7 events). A row with the
    wrong number of entries would
    silently drop cases - so add a test that asserts len(row) == 7 for every row of TABLE.
    """
    todo("fill the TABLE (3 rows of 7) and write the 3-line body")

def test_overtaking_scenario():
    """Exercise 2 - a Given/When/Then scenario with the plant model.
    Given cruise is ACTIVE at 100 km/h
    When the driver presses the accelerator for 5 s and then releases it
    Then cruise is ACTIVE again with the SAME target, and the throttle after release stays
    below 0.5

    Hint 1: cc = CruiseController(); cc.power_on(); cc.set(100)
    Hint 2: trace = run_cruise(cc, 100, 60, events={20.0: lambda c, car: c.accelerator(True),
    25.0: lambda c, car: c.accelerator(False)})
    (trace is a list of (time_s, speed_kmh, throttle))
    Hint 3: assert (cc.state, cc.target) == (S.ACTIVE, 100); max(u for t, _, u in trace if t >= 25)
    < 0.5
    """
    todo("write the scenario")

def test_resume_above_max_speed():
    """Exercise 3 - a design question. What should resume() do when the car is already faster than
    180 km/h?
    The spec is silent. CHOOSE a behaviour, write the test for it, and record the open question in a
    comment.
    Hint: set 100, cancel(), then resume(200): look at the return value and cc.target - is that what
    you want?
    """
    todo("choose, test, and document")

```

Part V — Finding the unexpected

Chapter 10 — Property-based testing and fuzzing

10.1 From examples to properties

An **example-based** test says: *for this input, expect that output*. Its weakness is obvious — it checks only the inputs **you thought of**. A **property-based** test says: *for **all** inputs of this kind, this statement is true*, and a tool generates hundreds of inputs to try to **falsify** it.

```
example test:    encode(100.0) == bytes 10 27
property test:   for every speed in [0, 300]: decode(encode(speed)) ≈ speed
```

The tool we use is **Hypothesis**. Its three core ideas:

1. **Strategies** describe how to generate data (`st.integers()` , `st.floats(0, 300)` , `st.binary(max_size=8)` , `st.lists(...)` , `st.builds(Class, ...)`).
2. **Search**: it tries many inputs, biased toward troublemakers (0, -1, huge values, NaN, empty collections).
3. **Shrinking**: when it finds a failure it *simplifies* the input to the smallest counter-example it can, so the bug report is readable.

10.2 A first property: round trip

```
import pytest
from hypothesis import given, strategies as st
from autotest.can import Signal

@given(st.floats(min_value=0, max_value=300, allow_nan=False))
def test_wheel_speed_round_trip_within_one_resolution_step(speed):
    sig = Signal("speed", 0, 16, factor=0.01, minimum=0, maximum=300)
    assert sig.to_physical(sig.to_raw(speed)) == pytest.approx(speed, abs=0.005 + 1e-9)
```

The tolerance is half the resolution (0.005): quantisation rounds to the nearest 0.01 step. Writing the property forced us to *state* that precision precisely — a specification question we might otherwise have skipped.

10.3 Shrinking in action

What if we (wrongly) claim the round trip is *exact*? Hypothesis finds a counter-example and shrinks it:

```
from hypothesis import find, strategies as st
from autotest.can import Signal

sig = Signal("speed", 0, 16, factor=0.01)
smallest_bad = find(st.floats(min_value=0, max_value=300),
                    lambda x: sig.to_physical(sig.to_raw(x)) != x)
print("smallest counter-example found:", smallest_bad)
assert smallest_bad != sig.to_physical(sig.to_raw(smallest_bad))
```

Failures are also stored in a local example database (`.hypothesis/`), so a bug found once is **re-tested first** on every later run — automatic regression protection.

10.4 Finding good properties

"What property?" is the hard part. A catalogue of patterns, with automotive examples from this course:

Pattern	Statement	Example
Round trip	<code>decode(encode(x)) ≈ x</code>	CAN signal codec; message with several signals
Detection guarantee	a corrupted input is always rejected	CRC detects <i>every</i> single-bit flip
Invariant / safety	something is true for all inputs	charge current is 0 outside 0–45 °C; always within [0, rated]
Monotonicity	more input → no more/less output	current never increases with SOC
Idempotence	doing it twice = once	<code>clear_all()</code> twice leaves the same state
Reference model	simple obviously-correct version agrees	a 10-line DTC debouncer vs the real manager
Never crashes	only documented errors occur	<code>CANFrame(...)</code> builds or raises <code>ValueError</code> , nothing else
Metamorphic	a known relation between runs	doubling the vehicle mass cannot shorten braking distance

```

from hypothesis import given, strategies as st
from autotest.bms import max_charge_current
from autotest.can import E2EStatus, e2e_check, e2e_protect

@given(st.binary(max_size=6), st.integers(0, 15), st.data())
def test_e2e_detects_any_single_bit_flip(payload, counter, data):
    protected = bytearray(e2e_protect(payload, counter))
    bit = data.draw(st.integers(0, len(protected) * 8 - 1)) # choose WHICH bit after seeing the
length
    protected[bit // 8] ^= 1 << (bit % 8)
    status, _ = e2e_check(bytes(protected))
    assert status is not E2EStatus.OK

@given(st.floats(0, 100), st.floats(-60, 120))
def test_charge_current_is_always_within_limits_and_zero_when_outside_temperature_window(soc, temp):
    i = max_charge_current(soc, temp)
    assert 0.0 <= i <= 100.0
    if temp < 0 or temp > 45:
        assert i == 0.0 # safety property

```

A **safety property** like the second one is worth a great deal: it covers *every* SOC and temperature, including ones no engineer would think to try.

A reference model

When the real code is intricate, write a **deliberately simple model** of the *specification* in the test and compare:

```

from hypothesis import given, strategies as st
from autotest.diagnostics import DiagnosticManager

@given(st.lists(st.booleans(), max_size=40), st.integers(1, 4), st.integers(1, 4))
def test_dtc_matches_reference_model(history, fail_n, heal_n):
    fails = passes = 0
    active = stored = False
    for failed in history:           # the SPEC in ten lines
        if failed:
            fails, passes = fails + 1, 0
            if fails >= fail_n:
                active = stored = True
        else:
            passes, fails = passes + 1, 0
            if passes >= heal_n:
                active = False
    mgr = DiagnosticManager(fail_threshold=fail_n, heal_threshold=heal_n)
    for failed in history:
        mgr.report("P0100", failed)
    assert (mgr.active_codes() == ["P0100"]) == (active and bool(history))
    assert (mgr.stored_codes() == ["P0100"]) == (stored and bool(history))

```

The model must come from the *requirement*, not from reading the code — otherwise you test the code against itself.

10.5 Steering the search

- `@settings(max_examples=500, deadline=None)` — more examples; disable the per-example time limit for slow code.
- `@example(80.0, 0.0)` — always include a specific edge case in addition to random ones.
- `assume(condition)` — discard inputs that do not satisfy a precondition. **Use sparingly.**
- `--hypothesis-seed=N` — reproduce a run; `--hypothesis-show-statistics` — see how many inputs were generated and discarded.

The filtering anti-pattern. Writing `assume(abs(a - b) < 0.01)` on two independent floats throws away nearly *every* input, and Hypothesis aborts with `FailedHealthCheck(filter_too_much)`. The fix is to **generate valid data directly** — here, a base value and a small *delta*:

```

from hypothesis import given, strategies as st
from autotest.bms import max_charge_current

@given(st.floats(0, 99.99), st.floats(0, 0.01))
def test_taper_is_continuous(soc, delta):
    assert abs(max_charge_current(soc, 25) - max_charge_current(min(soc + delta, 100), 25)) < 1.0

```

10.6 Fuzzing

Fuzzing feeds large volumes of malformed, unexpected or random data to an interface to provoke crashes, hangs and security problems. It is property-based testing with one property: **"the system survives and fails safely."**

Style	How inputs are made	Tools
Dumb / random	random bytes	a seeded loop (below)
Mutation-based	flip bits in valid examples (real traces)	custom scripts, AFL

Style	How inputs are made	Tools
Generation-based	build structured inputs from a grammar/spec	Hypothesis strategies, protocol fuzzers
Coverage-guided	evolve inputs that reach new code	AFL++, libFuzzer, Atheris (Python)

For an automotive diagnostic server the oracle is: *for any request, the server returns a **well-formed** response (positive, or 7F <sid> <nrc>), never raises, never hangs.*

```
import random
from autotest.diagnostics import DiagnosticManager

rng = random.Random(1234) # SEEDED: a failure can be replayed exactly
mgr = DiagnosticManager()
for _ in range(5000):
    blob = bytes(rng.randrange(256) for _ in range(rng.randrange(0, 12)))
    response = mgr.handle_request(blob)
    assert isinstance(response, bytes) and response
    assert response[0] in (0x7F, 0x59, 0x54, 0x62)
    if response[0] == 0x7F:
        assert len(response) == 3
print("5000 random requests: all answers well-formed")
```

Fuzzing does **not** prove security — it finds what it happens to reach. It is one layer next to design review and threat analysis (ISO/SAE 21434).

10.7 Stateful and model-based testing (taste)

Hypothesis can also generate **sequences of operations** against a model (`RuleBasedStateMachine`): random "power on / set / brake / ..." steps with invariants checked after every step — Chapter 9's exhaustive sequences, scaled to arbitrary length. It is the natural next step for state machines with more than a handful of events.

10.8 Strengths, weaknesses, advice

Strengths	Weaknesses
finds cases nobody listed (NaN, extremes, odd combinations)	needs <i>properties</i> , which require thought
readable minimal counter-examples (shrinking)	slower than a few examples; non-deterministic unless seeded/database-backed
failures are remembered and replayed	weak for "exact value" requirements

Combine the two: **examples document and pin specific requirements; properties search for the surprises.** Always keep a fixed seed or the example database in CI so a failure reproduces.

Check your understanding

1. Name the property pattern: "doubling a vehicle's mass never shortens its braking distance".
2. Why is " `result == the code computed again inside the test` " a poor property?
3. Hypothesis raises `FailedHealthCheck(filter_too_much)` . What went wrong and how do you fix it?
4. What is the single oracle used when fuzzing a diagnostic server?

Lab 7 — Property-based testing with Hypothesis

Duration: 3 h · **Type:** generative testing · **Code under test:** signal codec, E2E, `max_charge_current`

Why it matters

Example-based tests check the cases *you* thought of. Property-based tests state a rule that must hold for **all** inputs; Hypothesis generates hundreds of cases, then **shrinks** a failure to the smallest counter-example. It routinely finds the `0x7FF` vs `0x800` kind of bug nobody listed.

Learning objectives

Find properties (round-trip, invariant, idempotence, oracle/model, "never crashes"); write strategies (`st.floats`, `st.builds`, `st.data`); control the search (`@settings`, `@example`, `assume`); read a shrunk failure.

Property catalogue (automotive flavour)

Pattern	Example
Round trip	<code>decode(encode(x)) ≈ x</code> within one resolution step
Detection guarantee	CRC detects <i>any</i> single-bit flip
Safety invariant	never charge below 0 °C or above 45 °C, whatever SOC
Monotonicity	charge current never increases with SOC
Reference model	a 10-line reference DTC debouncer vs the real manager (exercise 2)
Crash-resistance	<code>CANFrame(...)</code> either builds or raises <code>ValueError</code> — nothing else

Run it

```
pytest labs/lab07_property_based -v
pytest labs/lab07_property_based --hypothesis-show-statistics
pytest labs/lab07_property_based --hypothesis-seed=0 # reproduce a CI failure
```

Guided tour

- `test_wrong_property_resolution_is_exact` is an `xfail` — remove the marker and read how Hypothesis shrinks a failure.
- The `assume()` anti-pattern: our first draft filtered two independent floats and Hypothesis raised `FailedHealthCheck(filter_too_much)`. The fix — generate the *delta* directly — is the rule: **generate valid data; don't filter invalid data.**

Exercises — `exercise_lab07.py`

Run: `python course.py exercise 7` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

Properties of `slip_ratio` · DTC debouncer vs a reference model · a `st.builds(CANFrame, ...)` strategy.

Debrief

- Why is a round-trip property weaker than an example with a known byte string? What does each catch?
- Hypothesis stores failures in `.hypothesis/`. How should CI treat that directory?
- What makes a *good* property — and why is "output equals a re-implementation of the code" a bad one?

Worked example — source

labs/lab07_property_based/test_lab07_property_based.py

```
"""Lab 7 - property-based testing with Hypothesis.

Instead of hand-picking inputs, state a PROPERTY that must hold for ALL inputs and let
Hypothesis search (and shrink) counter-examples.
"""

import pytest
from hypothesis import assume, example, given, settings, strategies as st

from autotest.bms import max_charge_current
from autotest.can import CANFrame, CANMessage, E2EStatus, Signal, crc8, e2e_check, e2e_protect
from autotest.sensors import kmh_to_ms, ms_to_kmh

payloads = st.binary(min_size=0, max_size=6)

# ---- property 1: round trip (encode then decode == identity) -----
@pytest.mark.requirement("REQ-CAN-002")
@given(st.floats(min_value=0, max_value=300, allow_nan=False))
def test_wheel_speed_round_trip_within_one_resolution_step(speed):
    sig = Signal("speed", 0, 16, factor=0.01, minimum=0, maximum=300)
    assert sig.to_physical(sig.to_raw(speed)) == pytest.approx(speed, abs=0.005 + 1e-9)

@given(st.integers(0, 65535), st.integers(-40, 215))
def test_message_round_trip_for_any_raw_values(rpm_raw, temp_raw):
    msg = CANMessage(1, "m", 4, (Signal("rpm", 0, 16, 0.25), Signal("t", 16, 8, offset=-40)))
    values = {"rpm": rpm_raw * 0.25, "t": float(temp_raw)}
    assert msg.decode(msg.encode(values)) == pytest.approx(values)

@given(st.floats(min_value=-1e4, max_value=1e4))
def test_unit_conversion_round_trip(kmh):
    assert ms_to_kmh(kmh_to_ms(kmh)) == pytest.approx(kmh)

# ---- property 2: end-to-end protection detects every corruption we can generate -----
@pytest.mark.requirement("REQ-CAN-003")
@given(payloads, st.integers(0, 15))
def test_e2e_accepts_what_it_protects(payload, counter):
    status, out = e2e_check(e2e_protect(payload, counter), counter)
    assert status is E2EStatus.OK and out == payload

@pytest.mark.requirement("REQ-CAN-003")
```

```

@given(payloads, st.integers(0, 15), st.data())
def test_e2e_detects_any_single_bit_flip(payload, counter, data):
    protected = bytearray(e2e_protect(payload, counter))
    bit = data.draw(st.integers(0, len(protected) * 8 - 1))
    protected[bit // 8] ^= 1 << (bit % 8)
    status, _ = e2e_check(bytes(protected))
    assert status is not E2EStatus.OK

@given(st.binary(max_size=8))
def test_crc_is_always_one_byte_and_deterministic(data):
    assert 0 <= crc8(data) <= 255 and crc8(data) == crc8(data)

# ---- property 3: safety invariants of the BMS -----
@pytest.mark.requirement("REQ-BMS-001")
@given(st.floats(0, 100), st.floats(-60, 120))
def test_charge_current_is_always_within_physical_limits(soc, temp):
    i = max_charge_current(soc, temp)
    assert 0.0 <= i <= 100.0
    if temp < 0 or temp > 45:
        assert i == 0.0 # safety property: never charge outside the temperature window

@given(st.floats(0, 100), st.floats(0, 100), st.floats(10, 45))
def test_charge_current_never_increases_with_soc(a, b, temp):
    low, high = sorted((a, b))
    assert max_charge_current(high, temp) <= max_charge_current(low, temp) + 1e-9

# ---- property 4: validation never crashes in unexpected ways -----
@given(st.integers(-10_000, 0x30000000), st.booleans(), st.binary(max_size=12))
def test_canframe_either_builds_or_raises_valueerror(arb_id, ext, data):
    try:
        frame = CANFrame(arb_id, data, ext)
    except ValueError:
        return
    assert frame.dlc == len(data) <= 8 and 0 <= frame.arbitration_id <= (0x1FFFFFFF if ext else
0x7FF)

# ---- steering Hypothesis: @example, settings, assume() (and why NOT to over-filter) -----
# Anti-pattern: `assume(abs(a - b) < 0.01)` on two independent floats rejects ~100 % of inputs
# and Hypothesis aborts with FailedHealthCheck(filter_too_much). Generate the SMALL DELTA directly.
@settings(max_examples=200, deadline=None)
@given(st.floats(0, 99.99), st.floats(0, 0.01))
@example(80.0, 0.0) # always re-test this edge case, in addition to random ones
def test_taper_is_continuous_at_80_percent(soc, delta):
    assume(soc + delta <= 100) # cheap, rarely-triggered filter: fine
    assert abs(max_charge_current(soc, 25) - max_charge_current(soc + delta, 25)) < 1.0

# ---- the point of shrinking: a deliberately wrong property, kept as a documented xfail ----
@pytest.mark.xfail(strict=True, reason="demo: Hypothesis shrinks to the smallest counter-example")
@given(st.floats(min_value=0, max_value=300))
def test_wrong_property_resolution_is_exact(speed):
    sig = Signal("speed", 0, 16, factor=0.01)
    assert sig.to_physical(sig.to_raw(speed)) == speed # false: quantisation error

```

Exercise skeleton — source

labs/lab07_property_based/exercise_lab07.py

```

"""Lab 7 exercises - properties with Hypothesis.

Run it:      python course.py exercise 7
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
               test; the test then passes only if what you wrote is correct.
Done when:   `python course.py progress` shows every test of Lab 7 passing.

```

```

Hypothesis crash course: @given(strategy, ...) generates many inputs for the test arguments.
    st.floats(min_value=0.6, max_value=100)    st.lists(st.booleans(), max_size=40)    st.integers(1, 4)
    """
from hypothesis import given, strategies as st

from autotest.abs import slip_ratio
from autotest.can import CANFrame, E2EStatus, e2e_check, e2e_protect
from autotest.diagnostics import DiagnosticManager
from autotest.learn import todo

speeds = st.floats(min_value=0, max_value=100, allow_nan=False)

@given(speeds, speeds)
def test_slip_is_always_between_0_and_1(vehicle, wheel):
    """Exercise 1a - property: whatever the two speeds are, slip_ratio(vehicle, wheel) is within [0,
1].

    Hint: assert 0.0 <= slip_ratio(vehicle, wheel) <= 1.0
    """
    todo("assert the range property")

def test_free_rolling_has_no_slip():
    """Exercise 1b - property: if the wheel runs as fast as the car, slip is 0 (for speeds above 0.5
m/s).

    Hint: add the decorator @given(st.floats(min_value=0.6, max_value=100)) and an argument `v`.
    """
    todo("write the property: slip_ratio(v, v) == 0")

def test_locked_wheel_is_full_slip():
    """Exercise 1c - property: a stopped wheel (0 m/s) on a moving car (> 0.5 m/s) is slip 1.
    Hint: same decorator as 1b; assert slip_ratio(v, 0.0) == 1.0
    """
    todo("write the property: slip_ratio(v, 0) == 1")

def test_dtc_matches_reference_model():
    """Exercise 2 - a REFERENCE MODEL. Generate pass/fail histories and compare DiagnosticManager
with a ten-line model
    of the SPEC: a code becomes active+stored after `fail_n` CONSECUTIVE failures, and stops being
active after `heal_n`
consecutive passes (it stays stored).

    Hint 1: decorate with @given(st.lists(st.booleans(), max_size=40), st.integers(1, 4),
st.integers(1, 4))
and take arguments (history, fail_n, heal_n).
    Hint 2: model: keep two counters (fails, passes) and two flags (active, stored); loop over the
history.
    Hint 3: mgr = DiagnosticManager(fail_threshold=fail_n, heal_threshold=heal_n); for failed in
history: mgr.report("P0100", failed)
then compare mgr.active_codes() == ["P0100"] with your model's `active` (and
`stored_codes`) with `stored`.
    Careful: with an EMPTY history the code was never reported, so both lists are empty.
    """
    todo("write the model and the property")

def test_custom_strategy():
    """Exercise 3 - build your own strategy for valid CAN frames and use it.
    Hint 1: frames = st.builds(CANFrame, arbitration_id=st.integers(0, 0x7FF),
data=st.binary(max_size=8))
    Hint 2: property: for every frame, e2e_check(e2e_protect(frame.data[:6], 3), 3) returns
E2EStatus.OK and the same payload.
    """
    todo("write the strategy and the property")

```

Chapter 11 — Robustness and fault injection

11.1 Functional correctness is not enough

A function can be perfectly correct for valid input and still be dangerous because of what it does with **invalid, unexpected or hostile** input and **partial failure** of its surroundings. Safety engineers ask a different question from "does it work?": "**how does it fail?**"

- **Fail-safe** — on failure, go to a state that cannot cause harm (charging current $\rightarrow$ 0 A).
- **Fail-operational** — keep (degraded) service where stopping is itself dangerous (steering assistance continues in limp-home mode).
- **Fail-silent** — stop producing output so others can detect the loss.

Robustness testing verifies the *safety mechanisms* that implement these policies: input plausibility checks, E2E protection, timeouts, watchdogs, debounced diagnostics, safe states.

11.2 Fault models: what can go wrong

You cannot inject "everything"; you inject a **fault model** — a defined catalogue of failures. A useful starter catalogue:

Where	Fault	Real-world cause	Typical expected reaction
Sensor	stuck-at (constant value)	frozen ADC, sensor fault	plausibility check, DTC
	short to ground / open circuit / short to supply	wiring	out-of-range reading $\rightarrow$ fault, substitute value
	noise, drift, offset (e.g. +5 km/h bias)	ageing, calibration	filtering, cross-checks
	NaN / out-of-range value	software, conversion	reject, safe value
Bus	corruption (bit flip)	EMI	CRC rejects frame
	loss (single, burst)	bus-off, overload	timeout, alive counter
	duplication, delay, re-ordering	gateway, congestion	alive counter, freshness check
	babbling idiot (flood)	stuck sender	filtering, bus guardian
Power / time	brown-out, reset mid-operation	start/stop	defined recovery, no half-written data
	clock jump, counter wrap	long run time	wrap-safe arithmetic
Software	exception, timeout, NaN	bugs	containment, safe state

Each row is a test idea; each "expected reaction" is the oracle.

11.3 Injecting faults at the sensor: the decorator

A **fault-injection decorator** wraps a healthy source and corrupts it on demand — keeping the test setup identical for every fault:

```

import pytest
from autotest.sensors import SensorFault, TemperatureSensor

class ConstADC:
    def __init__(self, v): self.v = v
    def read(self): return self.v

class FaultyADC:
    """Behaves normally for `after` reads, then injects the chosen fault."""
    def __init__(self, inner, mode, after=0):
        self.inner, self.mode, self.after, self.n, self.last = inner, mode, after, 0, None
    def read(self):
        value = self.inner.read(); self.n += 1
        if self.n <= self.after:
            self.last = value
            return value
        return {"stuck": self.last, "short_gnd": 0, "open": 4095}[self.mode]

for mode, message in [("short_gnd", "ground"), ("open", "open circuit")]:
    sensor = TemperatureSensor(FaultyADC(ConstADC(2000), mode, after=2))
    sensor.read(); sensor.read() # healthy period
    with pytest.raises(SensorFault, match=message): # then the fault must be REPORTED
        sensor.read()
print("hard faults are reported, not converted into a temperature")

```

The *stuck-at* mode shows the value of tests as **documentation of gaps**: today a stuck sensor looks perfectly healthy to `TemperatureSensor` — a plausibility check is missing. A test can state that honestly (and drive the implementation: *test first*, then add "100 identical readings $\Rightarrow$ `SensorFault('stuck')`").

11.4 Injecting faults at the bus, reproducibly

Random fault injection is powerful *if it is reproducible*: use a **seeded** random generator and print the seed.

```

import random
from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.clock import FakeClock
from autotest.ecu import InstrumentCluster, WheelSpeedECU

def run(seed):
    rng, bus, clock = random.Random(seed), VirtualCANBus(), FakeClock()
    ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)

    def flip_a_bit_sometimes(frame):
        if rng.random() < 0.3: # 30 % of frames get one bit
            data = bytearray(frame.data)
            data[rng.randrange(len(data))] ^= 1 << rng.randrange(8)
            return CANFrame(frame.arbitration_id, bytes(data))
        return frame

    bus.add_fault(flip_a_bit_sometimes)
    sent = set()
    for _ in range(200):
        speed = float(rng.randrange(0, 250))
        sent.add(f"{speed:.0f}")
        ecu.send_speed(speed)
        clock.advance(0.01)
        shown = cluster.displayed_speed()
        assert shown == "---" or shown in sent, f"seed {seed}: invented value {shown}" # the SAFETY

    for seed in range(5):
        run(seed)
    print("under random bit errors the cluster never displayed an invented speed")

```

The oracle is a **safety property**, not an exact value: *whatever is displayed was actually sent*. That is exactly the guarantee E2E protection is supposed to give.

11.5 Negative tests: invalid input

Negative testing checks that invalid input is **rejected cleanly** (specific exception or error code), not silently accepted or crashing in an uncontrolled way. Be specific about the expected error —

`pytest.raises(ValueError, match=...)` — because "raises *something*" also passes for a typo.

11.6 A case study: NaN

NaN ("not a number") is the classic hostile numeric. Every comparison with NaN is `False` — including `<` and `>` — so range checks written as `if x < low or x > high` let it **fall through**:

```
import math
from autotest.bms import BatteryManagementSystem, max_charge_current

print("charge current at NaN temperature:", max_charge_current(50, math.nan), "A")
print("fault for a NaN cell voltage      :", BatteryManagementSystem().evaluate([3.7, math.nan, 3.7],
25))
```

At the time of writing this prints **100.0 A** and `BMSFault.NONE`: an *unknown* temperature allows full-rate charging, and an *unknown* cell voltage reports "healthy". Neither was in any requirement, yet both are safety-relevant — the kind of defect only a deliberate hostile-input probe finds (Lab 8). The correct policy is *fail-safe*: **unknown is not OK**. Either reject the input with an error or treat it as the most restrictive state.

Pinning a known defect with `xfail(strict=True)` keeps the suite green *today* and turns it red the day the bug is fixed, so the marker cannot be forgotten.

11.7 Recovery and "healing"

Safety mechanisms must also **recover**. Test the full cycle: *normal* → *fault* → *detection* → *safe state* → *fault removed* → *recovery*, and what the system **remembers** afterwards. In our diagnostics, a DTC becomes *inactive* after enough healthy cycles but remains *stored* as history until cleared — a requirement worth its own test.

11.8 Where faults are injected in industry

Level	Mechanism
Software (this course)	decorators, hooks, mocks, mutating inputs
Bus	gateway or "rest-bus" simulator that drops/alters frames
HIL	fault-insertion units switch the pins of a real ECU (open circuit, short to ground/battery)
Vehicle	disconnecting connectors, controlled hazards on a proving ground

Software-level injection is cheap, safe and repeatable, so do the bulk of it there; reserve HIL for physical effects.

11.9 Principles

- **Define the fault model and the expected reaction first** — otherwise "robust" means nothing.
- **Assert on safety properties** (never unsafe output) rather than exact values.
- **Seed and log** randomness; every failure must be replayable.
- **Verify detection, reaction *and* recovery** (and that diagnostics recorded it).
- Treat inputs from *anywhere outside the module* — sensors, buses, files, other software — as untrusted.

Check your understanding

1. A function raises `ValueError` for NaN in one build and returns `0.0` in another. Are both acceptable for a charge-current limit? What is not acceptable?
2. What is the oracle in the "random bit errors" test, and why not an exact displayed value?
3. Why is `try: f(x) except ValueError: pass` a weak negative test?
4. List the three phases a recovery test should verify after a fault is injected and then removed.

Lab 8 — Robustness: negative testing, fault injection and fuzzing

Duration: 3 h · **Type:** robustness / fault-injection / fuzz · **Code under test:** sensor, bus, cluster, `DiagnosticManager.handle_request`, BMS

Why it matters

Functional correctness answers "does it work?". Robustness answers "**does it fail safely?**" — the question behind ISO 26262 safety mechanisms, ISO 21434 cybersecurity and the UDS diagnostic port that anybody with an OBD dongle can reach.

Learning objectives

Inject sensor faults with a decorator (stuck, short, open); inject bus faults with seeded randomness; fuzz an input parser and assert **universal** properties (never raises, always answers with a valid response); recognise hostile numerics (NaN, $\pm\text{inf}$); verify recovery and DTC history.

Techniques

Technique	Where
Fault-injection decorator (<code>FaultyADC</code>)	wraps any ADC
Bus fault hook with seeded RNG	<code>test_cluster_never_displays_an_invented_speed...</code>
Fuzzing: Hypothesis and a plain seeded loop	UDS handler
Documented gaps	stuck sensor is <i>not</i> detected today — the test says so
Strict <code>xfail</code> for known defects	BUG-201 / BUG-202 below

Two real defects found by this lab

- **BUG-201** `max_charge_current(50, nan)` returns **100 A** — NaN fails every comparison, so no limit applies. *Safety-relevant*.
- **BUG-202** `evaluate([3.7, nan, 3.7], 25)` reports `NONE` (healthy).

They are pinned with `xfail(strict=True)` : the suite stays green now and goes **red when someone fixes the bug**, forcing the marker to be removed. `solutions/bms_nan_fix.patch` is the verified fix.

Run it

```
pytest labs/lab08_robustness -v -rx
```


Exercises — exercise_lab08.py

Run: `python course.py exercise 8` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

Fix BUG-201/202 and clean up the markers · TDD a "stuck sensor" plausibility check · model a 5-frame burst loss and show recovery.

Debrief

- What does "fail-safe" mean for a charge-current limit when the temperature input is invalid?
- Why print/seed the RNG in fuzz tests?
- Fuzzing found nothing in the UDS handler. Does that prove it is secure? What *would* you need?

Worked example — source

labs/lab08_robustness/test_lab08_robustness.py

```
"""Lab 8 - robustness: negative tests, fault injection and fuzzing.

Question changes from "does it work?" to "does it fail SAFELY?".
"""
import math
import random

import pytest
from hypothesis import given, settings, strategies as st

from autotest.bms import BatteryManagementSystem, BMSFault, max_charge_current
from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.diagnostics import DiagnosticManager
from autotest.ecu import InstrumentCluster, WheelSpeedECU
from autotest.sensors import SensorFault, TemperatureSensor

pytestmark = pytest.mark.robustness

# ---- 1. sensor fault injection: wrap a healthy source and corrupt it -----
class FaultyADC:
    """Decorator around any ADC that injects classic automotive sensor faults."""

    def __init__(self, inner, mode, after=0):
        self.inner, self.mode, self.after, self.n, self.last = inner, mode, after, 0, None

    def read(self):
        value = self.inner.read()
        self.n += 1
        if self.n <= self.after:
            self.last = value
            return value
        if self.mode == "stuck":          # stuck-at last good value
            return self.last
        if self.mode == "short_gnd":
            return 0
        if self.mode == "open":
            return 4095
        raise AssertionError(self.mode)

class ConstADC:
    def __init__(self, v):
        self.v = v
```

```

def read(self):
    return self.v

@pytest.mark.parametrize("mode, message", [("short_gnd", "ground"), ("open", "open circuit")])
def test_hard_faults_are_reported_not_converted_to_a_temperature(mode, message):
    sensor = TemperatureSensor(FaultyADC(ConstADC(2000), mode, after=2))
    sensor.read(), sensor.read()
    with pytest.raises(SensorFault, match=message):
        sensor.read()

def test_stuck_sensor_is_NOT_detected_today_documenting_a_gap():
    sensor = TemperatureSensor(FaultyADC(ConstADC(2000), "stuck", after=1))
    values = {sensor.read() for _ in range(50)}
    assert len(values) == 1 # a stuck value looks perfectly healthy -> a plausibility check is
missing

# ---- 2. bus fault injection with a seeded RNG (reproducible!) -----
@pytest.mark.parametrize("seed", range(5))
def test_cluster_never_displays_an_invented_speed_under_bit_errors(seed, clock):
    rng = random.Random(seed)
    bus = VirtualCANBus()
    ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)

    def flip_a_bit_sometimes(frame):
        if rng.random() < 0.3:
            data = bytearray(frame.data)
            data[rng.randrange(len(data))] ^= 1 << rng.randrange(8)
            return CANFrame(frame.arbitration_id, bytes(data))
        return frame

    bus.add_fault(flip_a_bit_sometimes)
    sent = Set()
    for i in range(200):
        speed = float(rng.randrange(0, 250))
        sent.add(f"{speed:.0f}")
        ecu.send_speed(speed)
        clock.advance(0.01)
        shown = cluster.displayed_speed()
        assert shown == "--" or shown in sent, f"frame {i}: displayed invented value {shown}"

def test_duplicate_frames_are_rejected_by_alive_counter(clock):
    bus = VirtualCANBus()
    cluster = InstrumentCluster(bus, clock)
    ecu = WheelSpeedECU(bus)
    ecu.send_speed(50)
    duplicate = bus.log[-1]
    bus.send(duplicate) # replayed frame, same counter
    assert cluster.diag.failure_count("U0100") == 1

# ---- 3. fuzzing the diagnostic interface: random bytes in, SANE bytes out -----
@settings(max_examples=500, deadline=None)
@given(st.binary(max_size=20))
@pytest.mark.requirement("REQ-DIAG-002")
def test_uds_handler_never_raises_and_always_answers_with_a_valid_response(request_bytes):
    response = DiagnosticManager().handle_request(request_bytes)
    assert isinstance(response, bytes) and response
    assert response[0] in (0x7F, 0x59, 0x54, 0x62)
    if response[0] == 0x7F:
        assert len(response) == 3

@pytest.mark.requirement("REQ-DIAG-002")
@given(st.integers(0, 255).filter(lambda s: s not in (0x14, 0x19, 0x22)), st.binary(max_size=6))
def test_unknown_service_gets_negative_response_0x11(service, rest):
    assert DiagnosticManager().handle_request(bytes([service]) + rest) == bytes([0x7F, service,
0x11])

```

```

def test_seeded_random_fuzz_is_reproducible_in_ci():
    """Plain-pytest fuzzing: seed printed on failure -> anyone can replay it."""
    rng = random.Random(1234)
    mgr = DiagnosticManager()
    for _ in range(2000):
        blob = bytes(rng.randrange(256) for _ in range(rng.randrange(0, 12)))
        mgr.handle_request(blob)

# ---- 4. hostile numeric input: NaN / inf -----
@pytest.mark.parametrize("bad", [math.inf, -math.inf])
def test_infinite_temperature_is_handled_safely(bad):
    assert max_charge_current(50, bad) == 0.0

@pytest.mark.xfail(strict=True, reason="BUG-201: NaN temperature falls through every comparison ->
FULL charge current")
def test_nan_temperature_must_not_allow_charging():
    try:
        current = max_charge_current(50, math.nan)
    except ValueError: # rejecting the input is as safe as returning 0 A
        return
    assert current == 0.0

@pytest.mark.xfail(strict=True, reason="BUG-202: NaN cell voltage is never > max or < min ->
reported healthy")
def test_nan_cell_voltage_must_raise_a_fault_or_error():
    try:
        fault = BatteryManagementSystem().evaluate([3.7, math.nan, 3.7], 25)
    except ValueError:
        return
    assert fault is not BMSFault.NONE

# ---- 5. recovery: the system must come back after the fault is gone -----
def test_dtc_heals_after_enough_good_cycles():
    d = DiagnosticManager(fail_threshold=2, heal_threshold=3)
    for _ in range(2):
        d.report("U0121", failed=True)
    assert d.active_codes() == ["U0121"]
    for _ in range(3):
        d.report("U0121", failed=False)
    assert d.active_codes() == [] and d.stored_codes() == ["U0121"] # history is kept

```

Exercise skeleton — source

labs/lab08_robustness/exercise_lab08.py

```

"""Lab 8 exercises - robustness: fix real bugs, TDD a safety check, model a burst error.

Run it:      python course.py exercise 8
How it works: Exercise 1 asserts the SAFE behaviour and fails because of a real bug in src/ - fix
the code.

Exercises 2 and 3 start with todo(...) lines: delete them when you start on the
test.

Done when:   `python course.py progress` shows every test of Lab 8 passing.
"""
import math

import pytest

from autotest.bms import BatteryManagementSystem, BMSFault, max_charge_current
from autotest.bus import VirtualCANBus
from autotest.ecu import InstrumentCluster, WheelSpeedECU
from autotest.learn import todo
from autotest.sensors import SensorFault, TemperatureSensor

```

```

# -----
# Exercise 1 - FIX TWO REAL DEFECTS in src/autotest/bms.py.
# These tests are already written and fail today. They describe the FAIL-SAFE behaviour: an unknown
# (NaN) input must never be treated as "OK".
# Hint 1: every comparison with NaN is False, so `temp_c < 0 or temp_c > 45` lets NaN through.
# Hint 2: math.isnan(x) detects it. Reject the input with ValueError (or return the most restrictive
result).
# Afterwards: open test_lab08_robustness.py, delete the two xfail(strict=True) markers (they now go
RED
# on purpose - a strict xfail that passes is a failure: it forces you to clean up) and run the whole
suite.
# -----
def test_nan_temperature_must_not_allow_charging():
    try:
        current = max_charge_current(50, math.nan)
    except ValueError:
        # rejecting the input is as safe as returning 0 A
        return
    assert current == 0.0, "an unknown temperature must never allow charging"

def test_nan_cell_voltage_must_not_report_healthy():
    try:
        fault = BatteryManagementSystem().evaluate([3.7, math.nan, 3.7], 25)
    except ValueError:
        return
    assert fault is not BMSFault.NONE, "an unknown cell voltage must never be reported as healthy"

# -----
# Exercise 2 - TDD: a stuck sensor must be detected. Write the tests FIRST, then implement the
check.
# -----
class ConstADC:
    def read(self):
        return 2000

class Ramp:
    """An ADC whose value changes with every read - a healthy, moving signal."""

    def __init__(self):
        self.n = 1000

    def read(self):
        self.n += 1
        return self.n

class StuckAwareSensor(TemperatureSensor):
    """A TemperatureSensor that raises SensorFault('stuck') after 100 identical readings in a
row."""

    def read(self):
        todo("implement: call super().read(), count identical values, raise SensorFault('stuck') at
100")

    def test_stuck_sensor_is_detected():
        """Hint: with pytest.raises(SensorFault, match="stuck"): read 200 times from
StuckAwareSensor(ConstADC())"""
        todo("write the test (then make it pass by implementing StuckAwareSensor.read)")

    def test_a_moving_sensor_is_not_flagged():
        """Hint: the same loop with Ramp() must NOT raise. (A check that fires on healthy data is a
defect too.)"""
        todo("write the test")

# -----
# Exercise 3 - a BURST ERROR: five consecutive frames lost.
# -----
def test_burst_loss_and_recovery(bus, clock):

```

```

burst,
    """Show that (1) the cluster blanks during the silence, (2) it shows numbers again after the
    (3) the 'communication lost' DTC (U0121) is still STORED afterwards.

    Hint 1: ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock); send one good frame
    first.
    Hint 2: a hook that drops the first five frames: state = {"n": 0}; def drop_five(frame):
    state["n"] += 1; return None if state["n"] <= 5 else frame
    Hint 3: send 5 frames while advancing clock.advance(0.03) each; then cluster.displayed_speed()
    == "--" (twice, DTC debouncing needs 2 misses).
    Hint 4: afterwards send a few more frames; the first one after the burst is rejected by the
    alive counter (a resync),
        so check the display after 2-3 frames. Finally: "U0121" in cluster.diag.stored_codes().
    """
    todo("write the burst-loss scenario")

```

Part VI — Change, speed and simulation

Chapter 12 — Regression testing, golden files and replay

12.1 What regression means

A **regression** is behaviour that used to work and no longer does. Software changes constantly — new features, refactors, library updates, compiler upgrades — and every change can break something unrelated. **Regression testing** is re-running tests to detect that. Because a vehicle program lives for years and ships many variants, regression is where most automation pays for itself.

Three kinds of tests do regression duty:

1. The **whole existing suite**, run on every change (fast tests: every commit; slow ones: nightly).
2. **Bug-guard tests**: one test per fixed defect, so it can never silently return.
3. **Golden / snapshot and replay tests**: compare complex outputs with a trusted recording.

12.2 Bug-guard tests: the red–green discipline

When a defect is reported:

1. **Reproduce** it with a *failing* test (red). If you cannot reproduce it, you have not understood the report — it may even be invalid.
2. **Fix** the code until the test passes (green).
3. **Keep** the test forever, named after the ticket, with a docstring saying what broke and why.

```
from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.clock import FakeClock
from autotest.ecu import InstrumentCluster, WheelSpeedECU

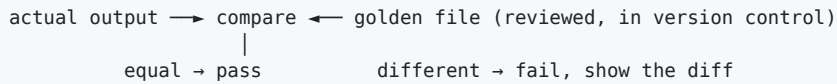
def test_bug_130_single_bit_error_must_cost_exactly_one_frame():
    """BUG-130: a CRC error left the expected alive counter unchanged, so the NEXT good
    frame failed the counter check too -> two lost frames and a DTC for ONE bit error."""
    bus, clock = VirtualCANBus(), FakeClock()
    ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)
    ecu.send_speed(10)
    bus.add_fault(lambda f: CANFrame(f.arbitration_id, bytes([f.data[0] ^ 1] + f.data[1:])))
    ecu.send_speed(20)                                     # corrupted in flight
    bus.clear_faults()
    ecu.send_speed(30)                                     # must be accepted immediately
    assert cluster.displayed_speed() == "30"
    assert cluster.diag.stored_codes() == []

test_bug_130_single_bit_error_must_cost_exactly_one_frame()
print("bug guard passes on the fixed code")
```

Prove the test is real: temporarily revert the fix — the test must go red. A bug-guard test that cannot fail is worthless (Chapter 15 generalises this idea).

12.3 Golden-file (snapshot / approval) testing

Some outputs are too large or intricate to assert field by field: a diagnostic report, a UDS session transcript, a rendered dashboard timeline. A **golden file** stores the *approved* output; the test compares fresh output with it:



The workflow depends on **discipline**:

- The first run *creates* the golden file; a human **reviews** it — it becomes a specification.
- When behaviour changes *intentionally*, regenerate it (`--update-golden`) and **review the diff in code review** — an updated golden file is a changed requirement, not a formality.
- Never "update to make it green" without understanding *why* it differs.

```
from pathlib import Path
from autotest.diagnostics import DiagnosticManager

def assert_matches_golden(path: Path, actual: str, update=False):
    if update or not path.exists():
        path.write_text(actual)
        return
    assert actual == path.read_text(), f"differs from {path.name}; inspect then update deliberately"

def test_dtc_report(tmp_path):
    d = DiagnosticManager(fail_threshold=2, heal_threshold=2)
    for _ in range(2):
        d.report("U0121", failed=True)          # confirmed
        d.report("P0300", failed=True)          # pending only
    golden = tmp_path / "dtc_report.txt"
    assert_matches_golden(golden, d.export_report()) # first run: creates the golden file
    assert_matches_golden(golden, d.export_report()) # second run: compares
    assert "U0121 active=True stored=True" in golden.read_text()
```

Make outputs deterministic before snapshotting: no timestamps, no memory addresses, sorted collections.
A golden file containing `2026-10-07 14:03:22` fails tomorrow.

12.4 Record and replay

Real vehicles produce **traces** (CAN logs in ASC/BLF/MDF format) of real drives. Replaying a recorded trace into a new software build and comparing the reaction with the previous release catches regressions nobody specified:

```
from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.clock import FakeClock
from autotest.ecu import InstrumentCluster, WheelSpeedECU

# 1. RECORD: a few frames, in the format <time_s> <can_id_hex> <data_hex>
rec_bus = VirtualCANBus()
ecu = WheelSpeedECU(rec_bus)
log, t = [], 0.0
for speed in (0, 20, 40, 60):
    ecu.send_speed(speed)
    f = rec_bus.log[-1]
    log.append(f"{t:.3f} {f.arbitration_id:03x} {f.data.hex()}")
    t += 0.01
```

```
# 2. REPLAY into a fresh system under test with a controlled clock
clock, bus = FakeClock(), VirtualCANBus()
cluster = InstrumentCluster(bus, clock)
seen = []
for line in log:
    ts, can_id, data = line.split()
    clock.advance(float(ts) - clock.now())
    bus.send(CANFrame(int(can_id, 16), bytes.fromhex(data)))
    seen.append(cluster.displayed_speed())

assert seen == ["0", "20", "40", "60"] # 3. COMPARE with the approved transcript
print("replay transcript:", seen)
```

Lab 9's recorded drive cycle contains a deliberate corrupted frame; replaying it exposed BUG-130 — a defect no one had specified a test for.

12.5 Choosing which tests to re-run

When the suite is large:

Strategy	Idea	Risk
Retest all	run everything	slow, but the safest
Selection	run only tests affected by the change (dependency analysis, coverage mapping)	misses indirect effects
Prioritisation	run most failure-prone / fastest tests first	still runs all eventually
Tiering	smoke on every commit, full unit/integration on merge, slow/HIL nightly	needs markers and discipline

pytest markers (`smoke` , `slow` , `performance`) implement tiering with one command-line flag.

12.6 Flaky tests: the enemy of regression

A **flaky** test sometimes fails with no code change. Flakiness destroys trust: people learn to re-run and ignore red. Causes and cures:

Cause	Cure
Real time (<code>sleep</code> , deadlines)	inject a fake clock
Randomness	seed it; log the seed
Test order / shared state	isolate with fresh fixtures; run in random order occasionally
Real I/O, network, shared files	fakes, <code>tmp_path</code>
Concurrency, race conditions	deterministic scheduling, explicit synchronisation
Resource limits on loaded CI	generous budgets; measure percentiles (Chapter 13)
Dirty environment (leftover files, ports)	per-test temp resources, cleanup

Never "fix" flakiness with retries or longer sleeps without finding the cause: a flaky test is often a *real* race condition in the product.

12.7 Test data and logs under change

Recorded logs and golden files are **assets** that age:

- Store them in version control next to the tests, with a short README on how they were produced and how to regenerate them.
- Interfaces evolve: when a message layout changes, regenerate or migrate logs deliberately.
- Keep a few **minimised** traces (smallest log that reproduces a bug) rather than only huge ones.

Check your understanding

1. In what order do you write the test and the fix for a reported defect, and what must you check about the test?
2. A golden-file test fails after a refactoring that should not change behaviour. What are your two options and which is almost always wrong?
3. Name three causes of flaky tests and the matching cure for each.
4. Why might replaying a recorded drive find defects that a requirement-based test suite does not?

Lab 9 — Regression, golden-file and log-replay testing

Duration: 2.5 h · **Type:** regression / snapshot / replay · **Code under test:** `InstrumentCluster`, `DiagnosticManager`

Why it matters

Software changes continuously over a 7-year programme; every release must not re-break what was fixed. Replaying **recorded drive cycles** (MDF/ASC logs) against new builds is how OEMs catch regressions that no one thought to specify.

Learning objectives

Write bug-guard tests named after tickets; compare complex output with **golden files**; update goldens deliberately (`--update-golden`) and review the diff; replay a recorded CAN log through the system.

The workflow

1. A defect is reported → write a **failing** test that reproduces it.
2. Fix → test passes → keep it forever (named `test_bug_NNN_...`, docstring says what broke).
3. Complex outputs (reports, transcripts, dashboards over a drive) → golden file.
4. Changing a golden file is a **reviewed code change**, never a reflex.

Run it

```
pytest labs/lab09_regression_golden -v
python tools/make_drive_log.py          # regenerate the recorded drive cycle
pytest labs/lab09_regression_golden --update-golden # only after reading the diff!
```

The finding behind this lab: BUG-130

Replaying `data/drive_cycle.log` (which contains one corrupted frame) showed the cluster discarding **two** frames, then setting DTC U0100: the CRC-failed frame did not advance the expected alive counter. The fix is in `ecu.py`; the guard is `test_bug_130_...`. Revert the fix (3 lines) and 3 tests go red — try it.

Exercises — `exercise_lab09.py`

Run: `python course.py exercise 9` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

Triage BUG-120 (reproduce first — the report may be wrong!) · golden transcript of a UDS session · mutate the log and read the diff.

Debrief

- What is the danger of `--update-golden` in a hurry? How does code review mitigate it?
- A golden file contains a timestamp. What happens in CI? How do you fix it?
- Where should recorded logs come from, and how do you keep them valid when the interface evolves?

Worked example — source

labs/lab09_regression_golden/test_lab09_regression_golden.py

```
"""Lab 9 - regression testing: bug-guard tests, golden files and log replay.

Rule: every defect that ever escaped gets a test BEFORE it is fixed. Golden files
protect complex outputs (reports, logs) that are tedious to assert field-by-field.

Create/refresh golden files deliberately with: pytest labs/lab09_regression_golden --update-golden
(then READ the git diff - an updated golden file is a reviewed change, not a formality)
"""
from pathlib import Path

import pytest

from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.clock import FakeClock
from autotest.diagnostics import DiagnosticManager
from autotest.ecu import InstrumentCluster, WheelSpeedECU

pytestmark = pytest.mark.regression
DATA = Path(__file__).parent / "data"

# ---- 1. bug-guard tests: name them after the ticket -----
def test_bug_087_alive_counter_wrap_15_to_0_was_rejected(clock):
    """BUG-087: after 16 frames the 4-bit counter wrapped and the cluster went blank."""
    bus = VirtualCANBus()
    ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)
    for i in range(40):
        ecu.send_speed(i)
        assert cluster.displayed_speed() == str(i), f"frame {i}"

def test_bug_093_crc_error_must_not_update_last_rx_time(clock):
    """BUG-093: garbage frames kept the timeout from expiring (cluster showed stale speed)."""
    bus = VirtualCANBus()
    cluster = InstrumentCluster(bus, clock)
    WheelSpeedECU(bus).send_speed(80)
    for _ in range(5):
        clock.advance(0.05)
        bus.send(CANFrame(0x1A0, b"\x00\x01\x02\x03\x04")) # invalid CRC
    assert cluster.displayed_speed() == "--"

# ---- 2. golden file for a text report -----
def test_dtc_report_matches_golden(golden):
    d = DiagnosticManager(fail_threshold=2, heal_threshold=2)
    for _ in range(2):
        d.report("U0121", failed=True) # confirmed
    d.report("P0300", failed=True) # pending only
    for _ in range(2):
        d.report("U0100", failed=True)
    for _ in range(2):
        d.report("U0100", failed=False) # confirmed then healed
    golden("dtc_report.txt", d.export_report())

# ---- 3. replaying a recorded drive cycle -----
-----
```

```

def replay(path: Path):
    """Feed a recorded log into a fresh cluster; return what the driver would have seen."""
    clock, bus = FakeClock(), VirtualCANBus()
    cluster = InstrumentCluster(bus, clock)
    seen = []
    for line in path.read_text(encoding="utf-8").splitlines():
        t, can_id, data = line.split()
        clock.advance(float(t) - clock.now())
        bus.send(CANFrame(int(can_id, 16), bytes.fromhex(data)))
        seen.append(f"{float(t):6.3f}s {cluster.displayed_speed():>3}")
    return "\n".join(seen) + "\n", cluster

def test_recorded_drive_cycle_shows_the_same_dashboard_as_last_release(golden):
    transcript, _ = replay(DATA / "drive_cycle.log")
    golden("drive_cycle_display.txt", transcript)

def test_replay_of_the_recording_raises_no_dtc():
    _, cluster = replay(DATA / "drive_cycle.log")
    assert cluster.diag.stored_codes() == [] # one recorded bit error is only debounced

def test_bug_130_single_bit_error_must_cost_exactly_one_frame(clock):
    """BUG-130 (found by the replay in this lab): a CRC error left the expected alive counter
    unchanged, so the NEXT good frame failed the counter check too -> two losses, and with
    fail_threshold=2 a DTC for what was a single bit error."""
    bus = VirtualCANBus()
    ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)
    ecu.send_speed(10)
    flip = lambda f: CANFrame(f.arbitration_id, bytes([f.data[0] ^ 1] + f.data[1:]))
    bus.add_fault(flip)
    ecu.send_speed(20) # corrupted in flight
    bus.clear_faults()
    ecu.send_speed(30) # must be accepted immediately
    assert cluster.displayed_speed() == "30"
    assert cluster.diag.stored_codes() == []

```

Exercise skeleton — source

labs/lab09_regression_golden/exercise_lab09.py

```

"""Lab 9 exercises - regression: triage a bug report, golden transcript, replay.

Run it:      python course.py exercise 9
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
test.
Done when:   `python course.py progress` shows every test of Lab 9 passing.

Note on golden files: the FIRST run of a golden test creates the file and fails on purpose (nothing
to compare
with yet). Read the new file carefully - it is now your specification - then run again.
"""
from autotest.can import CANFrame, e2e_protect
from autotest.diagnostics import DiagnosticManager
from autotest.ecu import WHEEL_SPEED_MSG, InstrumentCluster
from autotest.learn import todo

def test_bug_120_invalid_flag_shows_dashes(bus, clock):
    """Exercise 1 - you receive BUG-120: 'the cluster shows 0 instead of -- when the sender says
    valid=0'.

    Rule #1 of regression work: REPRODUCE IT WITH A TEST FIRST. (Be ready for a surprise.)

    Hint 1: payload = WHEEL_SPEED_MSG.encode({"speed_kmh": 0.0, "valid": 0}); data =
    e2e_protect(payload, 0)
    Hint 2: send it: bus.send(CANFrame(0x1A0, data)); then check cluster.displayed_speed()
    Hint 3: if your test PASSES, the bug is not reproducible: say so in a comment - the test stays
    as a guard,

```

```

        and the ticket is closed 'cannot reproduce' with this test as the evidence.
    """
    todo("reproduce BUG-120")

def test_uds_session_transcript(golden):
    """Exercise 2 - record a diagnostic session as a golden file `golden/uds_session.txt`:
    read DTCs, read VIN, clear DTCs, read DTCs again, an unknown service (hex bytes for each request
    and response).

        Hint 1: d = DiagnosticManager(fail_threshold=1); d.report("U0121", failed=True) # one stored
code
        Hint 2: requests: b"\x19\x02\xff" b"\x22\xF1\x90" b"\x14\xff\xff\xff"
b"\x19\x02\xff" b"\x3E\x00"
        Hint 3: build text lines like f"{label:<16} {req.hex(' ')} -> {d.handle_request(req).hex('
')}" and call golden("uds_session.txt", text)
        Then READ the generated file: do the response bytes match the UDS rules from Chapter 3 (positive
= SID + 0x40, negative = 7F ...)?
    """
    todo("record the session as a golden file")

    # Exercise 3 (no code) - edit labs/lab09_regression_golden/data/drive_cycle.log by hand: change one
speed value, run
    # `python course.py lab 9`, and read the diff pytest prints. Then undo the change with `git
checkout` on that file.
    # Write one sentence: what is the danger of blindly running pytest --update-golden ?

```

Chapter 13 — Performance, timing and resource testing

13.1 Non-functional requirements are requirements

A function that returns the right answer too late is wrong. In an ECU:

- A control task that needs 12 ms in a 10 ms cycle **misses its deadline** — a functional failure.
- A decoder that cannot keep up with a busy bus **drops frames**.
- A log that grows without bound **exhausts memory** after hours of driving.
- A search that is fine with 100 items but **quadratic** is unusable with 100 000.

Performance testing turns these into measurable, automated checks.

13.2 What to measure

Quantity	Meaning	Automotive example
Latency / execution time	time for one operation	one <code>control()</code> step
Throughput	operations per second	frames decoded per second
Jitter	variation of a periodic event	spread of the 10 ms task start time
Utilisation	execution time ÷ period	0.5 ms in a 10 ms task = 5 % CPU
WCET	worst-case execution time	upper bound used in schedulability analysis
Memory	RAM/stack/heap use, growth over time	log length, leaks
Scalability	how cost grows with input size	per-sample cost vs window length

Hard and soft deadlines

A **hard** real-time task must *never* miss its deadline (airbag, brake control): one miss is a failure. A **soft** task tolerates occasional misses with graceful degradation (infotainment). The strictness decides whether you test the **maximum** or a **high percentile**.

13.3 Statistics: never trust the mean

Timing data is skewed: most runs are fast, a few are slow (cache misses, garbage collection, a busy neighbour). The **mean hides the tail**.

Statistic	Tells you	Use for
mean	typical cost (misleading)	rough comparison only
median (p50)	the usual case	trend tracking
p95 / p99 / p99.9	the tail	soft deadlines, SLAs
max	the worst observation	hard deadlines (but only a lower bound on the true WCET)

13.4 A measurement helper

Good measurement discipline: **warm up** (first calls are slower), **repeat** many times, use a **monotonic high-resolution clock** (`time.perf_counter` , never `time.time`), and keep the **garbage collector** from pausing inside the sample.

```
import gc, statistics, time
from autotest.cruise import CruiseController

def measure(fn, repeat=2000, warmup=200):
    for _ in range(warmup):
        fn()
    gc.disable()
    try:
        out = []
        for _ in range(repeat):
            t0 = time.perf_counter()
            fn()
            out.append(time.perf_counter() - t0)
    finally:
        gc.enable()
    return out

cc = CruiseController()
cc.power_on(); cc.set(100)
samples = measure(lambda: cc.control(97.0, 0.01))
q = statistics.quantiles(samples, n=1000)
p99, worst = q[989], max(samples)
print(f"median {statistics.median(samples)*1e6:.1f} us    p99 {p99*1e6:.1f} us    max {worst*1e6:.1f} us")

CYCLE = 0.010                                     # a 10 ms control task
assert p99 < 0.10 * CYCLE                         # use at most 10 % of the cycle: 10x headroom
assert worst < CYCLE                             # and never miss the deadline
```

Rules for non-flaky performance tests

1. Assert **percentiles or maxima against generous budgets** (aim for $\geq 10\times$ headroom on a developer PC), never a mean against a tight number.
2. Prefer **relative** checks (scaling ratios, comparison to a baseline) to absolute milliseconds.
3. Warm up, repeat, and exclude set-up from the timed region.
4. Run them in a **separate CI stage** (`pytest -m performance`) so parallel jobs do not disturb them.
5. Remember: a host PC measures **algorithmic** cost. The real **WCET** must be established on the target hardware (measurement plus static analysis).

13.5 Throughput: can we keep up?

A 500 kbit/s CAN bus carries at most about **4 500 eight-byte frames per second** (111 bit-times per frame including intermission, before bit stuffing; stuffing only lowers the rate). If the receiver pipeline cannot decode that many per second, frames will be lost under load — so test it:

```
import time
from autotest.can import e2e_check, e2e_protect
from autotest.ecu import WHEEL_SPEED_MSG

frames = [e2e_protect(WHEEL_SPEED_MSG.encode({"speed_kmh": i % 300, "valid": 1}), i) for i in range(4500)]
t0 = time.perf_counter()
for i, raw in enumerate(frames):
    status, payload = e2e_check(raw, i)
```

```

WHEEL_SPEED_MSG.decode(payload)
elapsed = time.perf_counter() - t0
print(f"decoded one second of saturated bus traffic in {elapsed*1000:.1f} ms")
assert elapsed < 0.5 # generous: half a second for one second of
traffic

```

13.6 Scalability: complexity regressions

The most common real performance defect is code whose cost **grows with data size**. Catch it by measuring at two sizes and comparing the **ratio**, not absolute times:

```

import time
from autotest.tpms import LeakDetector

def per_call_cost(detector, fill, probe=200):
    for i in range(fill): # 1 kHz sampling: 'fill' ms of history inside the
        detector.add(i * 0.001, 230.0)
    t0 = time.perf_counter()
    for i in range(probe):
        detector.add((fill + i) * 0.001, 230.0)
    return (time.perf_counter() - t0) / probe

small = per_call_cost(LeakDetector(), 500)
large = per_call_cost(LeakDetector(), 8000)
print(f"16x more history -> {large / small:.1f}x more time per sample")
assert large / small > 1.5 # this detector recomputes max() over the whole
window: 0(window)

```

`LeakDetector.add` scans the entire 60 s window on every sample: at 1 kHz that is up to 60 000 elements per call. A monotonic-deque algorithm makes it $O(1)$ amortised, and the *same ratio test* then proves the fix — Lab 10's exercise.

13.7 Memory and resource tests

Python's `tracemalloc` shows how much memory is allocated and by whom. Use it to catch **leaks** and **unbounded growth**:

```

import tracemalloc
from autotest.bus import VirtualCANBus
from autotest.can import CANFrame

tracemalloc.start()
bus = VirtualCANBus()
for i in range(20_000):
    bus.send(CANFrame(0x100, bytes([i & 0xFF])))
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
print(f"{len(bus.log)} frames retained, {current / 1e6:.1f} MB")
assert len(bus.log) == 20_000 and current > 1_000_000 # BUG: the log is unbounded

```

The bus keeps every frame forever — fine for a short test, fatal in a gateway that runs for days. The cure is a **bounded buffer** (`collections.deque(maxlen=N)`) or opt-in logging, and the test that guards it asserts the *bound*.

Related resource checks: file-handle and thread leaks, stack depth (important on MCUs), and **soak tests** that run the system for hours.

13.8 Kinds of load testing

Test	Question
Load	does it meet its budgets at the <i>expected</i> maximum load?
Stress	what happens <i>beyond</i> the limit — does it degrade gracefully or fail unsafely?
Spike	can it absorb a sudden burst (a flood of frames)?
Soak / endurance	does it stay healthy for hours (leaks, drift, counter wrap)?

13.9 Finding the slow code: profiling

When a budget test fails, **profile** before optimising. `python -m cProfile -s cumtime script.py` shows where time goes; `pytest --durations=10` lists the suite's slowest tests (your own tests deserve performance monitoring too). Optimise the measured hot spot, then let the budget test confirm the gain.

Check your understanding

1. A task has a 10 ms period and 4 ms worst-case execution time. What is its utilisation, and what test would you write for the deadline?
2. Why is asserting `mean < 5 ms` a weak performance test?
3. How do you detect a quadratic algorithm without relying on absolute timings?
4. What does a host measurement tell you and not tell you about target WCET?

Lab 10 — Performance and real-time budget testing

Duration: 2.5 h · **Type:** performance / timing / memory · **Code under test:** `CruiseController.control`, `codec`, `LeakDetector`, `VirtualCANBus`

Why it matters

A control task that takes 12 ms in a 10 ms cycle is a functional failure. A decoder that cannot keep up with a saturated 500 kbit/s bus drops frames. A log that grows forever crashes the gateway after 3 days of driving.

Learning objectives

Build a measurement helper (warm-up, repeats, GC off, percentiles); express **budgets** instead of averages; test **scaling** (does cost grow with history?) with ratios, not wall-clock numbers; check **memory** with `tracemalloc`; keep performance tests from making CI flaky.

Rules for non-flaky performance tests

1. Assert on **p99 / max** versus a **generous** budget (10× headroom), never on a mean.
2. Prefer **relative** assertions (ratio between sizes) over absolute milliseconds.
3. Warm up, repeat, disable GC while measuring.
4. Run in a separate CI stage (`pytest -m performance`) so parallel jobs do not disturb it.
5. Real worst-case execution time is measured **on target hardware** — host timing only catches algorithmic regressions.

Run it

```
pytest labs/lab10_performance -v
pytest labs -m "not performance"      # exclude from the everyday loop
pytest --durations=10                  # the suite's own slowest tests
```

Two real scalability defects (exercises)

- `LeakDetector.add` recomputes `max()` over the whole 60 s window → **O(window)** per sample; at 1 kHz that is 60 000 elements per call.
- `VirtualCANBus.log` is unbounded → memory grows without limit.

The instructor solutions demonstrate both (ratio > 3 before, < 3 after; bounded deque).

Exercises — `exercise_lab10.py`

Run: `python course.py exercise 10` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

Scaling test + monotonic-deque fix · `tracemalloc` + bounded log · a `budget(seconds)` context manager.

Debrief

- Why is `time.time()` the wrong clock? Why disable GC?
- Your p99 is fine but max is 40× larger. Is that a problem for a hard real-time task? For a soft one?
- How would you move this test to the target ECU?

Worked example — source

labs/lab10_performance/test_lab10_performance.py

```
"""Lab 10 - performance and real-time budgets.

Rules for non-flaky performance tests in CI:
* assert on percentiles / worst cases against GENEROUS budgets (10x headroom), never means
* use time.perf_counter, warm up first, and repeat
* prefer RELATIVE assertions (scaling, ratio to a baseline) over absolute wall-clock numbers
* run them in a separate stage: pytest -m performance / pytest -m "not performance"
"""

import gc
import statistics
import time
import tracemalloc

import pytest

from autotest.can import CANMessage, Signal, crc8, e2e_check, e2e_protect
from autotest.cruise import CruiseController
from autotest.ecu import WHEEL_SPEED_MSG

pytestmark = pytest.mark.performance

def measure(fn, repeat=2000, warmup=200):
    """Return per-call durations in seconds."""
    for _ in range(warmup):
        fn()
    gc.disable() # avoid collector pauses polluting the sample
    try:
        out = []
        for _ in range(repeat):
            t0 = time.perf_counter()
            fn()
            out.append(time.perf_counter() - t0)
    finally:
        gc.enable()
    return out

def p(samples, pct):
    return statistics.quantiles(samples, n=1000)[int(pct * 10) - 1]

# ---- 1. real-time budget: the control task must fit its 10 ms cycle with margin -----
@pytest.mark.requirement("REQ-CC-003")
def test_cruise_control_step_meets_its_10ms_cycle_with_big_margin():
    cc = CruiseController()
    cc.power_on()
    cc.set(100)
    samples = measure(lambda: cc.control(97.0, 0.01))
    budget = 0.010 * 0.10 # use at most 10 % of the cycle time (1 ms)
    assert p(samples, 99) < budget, f"p99={p(samples, 99) * 1e6:.1f} us"
    assert max(samples) < 0.010, "worst case must never miss the cycle deadline"

# ---- 2. throughput: can we keep up with a saturated 500 kbit/s bus? -----
def test_decoder_keeps_up_with_a_saturated_500kbit_bus():
    """A 500 kbit/s bus carries at most ~4 500 eight-byte frames/s: 111 bits per frame including
    intermission, BEFORE bit stuffing. Stuffing only lengthens frames (worst case ~135 bits,
```

```

~3 700 frames/s), so 4 500 is a safe upper bound for the decoder to keep up with."""
frames = [e2e_protect(WHEEL_SPEED_MSG.encode({"speed_kmh": i % 300, "valid": 1}), i) for i in
range(4500)]
t0 = time.perf_counter()
for i, raw in enumerate(frames):
    status, payload = e2e_check(raw, i)
    WHEEL_SPEED_MSG.decode(payload)
elapsed = time.perf_counter() - t0
assert elapsed < 0.5, f"decoded 1 s of worst-case bus traffic in {elapsed:.3f} s"

# ---- 3. scaling: cost must not grow with history length -----
def test_crc_cost_scales_linearly_with_length():
    short, long_ = bytes(8), bytes(8 * 50)
    t_short = statistics.median(measure(lambda: crc8(short), repeat=500))
    t_long = statistics.median(measure(lambda: crc8(long_), repeat=500))
    ratio = t_long / t_short
    assert ratio < 50 * 3, f"50x more data cost {ratio:.0f}x more time (expected ~50x, linear)"

# ---- 4. memory: tracemalloc -----
--
def test_encoding_does_not_leak_memory():
    sig = CANMessage(1, "m", 2, (Signal("v", 0, 16, 0.1),))
    sig.encode({"v": 1.0}) # warm caches
    tracemalloc.start()
    before = tracemalloc.take_snapshot()
    for i in range(20000):
        sig.encode({"v": (i % 1000) * 0.1})
    after = tracemalloc.take_snapshot()
    tracemalloc.stop()
    growth = sum(s.size_diff for s in after.compare_to(before, "filename"))
    assert growth < 200_000, f"retained {growth} bytes after 20 000 encodes"

# ---- 5. finding the slow tests of the suite itself -----
-----
def test_timing_is_part_of_the_report():
    """Run `pytest --durations=10` to list the slowest tests - a free performance monitor."""
    assert True

```

Exercise skeleton — source

labs/lab10_performance/exercise_lab10.py

```

"""Lab 10 exercises - performance: scaling, memory, a budget helper.

Run it:      python course.py exercise 10
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
test.

Done when:   `python course.py progress` shows every test of Lab 10 passing.
Timing tests: use GENEROUS limits and ratios, never tight absolute numbers (see Chapter 13).
"""

import contextlib
import time
import tracemalloc
from collections import deque

import pytest

from autotest.bus import VirtualCANBus
from autotest.can import CANFrame
from autotest.learn import todo
from autotest.tpms import LeakDetector

def per_call_cost(detector, fill, probe=300):
    """Helper (given): average seconds per add() after `fill` samples (1 kHz sampling) are already
in the window."""

```

```

    for i in range(fill):
        detector.add(i * 0.001, 230.0)
    t0 = time.perf_counter()
    for i in range(probe):
        detector.add((fill + i) * 0.001, 230.0)
    return (time.perf_counter() - t0) / probe

def test_leak_detector_cost_grows_with_window_fill():
    """Exercise 1 - LeakDetector.add() recomputes max() over its whole 60 s window, so it gets
    slower as the window fills.
    Prove it with a RATIO, not an absolute time.

    Hint 1: small = per_call_cost(LeakDetector(), 500); large = per_call_cost(LeakDetector(), 8000)
    Hint 2: 16x more history should make each call clearly slower: assert large / small > 3
    Stretch: write a faster detector (a monotonic deque keeps the maximum at the front: O(1)
    amortised), and show the SAME ratio
    test now gives < 3 for your version while both versions return identical alarms on random data.
    """
    todo("measure at two sizes and assert on the ratio")

class BoundedBus(VirtualCANBus):
    """A bus whose log keeps only the last `capacity` frames."""

    def __init__(self, capacity):
        super().__init__()
        # TODO: replace self.log by a deque with maxlen=capacity
        todo("use collections.deque(maxlen=capacity)")

def test_bounded_bus_keeps_memory_flat():
    """Exercise 2 - VirtualCANBus.log keeps EVERY frame forever (a leak in a long-running gateway).
    Fix it in BoundedBus and prove it.

    Hint 1: tracemalloc.start(); bus = BoundedBus(1000); send 50_000 frames; current, _ =
    tracemalloc.get_traced_memory(); tracemalloc.stop()
    Hint 2: assert len(bus.log) == 1000 and current < 2_000_000
    """
    todo("write the memory test")

@contextlib.contextmanager
def budget(seconds):
    """Exercise 3 - a context manager that FAILS if the block takes longer than `seconds` of wall
    time.

    Hint: t0 = time.perf_counter(); yield; elapsed = time.perf_counter() - t0; assert elapsed <
    seconds, f"took {elapsed:.3f}s, budget {seconds}s"
    """
    todo("implement the budget helper")
    yield

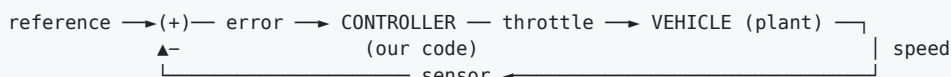
def test_budget_helper_works():
    """Use it twice: a fast block passes, and a slow one (time.sleep(0.05) with budget 0.01) raises
    AssertionError.
    Hint: with pytest.raises(AssertionError, match="budget"): with budget(0.01): time.sleep(0.05)
    """
    todo("test the helper")

```

Chapter 14 — Simulation, closed-loop testing and acceptance criteria

14.1 Why a closed loop changes everything

An **open-loop** test calls the controller with a number and checks the number it returns: "*with error 10 km/h the throttle request is 0.51.*" But a controller is only correct **in a loop**: the throttle matters because of what the vehicle does with it, which changes the next error, which changes the next throttle. Overshoot, oscillation and instability exist only in the loop.



Software-in-the-loop (SIL) runs the *production controller code* against a **plant model** on a PC. It is the first place the behaviour of the whole loop can be tested — and, being fast and deterministic, it can run thousands of scenarios in CI.

14.2 The plant model

A longitudinal vehicle model comes straight from Newton's second law:

$$m \cdot dv/dt = F_{\text{drive}} - F_{\text{drag}} - F_{\text{rolling}} - F_{\text{grade}} - F_{\text{brake}}$$
$$F_{\text{drag}} = \frac{1}{2} \rho \cdot C_d \cdot A \cdot v^2 \quad F_{\text{rolling}} = C_{rr} \cdot m \cdot g \quad F_{\text{grade}} = m \cdot g \cdot \sin(\theta)$$

To simulate it on a computer we **discretise time** with a fixed step `dt` (explicit Euler): `v ← v + a · dt`. Our `Vehicle.step` does exactly this. Four principles for the model itself:

1. **Keep it as simple as the question allows** — a point mass is enough to judge a cruise controller.
2. **Test the model first.** If coasting does not slow the car or the top speed is 400 km/h, every downstream result is meaningless. Plant sanity tests are real tests.
3. **State its validity range.** Drag/rolling/grade is not a model of hydroplaning.
4. **Pin the time step** so runs are deterministic and repeatable.

```
from autotest.vehicle import Vehicle

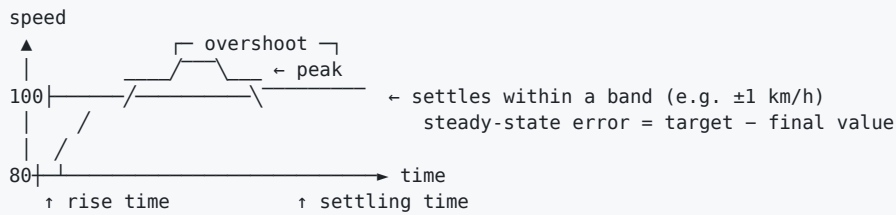
coasting = Vehicle(speed_ms=30)
for _ in range(600):
    coasting.step(0.0, 0.1)
assert 0 < coasting.speed_ms < 30

full = Vehicle()
for _ in range(20000):
    full.step(1.0, 0.1)
assert 200 < full.speed_kmh < 300

number = 100
print(f"top speed {full.speed_kmh:.0f} km/h")
```

14.3 Control performance: the vocabulary of a step response

When the set speed jumps from 80 to 100 km/h the speed *response* is judged by:



Metric	Definition	Acceptance criterion used in Lab 11
Overshoot	$(\text{peak} - \text{target}) / \text{step size}$	< 20 % (under 4 km/h on a 20 km/h step)
Settling time	last time outside the band	< 25 s to ± 1 km/h
Steady-state error	target – final speed	< 0.1 km/h (integral action)
Disturbance rejection	dip when a 5 % hill starts, and recovery	dip < 4 km/h, recovers

Acceptance criteria are requirements written as numbers. "Should feel smooth" cannot be tested; "overshoot < 4 km/h" can.

```

from autotest.cruise import CruiseController
from autotest.vehicle import run_cruise

cc = CruiseController()
cc.power_on(); cc.set(100)
trace = run_cruise(cc, start_kmh=80, duration_s=120) # list of (t, speed_kmh, throttle)

peak = max(v for _, v, _ in trace)
overshoot_pct = (peak - 100) / (100 - 80) * 100
settled_at = max(t for t, v, _ in trace if abs(v - 100) > 1.0)
final_error = abs(trace[-1][1] - 100)
print(f"overshoot {overshoot_pct:.0f} % settles after {settled_at:.1f} s final error
{final_error:.3f} km/h")

assert overshoot_pct < 20 and settled_at < 25 and final_error < 0.1
assert all(0.0 <= u <= 1.0 for _, _, u in trace) # the actuator command stays physical

```

14.4 Scenario testing and disturbances

A single step response is not enough. Build a **scenario catalogue**:

- different set speeds (40, 70, 100, 130, 160 km/h) — a *sweep*;
- disturbances: hills starting and ending, headwind;
- events inside the loop: **brake pressed** at $t = 30$ s; **cancel then resume**;
- degraded conditions: sensor bias, noise, delay.

Events are injected *into the simulated loop* by time, exactly like a test engineer would at a HIL bench:

```

from autotest.cruise import CruiseController, CruiseState
from autotest.vehicle import run_cruise

cc = CruiseController()
cc.power_on(); cc.set(100)
trace = run_cruise(cc, 100, 40, events={30.0: lambda controller, car: controller.brake()})
assert cc.state is CruiseState.STANDBY
assert all(u == 0.0 for t, _, u in trace if t > 30) # after braking: no throttle at all

```

14.5 A tuning requirement can fail

Lab 11 asks for an 8 % hill with a dip of at most 4 km/h. With the default gains ($K_p = 0.05$, $K_i = 0.01$) the car dips **5.4 km/h** — the *requirement* exposes weak tuning. Raising the gains (e.g. $K_p = 0.15$, $K_i = 0.02$) gives a 2.5 km/h dip. Closed-loop tests make such trade-offs concrete: a higher gain rejects disturbances better but reacts more aggressively to sensor noise.

14.6 ABS: testing against a physical phenomenon

The ABS plant is a **quarter-car**: one wheel carrying a quarter of the vehicle mass, with tyre force given by a friction curve that **peaks at about 20 % slip**. The control group is crucial:

- *With* ABS, slip stays near the peak: no lock-up while the car is fast, and a shorter stopping distance.
- *Without* ABS, the wheel locks and the distance is longer.

A good test asserts **both** and keeps the baseline: if "no ABS" did not actually lock the wheel, then "ABS prevents locking" would prove nothing.

```
from autotest.abs import simulate_braking

with_abs, without = simulate_braking(True), simulate_braking(False)
print(f"stopping distance: with ABS {with_abs['distance_m']:.1f} m, locked wheel
{without['distance_m']:.1f} m")
assert without["locked_s"] > 1.0 # baseline: braking without ABS locks
the wheel
assert with_abs["locked_s"] == 0.0 and with_abs["peak_slip"] < 0.35
assert with_abs["distance_m"] < 0.9 * without["distance_m"] # at least 10 % shorter
```

14.7 Numerical sensitivity: do not over-specify

Simulation results depend on the **time step**. Repeating the ABS stop with different `dt`:

```
from autotest.abs import simulate_braking

for dt in (0.0005, 0.001, 0.005, 0.01):
    a = simulate_braking(True, dt=dt)["distance_m"]
    b = simulate_braking(False, dt=dt)["distance_m"]
    print(f"dt={dt:<7} ABS {a:5.1f} m no ABS {b:5.1f} m improvement {100 * (1 - a / b):4.1f} %")
    assert a < 0.9 * b # the REQUIREMENT holds at every step size...
```

The ABS distance varies by several metres between step sizes (the controller and plant interact with the sampling interval), while the **relative** criterion — "at least 10 % shorter" — holds throughout. Two lessons: **pin dt in tests** (determinism), and **prefer acceptance criteria that are robust to modelling details** over exact numbers that merely freeze one run.

14.8 Limits of simulation

Passing in SIL does not mean the real vehicle passes. Ask:

- Which **model assumptions** could be wrong (tyre curve, mass, sensor delay, actuator dynamics)?
- Which effects are **absent** (quantisation, CAN latency, ECU scheduling)?
- Which scenarios are **missing** (cut-in vehicle, sensor dropout mid-brake)?

That is why the pyramid continues upward (PIL, HIL, vehicle) and why the **same scenarios** are re-run on each stage. Real programmes also use **co-simulation** (the FMI standard lets tools exchange models), **back-to-back tests** (model vs. generated code must agree within tolerance), and **scenario databases** with parameter sweeps and randomised variation.

Check your understanding

1. Why can't an open-loop unit test of the controller find integrator windup problems?
2. Your acceptance criterion is "overshoot < 5 %". Is it measurable? Rewrite it as one with explicit definitions.
3. Why must the *no-ABS* baseline stay in the test suite?
4. Why is `assert stopping_distance == 44.77` a poor ABS requirement?

Lab 11 — Software-in-the-loop (SIL) simulation and acceptance criteria

Duration: 3 h · **Type:** closed-loop system testing / model-based testing · **Code under test:**

CruiseController + Vehicle, ABSController + QuarterCar

Why it matters

Controllers are only "correct" in a loop: the throttle value matters because of what the car then does. SIL runs the production control code against a plant model long before a prototype exists; the same scenarios later run on HIL and in the car.

Learning objectives

Turn requirements into **measurable acceptance criteria** (overshoot, settling time, dip, steady-state error, stopping distance); **test the model too** (a wrong plant gives false confidence); use a *control group* (the scenario without ABS must fail); run scenario sweeps; detect a controller whose tuning fails a requirement.

Metrics used

Metric	Definition	Criterion
Overshoot	(peak – target) / step	< 20 % (< 4 km/h on a 20 km/h step)
Settling time	last time outside ± 1 km/h	< 25 s
Hill dip (5 %)	target – minimum after grade starts	< 4 km/h, full recovery
Steady-state error	after 120 s	< 0.1–0.5 km/h
ABS lock time	seconds with slip > 0.95 while $v \geq 5$ m/s	= 0
ABS distance	vs locked-wheel braking	≥ 10 % shorter

Run it

```
pytest labs/lab11_sil_simulation -v
pytest -m sil
```

Guided tour

- **Plant sanity tests first:** coasting, top speed, grade — if the model is wrong, everything downstream is noise.
- `test_baseline_without_abs_locks_the_wheel` — proves the scenario is hazardous, so "ABS prevents lock" is meaningful.
- Determinism test: fixed-step simulation $\Rightarrow$ identical results every run (a prerequisite for regression testing).

Exercises — `exercise_lab11.py`

Run: `python course.py exercise 11` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

8 % hill criteria (default tuning **fails** — find gains that pass) · ABS on ice · a 5 km/h biased speed sensor (what happens, what would detect it?).

Debrief

- Your simulation passes. What *model assumptions* could still make the real car fail?
- Which scenarios would you add for a safety case (cut-in vehicle, sensor dropout mid-brake)?
- When does a SIL result transfer to HIL, and what must be re-validated?

Worked example — source

labs/lab11_sil_simulation/test_lab11_sil_simulation.py

```
"""Lab 11 - software-in-the-loop (SIL): the controller runs against a simulated plant.

Open-loop unit tests say "the function returns 0.7". Closed-loop tests say "the CAR reaches
100 km/h without overshooting more than 4 km/h". Acceptance criteria are written as measurable
numbers.
"""
import pytest

from autotest.abs import ABSController, Valve, simulate_braking, tyre_mu
from autotest.cruise import CruiseController, CruiseState
from autotest.vehicle import Vehicle, run_cruise

pytestmark = [pytest.mark.sil, pytest.mark.system]

# ----- helpers: metrics -----
def settling_time(trace, target, band=1.0):
    """First time after which speed stays within +/- band of the target."""
    last_bad = max((t for t, v, _ in trace if abs(v - target) > band), default=-1.0)
    return last_bad + (trace[1][0] - trace[0][0])

def overshoot_pct(trace, target, start):
    return max(0.0, (max(v for _, v, _ in trace) - target) / (target - start) * 100)

@pytest.fixture
def cruise_at_100():
    cc = CruiseController()
    cc.power_on()
    cc.set(100)
    return cc

# ----- plant sanity (test the model!) -----
def test_plant_coasts_down_without_throttle():
    car = Vehicle(speed_ms=30)
    for _ in range(600):
        car.step(0.0, 0.1)
    assert 0 < car.speed_ms < 30

def test_plant_cannot_exceed_physical_top_speed():
    car = Vehicle()
    for _ in range(20000):
```

```

        car.step(1.0, 0.1)
    assert 200 < car.speed_kmh < 300 # drag balance

def test_plant_steep_grade_slows_the_car():
    flat, hill = Vehicle(speed_ms=25), Vehicle(speed_ms=25)
    for _ in range(100):
        flat.step(0.2, 0.1, 0.0)
        hill.step(0.2, 0.1, 0.08)
    assert hill.speed_ms < flat.speed_ms

# ----- cruise acceptance -----
def test_reaches_target_from_80_with_acceptable_dynamics(cruise_at_100):
    trace = run_cruise(cruise_at_100, start_kmh=80, duration_s=120)
    assert overshoot_pct(trace, 100, 80) < 20.0 # i.e. below 4 km/h on a 20 km/h step
    assert settling_time(trace, 100, band=1.0) < 25.0
    assert abs(trace[-1][1] - 100) < 0.1 # zero steady-state error (integral action)

def test_holds_speed_when_a_5_percent_hill_starts(cruise_at_100):
    trace = run_cruise(cruise_at_100, 100, 120, grade_profile=lambda t: 0.05 if t >= 20 else 0.0)
    assert min(v for t, v, _ in trace if t >= 20) > 96.0 # dip no larger than 4 km/h
    assert abs(trace[-1][1] - 100) < 0.5 # and fully recovered

def test_throttle_never_leaves_0_1_even_for_huge_errors():
    cc = CruiseController()
    cc.power_on()
    cc.set(180)
    trace = run_cruise(cc, 30, 60)
    assert all(0.0 <= u <= 1.0 for _, _, u in trace)

@pytest.mark.requirement("REQ-CC-002")
def test_brake_event_in_the_loop_hands_back_control(cruise_at_100):
    brake_at_30s = {30.0: lambda cc, car: cc.brake()}
    trace = run_cruise(cruise_at_100, 100, 40, events=brake_at_30s)
    assert cruise_at_100.state is CruiseState.STANDBY
    assert all(u == 0.0 for t, _, u in trace if t > 30)

def test_resume_brings_the_car_back_to_the_saved_speed(cruise_at_100):
    ev = {20.0: lambda cc, car: cc.cancel(), 40.0: lambda cc, car: cc.resume(car.speed_kmh)}
    trace = run_cruise(cruise_at_100, 100, 120, events=ev)
    mid = next(v for t, v, _ in trace if t == 39.0)
    assert mid < 100 # slowed down while cancelled
    assert abs(trace[-1][1] - 100) < 0.5 # and came back

@pytest.mark.parametrize("target", [40, 70, 100, 130, 160])
def test_converges_for_every_target_in_the_envelope(target):
    cc = CruiseController()
    cc.power_on()
    cc.set(target)
    trace = run_cruise(cc, target - 10, 200)
    assert abs(trace[-1][1] - target) < 0.5

# ----- ABS acceptance -----

def test_tyre_model_peaks_at_20_percent_slip():
    assert max(tyre_mu(s / 100) for s in range(101)) == pytest.approx(1.0)
    assert tyre_mu(0.2) > tyre_mu(1.0)

@pytest.mark.parametrize("vehicle, wheel, expected", [
    (30, 30, Valve.APPLY), # rolling freely
    (30, 21, Valve.HOLD), # 30 % slip? -> see below
    (30, 20, Valve.RELEASE),
    (3, 0, Valve.APPLY), # below 5 m/s ABS stays out of the way
])
def test_abs_decisions(vehicle, wheel, expected):

```

```

        actual = ABSController().decide(vehicle, wheel)
        if (vehicle, wheel) == (30, 21):
            expected = Valve.RELEASE
            # 30 % slip is above the 25 % release limit
        assert actual is expected

@pytest.mark.requirement("REQ-ABS-001")
def test_wheel_does_not_lock_while_abs_is_active():
    result = simulate_braking(use_abs=True)
    assert result["locked_s"] == 0.0
    assert result["peak_slip"] < 0.35

@pytest.mark.requirement("REQ-ABS-002")
def test_abs_shortens_stopping_distance_by_at_least_10_percent():
    with_abs, without = simulate_braking(True), simulate_braking(False)
    assert with_abs["distance_m"] < 0.9 * without["distance_m"]

def test_baseline_without_abs_locks_the_wheel():
    """The 'control group': proves the scenario really is dangerous, so the ABS test means
    something."""
    assert simulate_braking(use_abs=False)["locked_s"] > 1.0

def test_simulation_is_deterministic():
    assert simulate_braking(True) == simulate_braking(True)

```

Exercise skeleton — source

labs/lab11_sil_simulation/exercise_lab11.py

```

"""Lab 11 exercises - closed-loop (SIL) acceptance tests.

Run it:      python course.py exercise 11
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
test.

Done when:   `python course.py progress` shows every test of Lab 11 passing.
"""
import pytest

import autotest.abs as abs_module
from autotest.abs import simulate_braking
from autotest.cruise import CruiseController
from autotest.learn import todo
from autotest.vehicle import Vehicle, run_cruise

def hill_metrics(kp, ki, grade):
    """Helper (given): a hill of `grade` (0.08 = 8 %) starts at t = 20 s. Returns (dip_kmh,
    recovery_s)."""
    cc = CruiseController(kp, ki)
    cc.power_on()
    cc.set(100)
    trace = run_cruise(cc, 100, 120, grade_profile=lambda t: grade if t >= 20 else 0.0)
    dip = 100 - min(v for t, v, _ in trace if t >= 20)
    recovery = max(t for t, v, _ in trace if abs(v - 100) > 1) - 20
    return dip, recovery

def test_8_percent_hill_meets_the_acceptance_criteria():
    """Exercise 1 - requirement: on an 8 % hill the speed dips by at most 4 km/h and is back within
    1 km/h in 20 s.

    Does the DEFAULT controller (kp=0.05, ki=0.01) pass? Find out FIRST, then tune.

    Hint 1: print(hill_metrics(0.05, 0.01, 0.08)) in a Python shell, or temporarily in the test.
    Hint 2: try larger gains, e.g. (0.1, 0.02), (0.15, 0.02), (0.2, 0.03) - write the sweep as a
    parametrized test.
    Hint 3: dip, recovery = hill_metrics(kp, ki, 0.08); assert dip <= 4.0 and recovery <= 20.0

```

```

        Finally: state your chosen gains in a comment and one downside of raising them (hint: noise).
        """
        todo("write the sweep and the criteria")

    def test_abs_still_helps_on_ice(monkeypatch):
        """Exercise 2 - low-friction surface: scale the tyre friction curve to 0.3 of normal. ABS must
        still beat locked-wheel braking.

        Hint 1: original = abs_module.tyre_mu
        Hint 2: monkeypatch.setattr(abs_module, "tyre_mu", lambda slip: 0.3 * original(slip))
        Hint 3: compare simulate_braking(True)["distance_m"] with simulate_braking(False)["distance_m"];
        is the 10 % improvement criterion still met?
        """
        todo("monkeypatch the friction curve and compare")

    def test_biased_speed_sensor_shifts_the_real_speed():
        """Exercise 3 - the controller's speed input reads 5 km/h TOO LOW. Predict the real steady-state
        speed, then measure it.

        Hint 1: car = Vehicle(speed_ms=100 / 3.6); loop 3000 times: throttle =
        cc.control(car.speed_kmh - 5.0, 0.1); car.step(throttle, 0.1)
        Hint 2: the controller believes it is at target when the SENSOR says 100, so the real speed
        settles at ... ?
        Then, in a comment: which diagnostic or plausibility check would catch this in the field?
        """
        todo("simulate the biased sensor and assert the real speed")

```

Part VII — Adequacy, evidence and delivery

Chapter 15 — Test adequacy: coverage and mutation testing

15.1 "Are my tests any good?"

You have a suite and it is green. Does that mean the software is well tested? **Test adequacy** criteria try to answer this. Two families exist:

- **Coverage** — what fraction of the *code* (or *requirements*) did the tests execute?
- **Mutation testing** — if the code were *wrong*, would any test notice?

The difference is the whole point of this chapter: **coverage measures what ran; mutation measures what was checked**.

15.2 Structural coverage criteria

Structural (white-box) criteria are defined on the control-flow of the code. From weakest to strongest:

Criterion	A test set achieves it when...	Example weakness
Statement (line)	every statement executes	an <code>if</code> without <code>else</code> can be "covered" while its false case is never tried
Branch (decision)	every decision has been both true and false	<code>a</code> and <code>b</code> can be covered while <code>b</code> never decides anything
Condition	every atomic condition has been both true and false	the <i>combination</i> may never occur
Condition/decision	both of the above	does not show each condition <i>matters</i>
MC/DC (modified condition/decision)	additionally, each condition has been shown to independently affect the decision's outcome	the strongest practical criterion; needs $n + 1$ tests for n conditions
Path	every path through the function	exponential; impractical beyond small functions

Statement vs branch: a real measurement

```
import json, os, sys, tempfile, textwrap
import coverage

SRC = textwrap.dedent('''
def clamp_soc(x):
```

```

        if x > 100:
            x = 100
        return x
    '''

    with tempfile.TemporaryDirectory() as tmp:
        path = os.path.join(tmp, "clamp_mod.py")
        open(path, "w").write(SRC)
        sys.path.insert(0, tmp)
        cov = coverage.Coverage(branch=True, include=[path], data_file=None)
        cov.start()
        import clamp_mod
        assert clamp_mod.clamp_soc(150) == 100          # ONE test: only the 'true' side of the if
        cov.stop()
        report = os.path.join(tmp, "cov.json")
        cov.json_report(outfile=report)
        t = json.load(open(report))["files"][path]["summary"]
        print(f"statements {t['covered_lines']}/{t['num_statements']}    branches
{t['covered_branches']}/{t['num_branches']}")
        assert t["covered_lines"] == t["num_statements"]    # 100 % statement coverage ...
        assert t["covered_branches"] < t["num_branches"]    # ... but the 'x <= 100' case never

ran

```

One test achieves 100 % **statement** coverage yet leaves half the **branches** untried — the case where `x` is already ≤ 100 (and an `else`-style bug there would be invisible). That is why safety standards demand branch coverage at least.

MC/DC in one example

For a decision `A` or `B` such as `temp < 0` or `temp > 45`, MC/DC needs three tests, each *pair* demonstrating one condition's independent effect:

Test	<code>temp < 0</code>	<code>temp > 45</code>	Decision	Used to show
t1 (<code>temp = -5</code>)	T	F	T	A matters (compare with t2)
t2 (<code>temp = 25</code>)	F	F	F	baseline
t3 (<code>temp = 60</code>)	F	T	T	B matters (compare with t2)

For three conditions there are $2^3 = 8$ combinations, but only $n + 1 = 4$ are needed. The snippet below searches for a minimal MC/DC set for `a` and `(b or c)`:

```

from itertools import combinations, product

def decision(a, b, c):
    return a and (b or c)

rows = list(product((False, True), repeat=3))

def independent(cond_index, test_set):
    """Is there a pair in the set that differs ONLY in this condition and flips the outcome?"""
    for r1, r2 in combinations(test_set, 2):
        diff = [i for i in range(3) if r1[i] != r2[i]]
        if diff == [cond_index] and decision(*r1) != decision(*r2):
            return True
    return False

minimal = next(s for k in range(2, 9) for s in combinations(rows, k)
               if all(independent(i, s) for i in range(3)))
print(len(minimal), "tests suffice (n+1 = 4):", minimal)
assert len(minimal) == 4

```


In ISO 26262 the recommended coverage metric rises with the ASIL: statement and branch coverage broadly, and **MC/DC for the most demanding levels** (check the exact table in your edition).

Coverage in practice

```
lines    pytest --cov=autotest --cov-branch --cov-report=term-missing    # line + branch, show uncovered
htmlcov/ pytest --cov=autotest --cov-branch --cov-report=html          # browse annotated source in
```

`term-missing` lists the uncovered lines and *partial branches* (e.g. `30->32`): your to-do list for new tests.

15.3 What coverage cannot tell you

- It says nothing about **assertions**. A test that calls everything and asserts nothing yields 100 % coverage.
- It cannot find **missing code** — a requirement you forgot to implement has no lines to cover.
- High coverage with weak checks is **false confidence**; low coverage is **real** information (that code is untested).

Use coverage as a **negative indicator** — *what is definitely untested?* — not as a quality score. Setting "90 % coverage" as a target invites tests written to move the number.

15.4 Mutation testing: testing the tests

Idea. Make a small deliberate change (a **mutant**) to the code — flip `<` to `<=`, change a constant, swap `+` for `-` — and run the tests.

- If a test **fails** → the mutant is **killed** (good: your tests noticed).
- If all tests **pass** → the mutant **survives**: either a test is missing/weak, or the mutation does not change behaviour (*equivalent mutant*).

```
original:  if temp_c > 45:    return 0.0
mutant 1:  if temp_c >= 45:   return 0.0    ← killed only by a test AT exactly 45 °C
mutant 2:  if temp_c > 46:    return 0.0    ← killed only by a test at 46 (or 45.5)
```

The theory rests on two hypotheses: the **competent programmer hypothesis** (real bugs are small deviations from correct code) and the **coupling effect** (tests that catch simple faults also catch complex ones). Mutation analysis dates to the late 1970s (DeMillo, Lipton and Sayward).

A **mutation score** is `killed ÷ total mutants` (excluding equivalent mutants where recognised).

By hand

```
def limit(temp_c):                # code under test
    if temp_c < 0 or temp_c > 45:
        return 0.0
    return 100.0

MUTANTS = {
    "< -> <=":    lambda t: 0.0 if (t <= 0 or t > 45) else 100.0,
    "> -> >=":    lambda t: 0.0 if (t < 0 or t >= 45) else 100.0,
    "45 -> 46":   lambda t: 0.0 if (t < 0 or t > 46) else 100.0,
    "or -> and":  lambda t: 0.0 if (t < 0 and t > 45) else 100.0,
```

```

    }

    weak = [lambda f: f(25) == 100.0] # runs the code, barely checks
it
    strong = weak + [lambda f: f(-0.1) == 0.0, lambda f: f(0) == 100.0, # boundary tests (Chapter 6)
                     lambda f: f(45) == 100.0, lambda f: f(45.1) == 0.0]

    def killed(suite):
        return {name for name, m in MUTANTS.items() if not all(check(m) for check in suite)}

    assert all(check(limit) for check in strong) # both suites pass on the CORRECT code
    print("weak suite kills :", sorted(killed(weak)))
    print("strong suite kills:", sorted(killed(strong)))
    assert killed(weak) == set() # not one mutant is noticed
    assert killed(strong) == set(MUTANTS)

```

The *weak* suite passes on the correct code, yet **every** mutant survives — nothing in it would notice a wrong operator or constant. The boundary tests from Chapter 6 kill them all.

Equivalent mutants

Some mutants cannot be killed because they **do not change observable behaviour**. In `max_charge_current`, replacing `soc >= 100` by `soc > 100` changes nothing: at exactly 100 the taper formula `(100 - soc)/20` already yields 0 A. Such survivors are not noise — they often mean the original line is **redundant code** — but recognising them is a human judgement. A mutation score of 100 % is therefore neither achievable nor a sensible gate.

15.5 Mutation testing in practice

Aspect	Advice
Tools	Python: <code>mutmut</code> , <code>cosmic-ray</code> ; Java: <code>PIT</code> ; C/C++: <code>Mull</code> , <code>Dextool</code> . This course uses the readable <code>tools/mini_mutate.py</code> .
Cost	Every mutant needs a test run: scope it (one function, one module), run it nightly or on changed code only.
Where	Safety-critical logic: limit checks, state machines, arithmetic with boundaries.
Reading results	For each survivor ask: <i>which assertion should have failed?</i> Add it, or classify as equivalent.

A pitfall learned the hard way while building this course: the first version of `mini_mutate.py` gave **different scores on identical runs**. Python validates cached bytecode by source *mtime in whole seconds* and file size; two same-length mutants written within one second silently reused the previous mutant's compiled code. Setting `PYTHONDONTWRITEBYTECODE=1` for the test subprocess fixed it. **When a test tool disagrees with itself, suspect the environment before the tests.**

15.6 Other adequacy measures

- **Requirements coverage** — every requirement has ≥ 1 verifying test (Chapter 16): catches *missing* tests that structural coverage cannot.
- **Fault-injection coverage** — every safety mechanism has been exercised by an injected fault.
- **Defect-based metrics** — defects found per test level, escape rate (defects found *after* release), time-to-fix — the ultimate feedback loop.

15.7 Putting the measures together

Question	Measure
Is any code never executed by tests?	line + branch coverage
Do decisions with several conditions get proper checking?	MC/DC (at higher ASILs)
Would a wrong operator or constant be noticed?	mutation score
Is every requirement verified?	requirements coverage
Is the suite trustworthy?	flakiness rate, test failures vs. real bugs

Check your understanding

1. A single test calls `clamp_soc(150)` on `if x > 100: x = 100; return x`. What are the statement and branch coverages?
2. How many tests does MC/DC need for a decision with four conditions, at minimum?
3. What does a surviving mutant tell you, and what are the two possible explanations?
4. Why is "we have 95 % coverage" not evidence the tests are good?

Lab 12 — Test adequacy: coverage and mutation testing

Duration: 2.5 h · **Type:** tests of the tests · **Tools:** `pytest-cov`, `tools/mini_mutate.py`

Why it matters

ISO 26262 asks for structural coverage (statement → branch → MC/DC by ASIL). But **coverage only proves code ran, not that anything checked it**. Mutation testing asks the real question: *if the code were wrong, would a test fail?*

Learning objectives

Measure line/branch coverage and read the `term-missing` report; build a suite with 100 % coverage that is nearly worthless; run a mutation tester and classify survivors as **weak test** or **equivalent mutant**; understand the cost/benefit of both metrics.

Run it

```
pytest labs/lab12_coverage_mutation/weak_suite.py \
    --cov=autotest.bms --cov-branch --cov-report=term-missing      # full coverage, weak
assertions
python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current --tests
labs/lab12_coverage_mutation/weak_suite.py
python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current --tests
labs/lab03_black_box/test_lab03_black_box.py
python tools/mini_mutate.py ... --list                          # just list the mutants
pytest labs/lab12_coverage_mutation -v                          # automated assertions about both scores (slow, ~25
s)
```

What to expect

Suite	Line+branch coverage of the function	Mutation score
<code>weak_suite.py</code>	100 %	≈ 4 % (1/25)
Lab 3 suite	100 %	22/25 — all three survivors are equivalent mutants

Equivalent mutants: `soc >= 100` → `soc > 100` (or `>= 101`) and `soc > 80` → `soc >= 80` change the code but not the behaviour — the taper formula already yields 0 A at 100 % and the full limit at 80 %. No test can kill them. They are not noise: the explicit `soc >= 100` guard is **redundant code**, which a reviewer may now delete or document. Recognising equivalent mutants is a human judgement call.

A tool bug worth knowing. The first version of `mini_mutate.py` gave *non-deterministic* scores. Cause: Python validates cached bytecode by source mtime (whole seconds) and file size, so two same-length mutants written within one second (`100` → `101`) silently reused the previous mutant's `.pyc`. Fix: `PYTHONDONTWRITEBYTECODE=1` in the test subprocess. Lesson: when a test tool disagrees with itself, suspect the environment before the tests.

How `mini_mutate.py` works (read it — it is ~120 lines)

Parses the target with `ast`, enumerates mutation sites (comparison/arithmetic/boolean operators, numeric constants), rewrites one site at a time in a scratch copy of the project, runs `pytest`, and counts a mutant as *killed* if the run fails.

Exercises — `exercise_lab12.py` (terminal work)

Run: `python course.py exercise 12` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

Find the lowest-branch-coverage module · classify survivors of `CruiseController.control` (state-machine suite: 8/12; + SIL suite: 10/12 — which exact PI arithmetic is still unchecked?) · write one test that kills the most survivors in `DiagnosticManager.report` · argue why 100 % mutation score is not a sensible CI gate.

Debrief

- Can you reach 100 % branch coverage with a test that has no assertions? (Open `weak_suite.py`.)
- Why is MC/DC stricter than branch coverage, and which classes of defect does it target?
- Where would you spend a limited test budget: raising coverage 90→100 %, or mutation score 60→80 % on the safety core?

Worked example — source

`labs/lab12_coverage_mutation/test_lab12_coverage_mutation.py`

```
"""Lab 12 - coverage is necessary, not sufficient: mutation analysis as a test of the tests.

These tests drive tools/mini_mutate.py (a subprocess per mutant, so they are marked slow).
"""
import inspect
import json
import subprocess
import sys
from pathlib import Path

import pytest

from autotest.bms import max_charge_current

ROOT = Path(__file__).resolve().parents[2]
pytestmark = pytest.mark.slow

def mutation_score(tests: str) -> tuple[int, int, str]:
    out = subprocess.run(
        [sys.executable, str(ROOT / "tools" / "mini_mutate.py"),
         "--target", "src/autotest/bms.py", "--function", "max_charge_current", "--tests", tests],
        cwd=ROOT, capture_output=True, text=True, encoding="utf-8", errors="replace", timeout=300,
    ).stdout
    line = next(l for l in out.splitlines() if l.startswith("mutation score"))
    killed, total = line.split(":")[1].split("=")[0].strip().split("/")
    return int(killed), int(total), out

def test_the_strong_suite_from_lab3_kills_all_non_equivalent_mutants():
    killed, total, out = mutation_score("labs/lab03_black_box/test_lab03_black_box.py")
    survivors = total - killed
    # The three survivors are EQUIVALENT mutants - they change the code but not its behaviour:
    # `soc >= 100` -> `soc > 100` / `soc >= 101`, and `soc > 80` -> `soc >= 80`.
```

```

# The taper formula already yields 0 A at 100 % and the full limit at 80 %, so the explicit
# `soc >= 100` guard is REDUNDANT code - mutation testing just told us so.
# No test can kill an equivalent mutant; recognising them is a human skill.
assert survivors <= 3, out

def test_the_weak_suite_has_full_coverage_but_a_poor_mutation_score():
    killed, total, out = mutation_score("labs/lab12_coverage_mutation/weak_suite.py")
    assert killed / total < 0.6, out

def test_coverage_report_for_bms_is_complete_for_the_weak_suite(tmp_path):
    report = tmp_path / "cov.json"
    r = subprocess.run(
        [sys.executable, "-m", "pytest", "labs/lab12_coverage_mutation/weak_suite.py", "-q",
         "--cov=autotest.bms", "--cov-branch", f"--cov-report=json:{report}", "-p",
         "no:cacheprovider"],
        cwd=ROOT, capture_output=True, text=True, encoding="utf-8", errors="replace",
    )
    assert r.returncode == 0, r.stdout
    files = json.loads(report.read_text(encoding="utf-8"))["files"]
    # report keys use the OS path separator, so match on the normalised tail
    data = next(v for k, v in files.items() if k.replace("\\", "/").endswith("src/autotest/bms.py"))
    lines, first = inspect.getsourcelines(max_charge_current)
    inside = set(range(first, first + len(lines)))
    # The weak suite never touches the BatteryManagementSystem class (expected). What matters is
    # that NOTHING inside max_charge_current is missing - neither lines nor branches.
    assert not inside & set(data["missing_lines"]), data["missing_lines"]
    assert not inside & {a for a, _ in data["missing_branches"]}, data["missing_branches"]

```

Additional file

labs/lab12_coverage_mutation/weak_suite.py

```

"""A deliberately BAD test suite: 100 % line and branch coverage of max_charge_current,
yet it would not notice most defects. (Not collected by default: name is not test_*)

    pytest labs/lab12_coverage_mutation/weak_suite.py --cov=autotest.bms --cov-branch --cov-
report=term-missing
    python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current \
--tests labs/lab12_coverage_mutation/weak_suite.py
"""
from autotest.bms import max_charge_current

def test_runs_every_branch_but_checks_almost_nothing():
    for soc, temp in [(50, -5), (50, 60), (50, 5), (50, 25), (100, 25), (90, 25)]:
        result = max_charge_current(soc, temp)
        assert result is not None
        assert result >= 0 # true for almost any mutant too

def test_invalid_input_path_is_executed_too():
    try:
        max_charge_current(101, 25)
    except ValueError:
        pass # executed, never verified that the right error is raised

def test_nan_input_path_is_executed_too():
    # Before the Lab 8 fix this simply returns a number; after it, a ValueError is raised.
    # Either way the lines are EXECUTED (coverage) and nothing is actually verified (weak!).
    try:
        max_charge_current(50, float("nan"))
    except ValueError:
        pass

```

Exercise skeleton — source

labs/lab12_coverage_mutation/exercise_lab12.py

```
"""Lab 12 exercises - kill the survivors found by mutation testing.

Run it:      python course.py exercise 12
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
test.
Done when:   `python course.py progress` shows every test of Lab 12 passing AND the mutation
commands below
              report no survivors for the targeted code.

Step 1 (terminal, no code): measure coverage of the earlier labs
python tools/mini_mutate.py --target src/autotest/cruise.py --cov=autotest --cov-branch --cov-report=term-
missing
Which module has the lowest BRANCH coverage? Write down why 100 % is not the goal for every
module.

Step 2 (terminal): find the weak spots of the cruise-control tests
python tools/mini_mutate.py --target src/autotest/cruise.py --function control \
--tests labs/lab06_state_machine/test_lab06_state_machine.py labs/lab11_sil_simulation
You should see 10/12 killed: two mutants in the PI arithmetic SURVIVE. Which assertion should
have failed?

Step 3: write the tests below, then re-run Step 2 with this file ADDED to --tests:
--tests labs/lab06_state_machine/test_lab06_state_machine.py labs/lab11_sil_simulation
labs/lab12_coverage_mutation/exercise_lab12.py
Goal: 12/12.
"""
import pytest

from autotest.cruise import CruiseController
from autotest.diagnostics import DiagnosticManager
from autotest.learn import todo

def test_pi_law_matches_a_hand_computed_sequence():
    """Kill the PI-arithmetic survivors with EXACT numbers computed by hand.

    Set-up: CruiseController(kp=0.05, ki=0.01), power_on(), set(100); speed is 90 km/h; dt = 0.1 s
    -> error = 10.
        u1 = kp*e + ki*(integral + e*dt) = 0.05*10 + 0.01*(0 + 1.0) = ?
        the integral then becomes 1.0, so u2 = ? and u3 = ?
    Hint 1: compute the three values on paper first.
    Hint 2: results = [cc.control(90, 0.1) for _ in range(3)]; assert results ==
    pytest.approx([...])
    """
    todo("compute u1, u2, u3 by hand and assert them")

def test_healing_needs_exactly_the_configured_number_of_good_cycles():
    """DiagnosticManager(fail_threshold=1, heal_threshold=3): one failure activates the code; it
    must take EXACTLY 3 good cycles to heal.

    Hint 1: report failed=True once -> active_codes() == ["P0100"]
    Hint 2: report failed=False twice -> STILL active (2 good cycles are not enough); a third ->
    not active any more.
    """
    todo("test the boundary of the healing counter")
```

Chapter 16 — Requirements, traceability, standards and continuous integration

16.1 Tests only mean something relative to requirements

Every test answers "does the software do what it *should*?" — so *what it should do* must be written down. Requirements are the oracle for the whole project, and the connection between a requirement and the tests that verify it is called **traceability**.

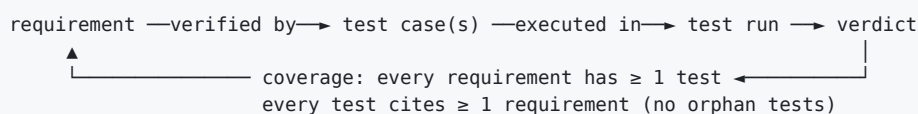
What makes a requirement testable?

Property	Bad	Good
Unambiguous	"charging shall be safe when cold"	"charging shall be inhibited below 0 °C and limited to 25 % of rated current from 0 to <10 °C"
Measurable	"the display shall update quickly"	"the cluster shall show -- when no valid speed frame arrived for 100 ms"
Atomic	one sentence, five obligations	one verifiable obligation, one ID
Identified	in a paragraph	REQ-CLU-001
Feasible & necessary	gold-plated	traceable to a safety goal or stakeholder need

If you cannot write a pass/fail criterion, send the requirement back. *Ambiguity found while designing tests is the cheapest defect you will ever find.*

16.2 Traceability: requirement ⇄ test ⇄ result

An assessor (ISO 26262, Automotive SPICE) asks: "Show me which test verifies requirement X — and its latest result." **Bidirectional traceability** answers that in both directions:



In this course the link is a custom pytest **marker**:

```
import pytest

@pytest.mark.requirement("REQ-CLU-001")
def test_cluster_blanks_after_100ms_without_frames(bus, clock):
    ... # arrange / act / assert go here
```

and a small plugin in `conftest.py` collects the markers and prints a matrix (`pytest --req-report`):

```
REQ-BMS-001  PASS  (14 tests)
REQ-CAN-003  PASS  (4 tests)
```


Two *meta-tests* make the traceability itself checkable, so it cannot rot:

```
import re

REQUIREMENTS = {"REQ-BMS-001", "REQ-BMS-002", "REQ-CC-001"}           # from the requirements
database
TEST_SOURCE = '''
@pytest.mark.requirement("REQ-BMS-001")
def test_cold(): ...
@pytest.mark.requirement("REQ-CC-001")
def test_set_range(): ...
@pytest.mark.requirement("REQ-XYZ-999")
def test_typo(): ...
'''

cited = set(re.findall(r'mark\.requirement\("(REQ-[A-Z]+\d{3})"\)', TEST_SOURCE))

untested = sorted(REQUIREMENTS - cited)           # requirements nobody verifies
unknown = sorted(cited - REQUIREMENTS)             # tests citing deleted/mistyped requirements
print("requirements without tests:", untested)
print("tests citing unknown requirements:", unknown)
assert untested == ["REQ-BMS-002"] and unknown == ["REQ-XYZ-999"]
```

A trap: a PASS in the matrix proves a test *ran* and passed — not that it checks the requirement's numbers. Pair traceability with adequacy measures (Chapter 15) and review.

16.3 Acceptance tests and scenarios

Acceptance tests express requirements in the stakeholder's language and decide whether the product is *accepted*. The *Given / When / Then* form (also called **BDD** — behaviour-driven development) keeps them readable:

```
Feature: Over-voltage protection
Scenario: One weak cell in a healthy pack
    Given a 96-cell pack where every cell is at 3.9 V
    When one cell rises to 4.21 V
    Then an overvoltage fault is raised
```

You can implement them as plain pytest tests with Given/When/Then comments (as Lab 13 does), or with a tool such as `pytest-bdd` that binds Gherkin text to step functions. Plain tests are enough until non-programmers must author the scenarios.

16.4 Functional safety standards in the verification flow

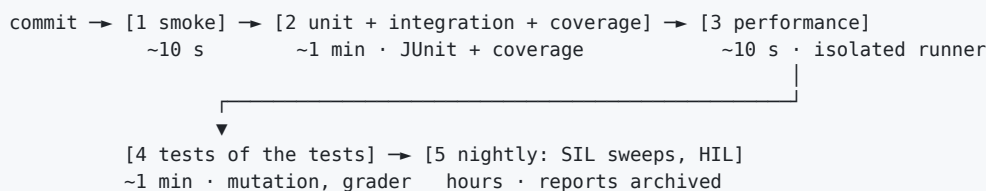
Standard	What it asks of verification (high level)
ISO 26262 (functional safety of road vehicles)	Verification at each level of the V-model with methods and rigour scaled by ASIL : requirements-based tests, interface tests, fault injection, resource-usage tests, structural coverage; evidence and traceability in the safety case; independence of reviewers/testers at higher ASILs
Automotive SPICE (process assessment)	Defined processes for unit verification, component/integration verification and software qualification test, with traceability and consistency between requirements, design and tests

Standard	What it asks of verification (high level)
ISO/SAE 21434 (cybersecurity engineering)	Verification includes vulnerability analysis and penetration/fuzz testing of external interfaces
ISO/IEC/IEEE 29119 (software testing)	General vocabulary, test processes and documentation templates

These standards describe *what evidence* to produce, not which tool to use. The techniques of this book — requirements-based design, boundary analysis, fault injection, coverage, traceability, automation — are how that evidence is generated. (*Consult the edition and tailoring that apply to your project and your safety manager; this book is not a compliance guide.*)

16.5 Continuous integration: tests run themselves

A test that nobody runs protects nothing. **Continuous integration (CI)** runs the suite automatically on every change, so a regression is reported within minutes to the author who caused it. Organise the pipeline as **stages**, cheapest and fastest first:



A GitHub Actions workflow for this course (excerpt):

```

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix: { python: ["3.10", "3.11", "3.12", "3.13"] }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "${{ matrix.python }}" }
      - run: pip install -r requirements.txt
      - run: pytest -m smoke -q
      - run: pytest -m "not slow and not performance" --cov=autotest --cov-branch --
junitxml=report.xml
      - run: pytest -m performance -q
      - run: pytest -m slow -q
      - uses: actions/upload-artifact@v4
        with: { name: reports, path: "report.xml\ncoverage.xml" }

```

Good CI habits:

- **Fail fast; keep the main branch green.** A red main branch teaches everyone to ignore red.
- **Publish artefacts:** JUnit XML (test results), coverage, traceability matrix — they *are* the audit evidence.
- **Matrix** across supported Python versions/platforms.
- **Quarantine flaky tests visibly** (tracked ticket, deadline), never ignore them silently.
- **Keep it fast.** A 40-minute pipeline is skipped; tier the slow tests (smoke → unit → nightly).

16.6 One test body, many back-ends: SIL and HIL

The *same* requirement should be verified on the simulator and, later, on the bench. Write the test against an **abstraction** and let a fixture supply the back-end; hardware-only variants are **skipped automatically** when no hardware is present:

```
import pytest
from autotest.sensors import TemperatureSensor

class SimulatedADC:
    def read(self): return 2048

@pytest.fixture(params=["sil", "hil"])
def adc(request):
    if request.param == "hil":
        pytest.skip("no hardware bench connected")    # in a real lab: return BenchADC(port=...)
    return SimulatedADC()

def test_mid_scale_temperature_is_plausible_on_every_back_end(adc):
    assert -40 < TemperatureSensor(adc).read() < 150
```

On a build server this runs once and reports one skipped variant; on the HIL bench the same test runs twice. This is how test assets are *reused across the X-in-the-loop stages*. Real bench adapters must also manage **safe state on abort**, **nondeterministic timing**, and **limited test time** — which is why only what *needs* hardware is run there.

16.7 Metrics that matter (and those that mislead)

Useful	Misleading on its own
requirements covered / verified / failing	raw test count
branch coverage + mutation score on safety logic	overall coverage percentage as a target
flaky-test rate; mean time to detect	"tests per developer"
defect escape rate (found after release)	pass rate with no context
pipeline duration (feedback time)	lines of test code

Check your understanding

1. Rewrite "the system shall react fast to a lost frame" as a testable requirement.
2. What two failure modes do the traceability meta-tests detect?
3. Why order a CI pipeline smoke → unit → performance → nightly?
4. How does a `params=["sil", "hil"]` fixture let one test serve both a simulator and a bench?

Lab 13 — Acceptance tests, requirements traceability and SIL/HIL switching

Duration: 3 h · **Type:** acceptance / requirements-based testing · **Artefacts:** docs/requirements.csv , --req-report

Why it matters

Assessors (ISO 26262, ASPICE SWE.4–SWE.6) do not ask "do you have tests?" but "**show me which test verifies requirement X, and its result.**" Traceability — requirement $\rightleftharpoons$ test $\rightleftharpoons$ result — is the evidence package.

Learning objectives

Write acceptance scenarios in **Given / When / Then** form; link tests to requirement IDs with a custom marker; produce a traceability matrix; make the test suite **fail when a requirement has no test** or a test cites an unknown requirement; run the same test body on a simulator and (optionally) on hardware.

The mechanism (see `conftest.py`)

```
@pytest.mark.requirement("REQ-BMS-003")    # on a test, a class, or a whole module (pytestmark)
def test_one_weak_cell_raises_an_overvoltage_fault():
    ...
```

- `pytest --req-report` prints each requirement → PASS/FAIL with its tests.
- Meta-tests scan the repository: every ID in `docs/requirements.csv` needs ≥ 1 verifying test; every marker must cite a known ID.
- `@pytest.fixture(params=["sil", pytest.param("hil", marks=pytest.mark.hil)])` — `hil` tests are **skipped unless** `--hil`, so one test body serves simulator and bench.

Run it

```
pytest labs/lab13_acceptance_traceability -v
pytest labs/solutions -m "not slow" -q --req-report    # the matrix
pytest labs/lab13_acceptance_traceability --hil        # BenchADC raises: shows what a missing
bench looks like
```

Exercises — `exercise_lab13.py`

Run: `python course.py exercise 13` — the tests start red (most with a `todo(...)` line: delete it when you start on that test). Hints are in each test's docstring; read them one at a time.

1. **Requirements-driven development:** REQ-TPMS-001/002 exist in the CSV but (without `solutions/`) nothing verifies them: the meta-test is red → write the tests → green.
2. An acceptance scenario for the dashboard during acceleration.
3. A second switchable back-end (`virtual` vs `pcan`) for the CAN tests.

Debrief

- A requirement has 14 tests, another has 1. Is the second one less verified? What would you ask?
- PASS in the matrix, but the test never asserts on the requirement's *numbers*. How would an assessor catch that? (Hint: Lab 12.)
- What must change in a HIL back-end that is *not* just "a different class"? (timing, nondeterminism, safe state on test abort.)

Worked example — source

labs/lab13_acceptance_traceability/test_lab13_acceptance_traceability.py

```
"""Lab 13 - acceptance testing, requirement traceability and hardware-in-the-loop (HIL) switching.

ISO 26262 / ASPICE style evidence: every requirement is linked to >= 1 verifying test and
the report shows PASS/FAIL per requirement:

    pytest labs --req-report -q
"""
import ast
import csv
import re
import subprocess
import sys
from pathlib import Path

import pytest

from autotest.bms import BatteryManagementSystem, BMSFault
from autotest.can import CANFrame
from autotest.diagnostics import DiagnosticManager
from autotest.sensors import TemperatureSensor

ROOT = Path(__file__).resolve().parents[2]
with (ROOT / "docs" / "requirements.csv").open(encoding="utf-8", newline="") as _f:  # closed
    promptly (no ResourceWarning)
    REQUIREMENTS = {row["id"]: row for row in csv.DictReader(_f)}

# =====
# A. Acceptance scenarios (Given / When / Then) - written in the customer's language
# =====
@pytest.mark.requirement("REQ-CAN-001")
class TestFeatureFrameValidation:
    """Feature: the gateway refuses malformed identifiers."""

    @pytest.mark.parametrize("ident, extended, ok", [
        (0x000, False, True), (0x7FF, False, True), (0x800, False, False),
        (0x800, True, True), (0x1FFFFFFF, True, True), (0x20000000, True, False), (-1, False,
False),
    ])
    def test_scenario_identifier_ranges(self, ident, extended, ok):
        # Given a gateway that builds frames
        # When it is asked for identifier <ident> (standard or extended)
        # Then the frame exists only if the identifier fits the identifier field
        if ok:
            assert CANFrame(ident, is_extended=extended).arbitration_id == ident
        else:
            with pytest.raises(ValueError):
                CANFrame(ident, is_extended=extended)

@pytest.mark.requirement("REQ-BMS-003")
class TestFeatureOvervoltageProtection:
    def test_scenario_one_weak_cell_in_a_healthy_pack(self):
        # Given a 96-cell pack where every cell is at 3.9 V
        cells = [3.9] * 96
```

```

        bms = BatteryManagementSystem()
        assert bms.evaluate(cells, 25) is BMSFault.NONE
        # When ONE cell rises to 4.21 V
        cells[57] = 4.21
        # Then an overvoltage fault is raised
        assert bms.evaluate(cells, 25) is BMSFault.OVERVOLTAGE

    def test_scenario_cell_exactly_at_the_limit_is_still_allowed(self):
        assert BatteryManagementSystem().evaluate([4.20] * 4, 25) is BMSFault.NONE

@pytest.mark.requirement("REQ-DIAG-001")
class TestFeatureDtcDebouncing:
    @pytest.mark.parametrize("n", [1, 2, 3, 5])
    def test_scenario_code_is_confirmed_only_after_n_consecutive_failures(self, n):
        # Given a manager that confirms after n failures
        diag = DiagnosticManager(fail_threshold=n)
        # When the monitor fails n-1 times
        for _ in range(n - 1):
            diag.report("P0301", failed=True)
        # Then nothing is stored; one more failure confirms it
        assert diag.stored_codes() == []
        diag.report("P0301", failed=True)
        assert diag.stored_codes() == ["P0301"]

    def test_scenario_a_good_cycle_resets_the_failure_streak(self):
        diag = DiagnosticManager(fail_threshold=3)
        for outcome in (True, True, False, True, True):
            diag.report("P0301", failed=outcome)
        assert diag.stored_codes() == [] # failures were never CONSECUTIVE enough

# =====
# B. Traceability checks - the test suite tests its own documentation
# =====
def _requirements_cited_by_finished_exercise(path):
    """Exercise files: a marked test only counts once the learner has deleted its `todo(...)`
stub."""
    cited = set()
    for node in ast.walk(ast.parse(path.read_text(encoding="utf-8"))):
        if isinstance(node, ast.FunctionDef) and node.name.startswith("test_"):
            is_stub = any(isinstance(n, ast.Call) and getattr(n.func, "id", "") == "todo" for n in
ast.walk(node))
            for deco in node.decorator_list:
                if not is_stub and isinstance(deco, ast.Call) and getattr(deco.func, "attr", "") ==
"requirement":
                    cited.add(deco.args[0].value)
    return cited

def _markers_in_repo():
    found = {}
    pattern = re.compile(r'mark.requirement\("(REQ-[A-Z]+\d{3})"\)')
    for folder in ("labs", "solutions"):
        for path in (ROOT / folder).rglob("*.py"):
            if path.name.startswith("exercise_"): # unfinished stubs must not count as
evidence
                reqs = _requirements_cited_by_finished_exercise(path)
            else:
                reqs = pattern.findall(path.read_text(encoding="utf-8"))
            for req in reqs:
                found.setdefault(req, set()).add(path.name)
    return found

def test_every_requirement_is_verified_by_at_least_one_test():
    verified = _markers_in_repo()
    missing = sorted(set(REQUIREMENTS) - set(verified))
    assert not missing, (
        f"requirements without a verifying test: {missing}\n"
        " -> If these are REQ-TPMS-*, this is EXPECTED until you do Exercise 1 of Lab 13 "
        "(python course.py exercise 13): write tests carrying @pytest.mark.requirement(...) and this
goes green.\n"

```

```

        " -> Otherwise a requirement in docs/requirements.csv has no test: add one."
    )

def test_no_test_refers_to_an_unknown_requirement():
    unknown = sorted(set(_markers_in_repo()) - set(REQUIREMENTS))
    assert not unknown, f"typo or deleted requirement? {unknown}"

def test_requirement_report_option_prints_a_matrix():
    r = subprocess.run(
        [sys.executable, "-m", "pytest", "labs/lab05_integration", "-q", "--req-report", "-p",
        "no:cacheprovider"],
        cwd=ROOT, capture_output=True, text=True, encoding="utf-8", errors="replace",
    )
    assert r.returncode == 0, r.stdout
    assert "REQ-CLU-001" in r.stdout and "PASS" in r.stdout
    assert "REQ-CAN-003" in r.stdout

# =====
# C. SIL / HIL: one test body, two back-ends
# =====
class SimulatedADC:
    """SIL back-end: always available."""

    def __init__(self, counts=2048):
        self.counts = counts

    def read(self):
        return self.counts

class BenchADC:
    """HIL back-end: talks to a real bench (serial / CAN-FD / NI-DAQ ...). Not available here."""

    def __init__(self):
        raise RuntimeError("no hardware bench connected") # replaced by a real driver in the lab

    def read(self): # pragma: no cover
        raise NotImplementedError

@pytest.fixture(params=["sil", pytest.param("hil", marks=pytest.mark.hil)])
def adc(request):
    return SimulatedADC() if request.param == "sil" else BenchADC()

def test_mid_scale_temperature_is_plausible_on_every_back_end(adc):
    # Same assertions on the simulator (default) and on the bench (pytest --hil)
    assert -40 < TemperatureSensor(adc).read() < 150

def test_hil_tests_are_skipped_without_the_flag(pytestconfig):
    if pytestconfig.getoption("--hil"):
        pytest.skip("running with --hil: this check only applies to the default run")
    r = subprocess.run(
        [sys.executable, "-m", "pytest", __file__, "-q", "-rs", "-p", "no:cacheprovider", "-k",
        "mid_scale"],
        cwd=ROOT, capture_output=True, text=True, encoding="utf-8", errors="replace",
    )
    assert "needs hardware" in r.stdout

```

Exercise skeleton — source

labs/lab13_acceptance_traceability/exercise_lab13.py

```

"""Lab 13 exercises - requirements-driven testing, acceptance scenarios, SIL/HIL back-ends.

```

```

Run it:      python course.py exercise 13
How it works: every test starts with a todo(...) line. DELETE that line when you start on the
test.

Done when:   `python course.py progress` shows every test of Lab 13 passing.
"""
import pytest

from autotest.ecu import InstrumentCluster, WheelSpeedECU
from autotest.learn import todo
from autotest.sensors import TemperatureSensor
from autotest.tpms import LeakDetector, TyreStatus, classify

# -----
# Exercise 1 - REQUIREMENTS-DRIVEN development.
# docs/requirements.csv already lists REQ-TPMS-001 and REQ-TPMS-002, but (without the instructor's
solutions folder)
# nothing verifies them: the meta-test `test_every_requirement_is_verified_by_at_least_one_test`
is RED.
# Make it green by writing verifying tests that carry the requirement marker.
# -----
@pytest.mark.requirement("REQ-TPMS-001")
def test_critical_pressure_is_classified_critical():
    """REQ-TPMS-001: a tyre below 60 % of nominal (230 kPa), normalised to 20 degC, is CRITICAL.
    Hint 1: 60 % of 230 is 138 kPa. Test just below (137.99), on (138.0 -> LOW, not CRITICAL!) and a
clearly critical value (0).
    Hint 2: classify(kpa, temp_c) returns a TyreStatus; compare with `is`.
    """
    todo("verify REQ-TPMS-001")

@pytest.mark.requirement("REQ-TPMS-002")
def test_a_pressure_drop_of_20_kpa_raises_the_leak_alarm():
    """REQ-TPMS-002: a drop of 20 kPa or more within 60 s raises a rapid-loss alarm.
    Hint 1: leak = LeakDetector(); leak.add(0, 230) records a sample at t = 0 s; leak.add(10,
210) returns True/False.
    Hint 2: test exactly 20 kPa (alarm) and 19.99 kPa (no alarm).
    """
    todo("verify REQ-TPMS-002")

# Bonus (terminal): add a line REQ-TPMS-003 (sensor-id validation) to docs/requirements.csv, watch
the meta-test go red again,
# then write the verifying test and watch it turn green.

# -----
# Exercise 2 - an acceptance scenario in Given / When / Then style.
# -----
@pytest.mark.requirement("REQ-CLU-001")
def test_dashboard_never_goes_backwards_during_acceleration(bus, clock):
    """Given the bus is healthy
    When the car accelerates from 0 to 100 km/h in ten steps of 10 km/h (one frame every 10 ms)
    Then the displayed speed never decreases.

    Hint 1: ecu, cluster = WheelSpeedECU(bus), InstrumentCluster(bus, clock)
    Hint 2: collect int(cluster.displayed_speed()) after each send_speed(...) and
clock.advance(0.01)
    Hint 3: assert shown == sorted(shown)
    """
    todo("write the scenario with Given/When/Then comments")

# -----
# Exercise 3 - one test body, two back-ends (simulator now, hardware later).
# -----
class SimulatedADC:
    def read(self):
        return 2048

@pytest.fixture(params=["virtual", "pcan"])
def adc_backend(request):

```



```

    """Return an ADC for the requested back-end. 'pcan' stands for a real bench adapter: skip it
    when no hardware is connected.

    Hint 1: if request.param == "pcan": pytest.skip("no hardware bench connected")
    Hint 2: otherwise return SimulatedADC()
    """
    todo("implement the fixture so the hardware variant is skipped")

def test_mid_scale_temperature_is_plausible_on_every_back_end(adc_backend):
    """Hint: the same assertion for both back-ends: -40 < TemperatureSensor(adc_backend).read() <
150    Run it: you should see one pass and one SKIPPED (with the reason)."""
    todo("write the shared assertion")

```

Part VIII — Capstone

Chapter 17 — Putting it all together: building a test strategy

You now have the tools; the capstone asks you to *use* them on an unfamiliar module. This chapter is a recipe, a checklist and a list of habits to take back to real projects.

17.1 From specification to a test plan: a recipe

Take the TPMS specification in the capstone (S1–S9). A strategy is built in layers:

Step 1 — Read the spec as a tester. Underline every *number*, *comparison*, *unit* and *error condition*. Each is a potential defect site. Write down every **question** the spec leaves open — these are findings.

Step 2 — Decompose into testable units. Normalisation (a calculation), classification (thresholds, four outputs), validation (sensor id, ranges), leak detection (stateful, time window).

Step 3 — Pick techniques per unit.

Unit	Techniques (chapter)
Normalisation formula	exact hand-computed values at cold / nominal / hot; one <i>property</i> (monotone in temperature) — (6, 10)
Classification	equivalence classes for the four outputs; three-value boundaries at every threshold; check the <i>default</i> nominal and an <i>override</i> (6)
Input validation	negative tests; both sides of every inclusive limit , on both axes; format classes for ids (6, 11)
Leak detector	state-based: drop sizes (just below / exactly / above the threshold), the window edge (exactly at, just after), expiry of old samples, rising pressure, slow drift; fixtures for a fresh detector each test (6, 9)
Whole module	one property-based test (e.g. a better pressure never yields a worse status) (10)

Step 4 — Design with oracles from the spec. Compute expected values by hand or from the formulas. Never paste output from the code.

Step 5 — Write them to be readable. Parametrize with ids; one reason to fail per test; names that state behaviour and condition; fixtures for repeated set-up; a requirement marker on tests that verify a stated requirement.

Step 6 — Check adequacy. Run coverage (`--cov-branch --cov-report=term-missing`). Then ask the mutation question for every `<`, `<=`, constant and `max/min` in the spec: *which assertion fails if this is wrong?* Use `capstone/grade.py` as an independent measure.

Step 7 — Make it maintainable. Deterministic (no sleeps, seeded randomness), fast, independent, documented.

17.2 A test-plan skeleton

1. Scope – what is under test, what is explicitly out of scope and why
2. References – specification version, requirements IDs
3. Strategy – levels, techniques, tools, environments (SIL/HIL)
4. Test design – per unit: partitions, boundaries, decision/state tables
5. Entry / exit – when do we start; "done" means: all requirements verified, branch coverage $\geq X$ on safety code, mutation score $\geq Y$ on <modules> (survivors reviewed), no open critical defects
6. Environment – versions, simulators, fixtures, data (logs/golden files)
7. Risks – what is hard to test and what we do about it
8. Reporting – CI stages, artefacts (JUnit, coverage, traceability matrix)

17.3 A checklist for reviewing a test

1. Does the **name** say what behaviour and under which condition?
2. Is the **expected value** derived from the specification?
3. Is there **exactly one reason** for it to fail?
4. Would it **fail if the code were wrong** in the obvious ways (try a mutation)?
5. Is it **deterministic** (clock, randomness, ordering) and **independent**?
6. Is the *arrange* step **obvious** without hunting through fixtures?
7. Does it test **behaviour**, not implementation details, so refactoring does not break it?
8. Is it at the **right level** — could a faster, lower-level test catch the same defect?

17.4 Common anti-patterns (and the lab that cures them)

Anti-pattern	Cure
Expected values copied from the code's output	derive from the spec (Ch. 6)
<code>time.sleep()</code> in tests	inject a clock (Ch. 7)
Everything mocked; assertions only on calls	state verification, real cheap collaborators (Ch. 7)
One giant test with many asserts	focused tests, parametrization (Ch. 5)
Shared mutable fixtures	function-scoped fixtures (Ch. 5)
Tests that depend on execution order	isolation (Ch. 4, 5)
"100 % coverage" as the goal	mutation score + review (Ch. 15)
Updating golden files to turn red → green	review diffs deliberately (Ch. 12)
Ignoring/retrying flaky tests	find the cause (Ch. 12)
Only happy-path tests	boundaries, negatives, fault injection (Ch. 6, 11)
Ice-cream-cone test mix	pyramid (Ch. 2)
No link between tests and requirements	traceability (Ch. 16)

17.5 Habits to take to a real project

- **Test first, or at least test together.** For every bug: red → green → keep.
- **Make the code testable.** Inject clocks, hardware, randomness; prefer pure functions; small interfaces.

- **Automate and run constantly.** If it is not in CI, it does not exist.
- **Treat tests as production code:** review, refactor, delete dead tests.
- **Measure the tests** (coverage for gaps, mutation for strength, flakiness for trust).
- **Be honest about what is not tested** — every strategy has gaps; write them down with the risk.
- **Keep learning from escapes.** Every defect found late is a missing test *and* a question about the strategy.

17.6 What to do next

1. Complete the **capstone** and aim for a perfect defect-catch score.
2. Apply the **mutation tool** to your own project's most critical function.
3. Replace one slow, flaky end-to-end test with a seeded simulation or a fake clock.
4. Add a **smoke stage** and a **traceability check** to your CI.
5. Read about **TDD**, **property-based stateful testing**, **coverage-guided fuzzing**, and **MC/DC tooling** (see the reading list).

Final self-assessment

1. Choose a function from your own work. Which of its inputs have *boundaries*? Which *combinations* matter? What *hostile* values could reach it?
2. Which collaborators would you replace with doubles, and what *seam* would you add to allow it?
3. What *property* must hold for every input? What *safety* property?
4. How would you know your tests are strong enough — which number would you look at, and what would it *not* tell you?

Capstone — Tyre-Pressure Monitoring bug hunt

Duration: 4–6 h (individual or pairs) · **Uses:** every technique from Labs 1–13

Scenario

You join a TPMS team. The module `src/autotest/tpms.py` has no tests. Your job: write a test suite that is **good enough to catch regressions** — the team will later "refactor" the module, and your tests are the safety net.

Specification (the only thing you may rely on)

Pressures in kPa, temperatures in °C.

ID	Rule
S1	Normalise to 20 °C: $p_{20} = p \cdot 293.15 / (273.15 + T)$
S2	Nominal pressure 230 kPa (overridable)
S3	$p_{20} < 60\%$ of nominal → CRITICAL
S4	$60\% \leq p_{20} < 80\%$ → LOW
S5	$80\% \leq p_{20} \leq 130\%$ → OK
S6	$p_{20} > 130\%$ → HIGH
S7	A drop of ≥ 20 kPa within a 60 s window (relative to the highest pressure in the window) → rapid-loss alarm
S8	Sensor ids: exactly 8 hex digits, either case; invalid → <code>ValueError</code> ; valid ids are returned upper-case
S9	Readings outside 0...700 kPa or -40...125 °C → <code>ValueError</code>

Requirements for traceability: **REQ-TPMS-001** (S3 critical), **REQ-TPMS-002** (S7 leak alarm).

Deliverables

1. `capstone/test_tpms_student.py` (replace the starter; split into several files if you like).
2. Use **at least**: parametrization with ids · boundary value analysis · a fixture · one property-based test · a `requirement` marker on the relevant tests · an equivalence-class argument in comments.
3. A short `capstone/REPORT.md` (≤ 1 page): your test strategy, what you did *not* test and why, the coverage figure, and which technique found the most.

How you are graded

```
python capstone/grade.py # injects 17 hidden defects one at a time; you must catch them
pytest capstone --cov=autotest.tpms --cov-branch --cov-report=term-missing
```

Criterion	Weight
Defects caught (<code>grade.py</code> score)	40 %
Suite is green on the correct code and deterministic (no sleeps, seeded randomness)	15 %
Test design quality: boundaries, classes, readable names, one reason to fail	20 %
Traceability + coverage evidence	10 %
Report	15 %

The grader prints *symptoms* of missed defects (e.g. "classification wrong exactly at the LOW/OK threshold") — never the code change — so you can iterate. Aim for 100 % (the instructor's reference suite scores 17/17).

Hints (read only if stuck)

▼ Hint 1

For each threshold in S3–S6 test just-below / on / just-above at 20 °C where the float arithmetic is exact.

▼ Hint 2

Limits in S9 are inclusive: test the limit itself *and* the next representable value beyond it, on both axes.

▼ Hint 3

For S7 think about: exactly 20, just under 20, an old peak that should have expired, the window edge at exactly 60 s, and a gentle slope that adds up to a big drop over many minutes.

▼ Hint 4

Mutation thinking: for every `<`, `<=`, constant and `max/min` in the spec, ask "which assertion fails if this is wrong?"

Starter test suite — source

capstone/test_tpms_student.py

```

"""Capstone starter - replace/extend with YOUR test suite for autotest.tpms.

1. Read capstone/README.md (the specification S1..S9) and Chapter 17 (the recipe).
2. Write tests here (split into more files if you like). They must PASS on the correct code.
3. Grade yourself:  python course.py capstone      (injects 17 hidden defects, one at a time)
    The grader tells you the SYMPTOM of every defect you missed - use it to find the test you
forgot.
"""
import pytest

from autotest.tpms import LeakDetector, TyreStatus, classify, normalise_pressure, validate_sensor_id

@pytest.mark.smoke
def test_nominal_tyre_is_ok():
    assert classify(230, 20) is TyreStatus.OK

# TODO 1  normalisation (S1)                -> exact values, cold and hot
# TODO 2  classification (S3-S6)            -> every threshold: just below / on / just above
# TODO 3  nominal override                  -> classify(..., nominal=250)
# TODO 4  input validation (S8, S9)         -> every limit: on it (ok) and just beyond (ValueError)
# TODO 5  leak detection (S7)               -> drop sizes, window edges, recovery, falling slowly

```

```
# TODO 6 one property-based test      -> classify is monotonic in pressure  
# TODO 7 mark requirement tests      -> @pytest.mark.requirement("REQ-TPMS-001") / "REQ-TPMS-  
002"
```

Appendices

Appendix A — Glossary

Term	Meaning
Acceptance test	Test deciding whether a system satisfies stakeholder needs; often written as Given/When/Then scenarios.
Alive counter	A small counter incremented with every message so receivers can detect lost, repeated or stale frames.
Anti-windup	Control-law technique that stops the integral term accumulating while the actuator is saturated.
Arrange–Act–Assert (AAA)	The three phases of a test: set up, perform the action, check the result. (Given–When–Then.)
ASIL	Automotive Safety Integrity Level (A–D, plus QM): ISO 26262's measure of required rigour.
Assertion	A statement in a test that must hold; failure signals a defect.
Back-to-back test	Comparing two implementations (e.g. model and generated code) on the same inputs.
BDD	Behaviour-driven development: specifying behaviour as readable scenarios that double as tests.
BMS	Battery management system: monitors and protects a battery pack.
Boundary value analysis	Test design that checks values on and either side of class boundaries.
Branch coverage	Fraction of decision outcomes (true/false) exercised by the tests.
CAN	Controller Area Network: the standard in-vehicle serial bus.
CI	Continuous integration: automatically building and testing every change.
Contract test	Test pinning an interface's exact behaviour (e.g. the bytes on the wire).
Coverage	A measure of how much code (or how many requirements) the tests exercised.
CRC	Cyclic redundancy check: a checksum detecting corrupted data.
Decision table	Table of condition combinations and the resulting action.
DBC / ARXML	Formats describing CAN messages and signals (layout, scaling).
Dependency injection	Passing a component's collaborators in from outside instead of creating them inside.
DTC	Diagnostic trouble code: stored identifier of a detected fault.
Debouncing	Requiring a condition to persist (N cycles) before acting on it.
E2E protection	End-to-end protection of safety-relevant data (CRC + counter + timeout).
ECU	Electronic control unit: an embedded computer in a vehicle.

Term	Meaning
Equivalence class	Set of inputs the specification treats identically.
Equivalent mutant	A mutant whose behaviour is identical to the original; cannot be killed by any test.
Error / fault / failure	Human mistake / the defect it leaves in the code / the wrong behaviour observed at run time.
Fail-safe / fail-operational	Reaction that moves to a harmless state / that keeps degraded service.
Fake	A simplified working implementation used in tests (e.g. in-memory bus).
Fault injection	Deliberately introducing faults to verify detection and reaction.
Fixture	Prepared environment or object a test needs.
Flaky test	A test with non-deterministic outcome without code change.
FMI	Functional Mock-up Interface: standard for exchanging simulation models.
Fuzzing	Feeding random/malformed inputs to find crashes and vulnerabilities.
Golden file	Stored, approved output that new output is compared against.
Hardware-in-the-loop (HIL)	Testing a real ECU against a real-time simulation of its environment.
Hypothesis	Python library for property-based testing.
Integration test	Test of several components working together and their interfaces.
Invariant	A condition that must hold in every reachable state.
ISO 26262	Functional safety standard for road vehicles.
Jitter	Variation in timing of a periodic event.
MC/DC	Modified condition/decision coverage: each condition shown to independently affect the outcome.
Metamorphic testing	Testing a relation between outputs of related inputs when no exact oracle exists.
MIL / SIL / PIL	Model- / software- / processor-in-the-loop.
Mock	A test double that verifies interactions (calls) against expectations.
Mutation score	Fraction of mutants killed by a test suite.
Oracle	The mechanism deciding whether an observed result is correct.
Overshoot	How far a controlled quantity exceeds its target during a step response.
Parametrization	Running one test body with several data sets.
PI controller	Controller with proportional and integral terms.
Plant	The controlled system (vehicle, tyre, battery) in a control-loop simulation.
Property-based test	Test asserting a statement for all generated inputs.
pytest	Third-party Python test framework; de-facto standard.
Regression	Previously working behaviour that breaks after a change.

Term	Meaning
Replay	Feeding recorded traces into the system under test.
Seam	A point where behaviour can be substituted without editing the code.
Shrinking	Simplifying a failing input to a minimal counter-example.
Slip ratio	$(\text{vehicle speed} - \text{wheel speed}) / \text{vehicle speed}$; 0 free rolling, 1 locked.
Smoke test	Minimal check that the system is basically alive.
SOC	State of charge of a battery.
Spy	A test double that records how it was used.
State-transition testing	Test design from a state machine: states, events, transitions.
Stub	A test double returning canned answers.
SUT / CUT	System / component under test.
Test double	Any stand-in for a real collaborator (dummy, stub, fake, spy, mock).
Test pyramid	Many fast unit tests, fewer integration tests, few slow system tests.
TPMS	Tyre pressure monitoring system.
UDS	Unified Diagnostic Services (ISO 14229): the diagnostic protocol of ECUs.
unittest	Python's built-in xUnit-style test framework.
V-model	Process model pairing each development level with a verification level.
Validation / verification	Building the right product / building the product right.
WCET	Worst-case execution time of a task.
xfail	A test expected to fail (known bug), tracked until fixed.

Appendix B — Further reading and resources

Testing fundamentals

- Glenford J. Myers, Corey Sandler, Tom Badgett — *The Art of Software Testing*. The classic that introduced equivalence classes and boundary analysis to many engineers.
- Paul Ammann and Jeff Offutt — *Introduction to Software Testing*. Rigorous treatment of coverage criteria (including MC/DC) and mutation testing.
- Paul C. Jorgensen — *Software Testing: A Craftsman's Approach*. Test-design techniques in depth.
- ISTQB — *Foundation Level syllabus* (free). The seven principles and a common vocabulary.
- ISO/IEC/IEEE 29119 — *Software testing* (vocabulary, processes, documentation).

Python testing

- Brian Okken — *Python Testing with pytest*. Practical, thorough guide to fixtures, parametrization and plugins.
- The official documentation: [docs.python.org](https://docs.python.org/3/library/unittest.html) (`unittest` , `unittest.mock`), [docs.pytest.org](https://docs.pytest.org/en/latest/), [hypothesis.readthedocs.io](https://hypothesis.readthedocs.io/en/latest/), [coverage.readthedocs.io](https://coverage.readthedocs.io/en/4.5.3/).
- David R. MacIver — articles on the philosophy of Hypothesis and property-based testing ("What is property-based testing?").

Design for testability and doubles

- Gerard Meszaros — *xUnit Test Patterns: Refactoring Test Code*. Source of the test-double taxonomy and many test smells.
- Kent Beck — *Test-Driven Development: By Example*.
- Steve Freeman and Nat Pryce — *Growing Object-Oriented Software, Guided by Tests*.
- Martin Fowler — article *Mocks Aren't Stubs* (classicist vs mockist styles).

Robustness and resilience

- Michael T. Nygard — *Release It!* Patterns for stability under failure.
- Articles and documentation on coverage-guided fuzzing: AFL++, libFuzzer, Atheris (Python).

Automotive and safety

- ISO 26262 *Road vehicles — Functional safety*, Part 6 (product development at the software level) and Part 8 (supporting processes).
- Automotive SPICE *Process Assessment / Reference Model* (VDA QMC) — SWE.4, SWE.5, SWE.6.
- AUTOSAR *Specification of SW-C End-to-End Communication Protection Library* (E2E profiles).
- ISO 11898 (CAN), ISO 14229 (UDS), ISO/SAE 21434 (cybersecurity engineering).
- Robert Bosch GmbH — *Bosch Automotive Handbook* for vehicle dynamics and ABS background.

Control and simulation

- Karl Åström and Richard Murray — *Feedback Systems: An Introduction for Scientists and Engineers* (free from the authors).
- Rajesh Rajamani — *Vehicle Dynamics and Control* (longitudinal control, ABS).
- The FMI standard (fmi-standard.org) for co-simulation.

Tools to explore next

`pytest-xdist` (parallel), `pytest-bdd` (Gherkin), `mutmut` / `cosmic-ray` (mutation), `python-can` and `cantools` (real CAN logs and DBC files), `tox` / `nox` (environment matrices), `ruff` / `mypy` (static analysis), `allpairspy` (pairwise generation).

Appendix C — Cheat sheet

Task	unittest	pytest
Test discovery	<code>class X(unittest.TestCase)</code> , methods <code>test_*</code>	any function <code>test_*</code> (classes optional, no base class)
Assertion	<code>self.assertEqual(a, b)</code> and ~30 other <code>assertXxx</code>	<code>assert a == b</code> (introspected failure message)
Floats	<code>assertAlmostEqual(a, b, places=7)</code>	<code>assert a == pytest.approx(b, rel=1e-6, abs=1e-12)</code>
Exception	<code>with self.assertRaises(E): / assertRaisesRegex</code>	<code>with pytest.raises(E, match="re") as info:</code>
Setup / teardown	<code>setUp</code> , <code>tearDown</code> , <code>setUpClass</code> , <code>tearDownClass</code>	fixtures with <code>yield</code> ; scopes <code>function/class/module/session</code>
Many inputs	<code>with self.subTest(x=x):</code> in a loop	<code>@pytest.mark.parametrize("x", [...], ids=[...])</code>
Skip	<code>@unittest.skip</code> , <code>skipIf</code> , <code>skipUnless</code>	<code>@pytest.mark.skip</code> , <code>skipif(cond, reason=...)</code> , <code>pytest.skip()</code>
Known failure	<code>@unittest.expectedFailure</code>	<code>@pytest.mark.xfail(reason=..., strict=True)</code>
Temp files	<code>tempfile.TemporaryDirectory</code>	<code>tmp_path</code> , <code>tmp_path_factory</code>
Patch	<code>mock.patch("pkg.mod.name")</code>	<code>monkeypatch.setattr/setenv/delattr</code> (or <code>mock.patch</code>)
Capture output	<code>contextlib.redirect_stdout</code>	<code>capsys</code> , <code>caplog</code>
Run	<code>python -m unittest discover -v</code>	<code>pytest -v</code> (also runs unittest tests)

Everyday pytest commands

```
pytest -x                    # stop at first failure
pytest --lf                  # re-run only last failures
pytest -k "cruise and not hill" # select by name expression
pytest -m "smoke"            # select by marker; -m "not slow and not performance"
pytest path/file.py::test_name[param-id] # one test
pytest -vv --tb=short        # verbose, compact tracebacks
pytest --durations=10        # slowest tests
pytest -rs                   # why were tests skipped?
pytest --pdb                 # debugger at failure
pytest --cov=autotest --cov-branch --cov-report=term-missing
pytest --junitxml=report.xml
pytest --hil / --update-golden / --req-report # this course's options
```

Choosing the right double

Double	Purpose	Example in this course
Dummy	fills a parameter, never used	listener subscribed to another id
Stub	returns canned answers	<code>StubADC(0)</code> → short-to-ground
Fake	simplified working implementation	<code>FakeClock</code> , <code>VirtualCANBus</code> , <code>SineWaveADC</code>
Spy	real object + call recording	<code>SpyADC.calls</code> , <code>SpyListener.frames</code>
Mock	verifies interactions	<code>Mock(spec=DiagnosticManager).report.assert_called_with(...)</code>

Prefer **state/outcome** assertions over interaction assertions; mock only what is slow, non-deterministic or hardware.

Test-design checklist

- One reason to fail per test; name says *what* and *under which condition*.
- Arrange – Act – Assert (or Given – When – Then).
- Boundaries: on, just below, just above. Empty, one, many. Zero, negative, NaN, huge.
- Determinism: inject clocks and RNG seeds; no `sleep`, no network, no shared state.
- Every defect fixed gets a regression test **first** (red → green).
- Tests are code: review them, refactor them, delete the ones that no longer protect anything.

The course runner

```
python course.py doctor | labs | progress | test | smoke | coverage | mutation | capstone | check-  
course | pdf  
python course.py lab N           # worked examples of lab N (verbose)  
python course.py exercise N      # your exercises of lab N
```

Appendix D — Code under test

The complete `src/autotest` library that every lab tests. Read it only after designing tests from the specifications — that is the point of Chapter 6 and Lab 3.

init.py

`src/autotest/__init__.py`

```
"""Automotive software under test for the 'Testing with Python' course."""
```

abs.py

`src/autotest/abs.py`

```
"""ABS slip controller and a quarter-car plant for software-in-the-loop tests.

The plant is ONE wheel carrying a quarter of the car. Speeds are in m/s, force in N,
torque in Nm, pressure is a fraction 0..1 of the maximum brake pressure.
"""

from __future__ import annotations

import enum
from dataclasses import dataclass

G = 9.81

class Valve(enum.Enum):
    """What the hydraulic modulator does to the brake pressure."""

    APPLY = "apply"
    HOLD = "hold"
    RELEASE = "release"

def slip_ratio(vehicle_ms: float, wheel_ms: float) -> float:
    """0 = free rolling, 1 = locked wheel. Undefined (0) at standstill."""
    if vehicle_ms <= 0.5:
        return 0.0
    return min(1.0, max(0.0, (vehicle_ms - wheel_ms) / vehicle_ms))

class ABSController:
    """Three-state bang-bang slip controller with a hysteresis band between the two limits."""

    RELEASE_ABOVE = 0.25
    APPLY_BELOW = 0.15
    MIN_ACTIVE_SPEED = 5.0 # m/s - ABS is disabled below this

    def decide(self, vehicle_ms: float, wheel_ms: float) -> Valve:
        if vehicle_ms < self.MIN_ACTIVE_SPEED:
            return Valve.APPLY
        s = slip_ratio(vehicle_ms, wheel_ms)
        if s > self.RELEASE_ABOVE:
            return Valve.RELEASE
        if s < self.APPLY_BELOW:
            return Valve.APPLY
        return Valve.HOLD

def tyre_mu(slip: float) -> float:
```

```

"""Simplified friction curve: peak 1.0 at 20 % slip, 0.7 when locked."""
if slip <= 0.2:
    return slip / 0.2
return 1.0 - 0.3 * (slip - 0.2) / 0.8

@dataclass
class QuarterCar:
    speed_ms: float = 27.8      # ~100 km/h
    wheel_ms: float = 27.8
    pressure: float = 0.0       # 0..1
    distance_m: float = 0.0
    mass: float = 400.0
    radius: float = 0.3
    inertia: float = 1.5
    max_brake_torque: float = 2000.0

    def step(self, dt: float, valve: Valve) -> None:
        """Advance the physics by `dt` seconds (explicit Euler) under the valve command."""
        # 1. Hydraulics: the valve changes brake pressure at a fixed rate (per second).
        rate = {Valve.APPLY: 10.0, Valve.HOLD: 0.0, Valve.RELEASE: -40.0}[valve]
        self.pressure = min(1.0, max(0.0, self.pressure + rate * dt))

        # 2. Tyre: slip decides how much of the normal force becomes braking force.
        mu = tyre_mu(slip_ratio(self.speed_ms, self.wheel_ms))
        force = mu * self.mass * G                                # N, decelerates the car

        # 3. Wheel: the road force spins the wheel UP, the brake torque slows it DOWN.
        wheel_omega = self.wheel_ms / self.radius                # rad/s
        brake_torque = self.pressure * self.max_brake_torque if wheel_omega > 0 else 0.0
        wheel_omega += (force * self.radius - brake_torque) / self.inertia * dt
        wheel_omega = max(0.0, wheel_omega)                        # a wheel cannot spin backwards

        if self.speed_ms <= 0.5 and wheel_omega * self.radius > self.speed_ms:
            wheel_omega = self.speed_ms / self.radius            # standing car: wheel cannot

        self.wheel_ms = wheel_omega * self.radius

        # 4. Vehicle: integrate speed and distance.
        self.speed_ms = max(0.0, self.speed_ms - force / self.mass * dt)
        self.distance_m += self.speed_ms * dt

    def simulate_braking(use_abs: bool, dt: float = 0.001, t_max: float = 15.0) -> dict:
        """Brake from ~100 km/h to a stop. Returns distance_m, time_s, peak_slip, locked_s.

        peak_slip and locked_s are only measured while ABS is allowed to act (speed >= 5 m/s).
        """
        car, ctrl = QuarterCar(), ABSController()
        t, peak_slip, locked_time = 0.0, 0.0, 0.0
        while car.speed_ms > 0.5 and t < t_max:
            valve = ctrl.decide(car.speed_ms, car.wheel_ms) if use_abs else Valve.APPLY
            car.step(dt, valve)
            if car.speed_ms >= ABSController.MIN_ACTIVE_SPEED: # below this ABS is off by design
                s = slip_ratio(car.speed_ms, car.wheel_ms)
                peak_slip = max(peak_slip, s)
                if s > 0.95:
                    locked_time += dt
            t += dt
        return {"distance_m": car.distance_m, "time_s": t, "peak_slip": peak_slip, "locked_s":
locked_time}

```

bms.py

src/autotest/bms.py

```

"""Battery management: charge-current derating, protection and state of charge (SOC).

Units used throughout: SOC in percent (0..100), temperature in degC, voltage in V,

```



```

current in A (positive = charging), capacity in Ah, time in s.
"""

from __future__ import annotations

import enum

class BMSFault(enum.Enum):
    """What the protection logic reports. NONE means "healthy"."""

    NONE = "none"
    OVERVOLTAGE = "overvoltage"
    UNDERVOLTAGE = "undervoltage"
    OVERTEMPERATURE = "overtemperature"
    UNDERTEMPERATURE = "undertemperature"

def max_charge_current(soc: float, temp_c: float, rated_a: float = 100.0) -> float:
    """Allowed charge current (A).

    Decision table (see Lab 3):
        temp < 0 or temp > 45    -> 0 A    (lithium plating / thermal risk)
        0 <= temp < 10          -> 25 % of rated
        10 <= temp <= 45        -> 100 % of rated
        soc >= 100              -> 0 A
        soc > 80                -> taper linearly from the temperature limit to 0 at 100 %
    """
    if not 0 <= soc <= 100:
        raise ValueError(f"soc must be within 0..100 percent, got {soc}")
    if temp_c < 0 or temp_c > 45:
        return 0.0
    limit = rated_a * (0.25 if temp_c < 10 else 1.0)
    if soc >= 100:
        return 0.0
    if soc > 80:
        limit *= (100 - soc) / 20
    return limit

class BatteryManagementSystem:
    """A pack monitor: protection limits (class constants) and a coulomb-counting SOC."""

    CELL_V_MAX = 4.20
    CELL_V_MIN = 3.00
    TEMP_MAX = 60.0
    TEMP_MIN = -20.0

    def __init__(self, capacity_ah: float = 50.0, soc: float = 50.0):
        if capacity_ah <= 0:
            raise ValueError(f"capacity must be positive (Ah), got {capacity_ah}")
        self.capacity_ah = capacity_ah
        self.soc = soc

    def evaluate(self, cell_voltages: list[float], temp_c: float) -> BMSFault:
        """Return the most severe fault. Priority: over-voltage, under-voltage, over-temp, under-
temp."""
        if not cell_voltages:
            raise ValueError("no cell data: cell_voltages is empty")
        if max(cell_voltages) > self.CELL_V_MAX:
            return BMSFault.OVERVOLTAGE
        if min(cell_voltages) < self.CELL_V_MIN:
            return BMSFault.UNDERVOLTAGE
        if temp_c > self.TEMP_MAX:
            return BMSFault.OVERTEMPERATURE
        if temp_c < self.TEMP_MIN:
            return BMSFault.UNDERTEMPERATURE
        return BMSFault.NONE

    def update_soc(self, current_a: float, dt_s: float) -> float:
        """Coulomb counting. Positive current charges the pack."""
        if dt_s < 0:
            raise ValueError(f"dt must be >= 0 seconds, got {dt_s}")
        delta = current_a * dt_s / 3600 / self.capacity_ah * 100

```

```

self.soc = min(100.0, max(0.0, self.soc + delta))
return self.soc

```

bus.py

src/autotest/bus.py

```

"""A virtual CAN bus with fault-injection hooks, used for integration tests."""
from __future__ import annotations

from typing import Callable, Optional

from .can import CANFrame

Listener = Callable[[CANFrame], None]
FaultHook = Callable[[CANFrame], Optional[CANFrame]]

class VirtualCANBus:
    def __init__(self) -> None:
        self._listeners: list[tuple[Listener, Optional[set[int]]]] = []
        self._fault_hooks: list[FaultHook] = []
        self.log: list[CANFrame] = []

    def subscribe(self, listener: Listener, ids: set[int] | None = None) -> None:
        self._listeners.append((listener, ids))

    def add_fault(self, hook: FaultHook) -> None:
        """A hook may return a modified frame, or None to drop it."""
        self._fault_hooks.append(hook)

    def clear_faults(self) -> None:
        self._fault_hooks.clear()

    def send(self, frame: CANFrame) -> bool:
        """Put a frame on the bus. Returns False if a fault hook dropped it. Always logs the ORIGINAL frame."""
        self.log.append(frame)
        delivered = frame
        for hook in self._fault_hooks:
            # each hook may corrupt the frame or drop it
            result = hook(delivered)
            if result is None:
                return False
            # frame lost on the bus: nobody receives it
            delivered = result
        for listener, ids in self._listeners:
            if ids is None or delivered.arbitration_id in ids:
                listener(delivered)
        return True

```

can.py

src/autotest/can.py

```

"""CAN frames, signal packing and end-to-end (E2E) protection."""
from __future__ import annotations

import enum
from dataclasses import dataclass, field

class E2EStatus(enum.Enum):
    OK = "ok"
    TOO_SHORT = "too_short"
    CRC_ERROR = "crc_error"
    COUNTER_ERROR = "counter_error"

```

```

def crc8(data: bytes, poly: int = 0x1D, init: int = 0xFF, xor_out: int = 0xFF) -> int:
    """CRC-8 SAE J1850 (the polynomial used by AUTOSAR E2E profile 1).

    Bitwise algorithm: XOR each byte into the register, then shift 8 times, XOR-ing in the
    polynomial whenever the top bit falls out. Catalogue check value: crc8(b"123456789") == 0x4B.
    """
    crc = init
    for byte in data:
        crc ^= byte
        for _ in range(8):
            crc = ((crc << 1) ^ poly) & 0xFF if crc & 0x80 else (crc << 1) & 0xFF
    return crc ^ xor_out

@dataclass(frozen=True)
class CANFrame:
    arbitration_id: int
    data: bytes = b""
    is_extended: bool = False

    def __post_init__(self) -> None:
        limit = 0x1FFFFFFF if self.is_extended else 0x7FF
        if not 0 <= self.arbitration_id <= limit:
            kind = "extended (29-bit)" if self.is_extended else "standard (11-bit)"
            raise ValueError(f"arbitration id 0x{self.arbitration_id:X} out of range for {kind}")
        frames (max 0x{limit:X})"
        if len(self.data) > 8:
            raise ValueError(f"classic CAN carries at most 8 data bytes, got {len(self.data)}")

    @property
    def dlc(self) -> int:
        return len(self.data)

@dataclass(frozen=True)
class Signal:
    """A physical value packed into a frame (Intel / little-endian bit order)."""

    name: str
    start_bit: int
    length: int
    factor: float = 1.0
    offset: float = 0.0
    minimum: float | None = None
    maximum: float | None = None
    signed: bool = False

    def __post_init__(self) -> None:
        if not 1 <= self.length <= 64 or self.start_bit < 0:
            raise ValueError(f"signal {self.name}: invalid geometry (start_bit={self.start_bit},
length={self.length}); length must be 1..64)")
        if self.factor == 0:
            raise ValueError(f"signal {self.name}: factor must not be zero")

    def raw_range(self) -> tuple[int, int]:
        if self.signed:
            return -(1 << (self.length - 1)), (1 << (self.length - 1)) - 1
        return 0, (1 << self.length) - 1

    def to_raw(self, physical: float) -> int:
        """physical value -> integer raw value (rounded to the nearest step), with range checks."""
        if physical != physical: # NaN is the only float not equal to itself
            raise ValueError(f"{self.name}: value is not a number (NaN)")
        if self.minimum is not None and physical < self.minimum:
            raise ValueError(f"{self.name}: {physical} below minimum {self.minimum}")
        if self.maximum is not None and physical > self.maximum:
            raise ValueError(f"{self.name}: {physical} above maximum {self.maximum}")
        raw = round((physical - self.offset) / self.factor)
        low, high = self.raw_range()
        if not low <= raw <= high:
            raise ValueError(f"{self.name}: raw value {raw} does not fit {self.length} bits")

```

```

        return raw

    def to_physical(self, raw: int) -> float:
        return raw * self.factor + self.offset

@dataclass(frozen=True)
class CANMessage:
    """Message definition: a frame id, a payload length (dlc) and its signals.

    `encode` / `decode` convert between {signal name: physical value} and payload bytes.
    """

    frame_id: int
    name: str
    dlc: int
    signals: tuple[Signal, ...] = field(default_factory=tuple)

    def __post_init__(self) -> None:
        if not 0 <= self.dlc <= 8:
            raise ValueError(f"message {self.name}: dlc must be 0..8, got {self.dlc}")
        for s in self.signals:
            if s.start_bit + s.length > self.dlc * 8:
                raise ValueError(f"signal {s.name} does not fit in {self.dlc} bytes")

    def encode(self, values: dict[str, float]) -> bytes:
        unknown = set(values) - {s.name for s in self.signals}
        if unknown:
            raise KeyError(f"unknown signals: {sorted(unknown)}")
        word = 0
        for s in self.signals:
            raw = s.to_raw(values.get(s.name, s.offset))
            word |= (raw & ((1 << s.length) - 1)) << s.start_bit
        return word.to_bytes(self.dlc, "little")

    def decode(self, payload: bytes) -> dict[str, float]:
        if len(payload) != self.dlc:
            raise ValueError(f"{self.name}: expected {self.dlc} bytes, got {len(payload)}")
        word = int.from_bytes(payload, "little")
        out: dict[str, float] = {}
        for s in self.signals:
            raw = (word >> s.start_bit) & ((1 << s.length) - 1)
            if s.signed and raw >= 1 << (s.length - 1):
                raw -= 1 << s.length
            out[s.name] = s.to_physical(raw)
        return out

# ---- End-to-end protection: [crc8][counter][payload...] -----

def e2e_protect(payload: bytes, counter: int) -> bytes:
    """Wire format: [crc8][counter 0..15][payload...]; the CRC covers counter + payload."""
    counter &= 0x0F
    body = bytes([counter]) + payload
    return bytes([crc8(body)]) + body

def e2e_check(data: bytes, expected_counter: int | None = None) -> tuple[E2EStatus, bytes]:
    """Return (status, payload). Payload is empty unless status is OK."""
    if len(data) < 2:
        return E2EStatus.TOO_SHORT, b""
    if crc8(data[1:]) != data[0]:
        return E2EStatus.CRC_ERROR, b""
    if expected_counter is not None and data[1] != (expected_counter & 0x0F):
        return E2EStatus.COUNTER_ERROR, b""
    return E2EStatus.OK, data[2:]

```

clock.py

src/autotest/clock.py

```

"""Injectable clocks: production code takes a clock, tests pass a FakeClock."""
import time

class SystemClock:
    def now(self) -> float:
        return time.monotonic()

class FakeClock:
    """Deterministic clock - the key trick for testing timeouts without sleeping."""

    def __init__(self, start: float = 0.0):
        self._t = start

    def now(self) -> float:
        return self._t

    def advance(self, seconds: float) -> None:
        if seconds < 0:
            raise ValueError("time cannot go backwards")
        self._t += seconds

```

cruise.py

src/autotest/cruise.py

```

"""Cruise control: a mode state machine (OFF/STANDBY/ACTIVE/OVERRIDE) plus a PI speed controller.

Speeds in km/h, time in s, throttle request in 0..1. Driver inputs are methods; `control()`
is the periodic control task that turns speed error into a throttle request.
"""

from __future__ import annotations

import enum

class CruiseState(enum.Enum):
    OFF = "off"
    STANDBY = "standby"
    ACTIVE = "active"
    OVERRIDE = "override" # driver is pressing the accelerator

class CruiseController:
    MIN_SET_SPEED = 30.0 # km/h
    MAX_SET_SPEED = 180.0 # km/h

    def __init__(self, kp: float = 0.05, ki: float = 0.01):
        self.kp, self.ki = kp, ki
        self.state = CruiseState.OFF
        self.target: float | None = None
        self.saved_target: float | None = None
        self._integral = 0.0

    # --- driver inputs -----
    def power_on(self) -> None:
        """OFF -> STANDBY (ignored in any other state)."""
        if self.state is CruiseState.OFF:
            self.state = CruiseState.STANDBY

    def power_off(self) -> None:
        """Any state -> OFF, forgetting both the target and the saved target."""
        self.state = CruiseState.OFF
        self.target = self.saved_target = None
        self._integral = 0.0

    def set(self, speed_kmh: float) -> bool:

```

```

        """Engage / change the set speed. Returns False if the request is refused."""
        if self.state is CruiseState.OFF:
            return False
        if not self.MIN_SET_SPEED <= speed_kmh <= self.MAX_SET_SPEED:
            return False
        self.target = speed_kmh
        self.saved_target = speed_kmh
        self.state = CruiseState.ACTIVE
        self._integral = 0.0
        return True

    def resume(self, current_speed_kmh: float) -> bool:
        """STANDBY -> ACTIVE at the saved target; refused without one or below the minimum speed."""
        if self.state is not CruiseState.STANDBY or self.saved_target is None:
            return False
        if current_speed_kmh < self.MIN_SET_SPEED:
            return False
        return self.set(self.saved_target)

    def cancel(self) -> None:
        """ACTIVE/OVERRIDE -> STANDBY, remembering the target so `resume` can restore it."""
        if self.state in (CruiseState.ACTIVE, CruiseState.OVERRIDE):
            self.saved_target = self.target
            self.target = None
            self.state = CruiseState.STANDBY
            self._integral = 0.0

    brake = cancel # pressing the brake pedal always cancels

    def accelerator(self, pressed: bool) -> None:
        """Driver override: pressing in ACTIVE -> OVERRIDE; releasing in OVERRIDE -> ACTIVE."""
        if pressed and self.state is CruiseState.ACTIVE:
            self.state = CruiseState.OVERRIDE
        elif not pressed and self.state is CruiseState.OVERRIDE:
            self.state = CruiseState.ACTIVE
            self._integral = 0.0

# --- control law -----
def control(self, speed_kmh: float, dt: float) -> float:
    """Throttle request 0..1. Zero unless ACTIVE.

    PI law:  $u = K_p * e + K_i * (\text{integral of } e)$ . The result is clamped to 0..1, and the integral is
    only updated while the output is NOT clamped (conditional integration = anti-windup).
    """
    if self.state is not CruiseState.ACTIVE or self.target is None:
        return 0.0
    error = self.target - speed_kmh # km/h, positive = too slow
    unsaturated = self.kp * error + self.ki * (self._integral + error * dt)
    output = min(1.0, max(0.0, unsaturated))
    if output == unsaturated: # not saturated: safe to
        self._integral += error * dt
    return output

```

diagnostics.py

src/autotest/diagnostics.py

```

"""Diagnostic trouble codes (DTC) with debouncing and a UDS-lite request handler."""
from __future__ import annotations

from dataclasses import dataclass

NRC_SERVICE_NOT_SUPPORTED = 0x11
NRC_INCORRECT_LENGTH = 0x13
NRC_REQUEST_OUT_OF_RANGE = 0x31

@dataclass

```

```

class _DTC:
    fail_count: int = 0
    pass_count: int = 0
    active: bool = False
    stored: bool = False

class DiagnosticManager:
    """A DTC is confirmed after `fail_threshold` consecutive failures and
    stops being active after `heal_threshold` consecutive passes (it stays stored)."""

    VIN = b"WAUZZZ8K9BA123456"

    def __init__(self, fail_threshold: int = 3, heal_threshold: int = 5):
        if fail_threshold < 1 or heal_threshold < 1:
            raise ValueError(f"thresholds must be >= 1, got fail={fail_threshold}, heal={
{heal_threshold}")
        self.fail_threshold = fail_threshold
        self.heal_threshold = heal_threshold
        self._dtcs: dict[str, _DTC] = {}

    @staticmethod
    def _validate(code: str) -> None:
        if len(code) != 5 or code[0] not in "PCBU" or not all(c in "0123456789ABCDEF" for c in
code[1:]):
            raise ValueError(f"malformed DTC {code!r}")

    def report(self, code: str, failed: bool) -> None:
        """Feed one monitoring cycle: `failed=True` if the monitor saw the fault in this cycle."""
        self._validate(code)
        d = self._dtcs.setdefault(code, _DTC())
        if failed:
            d.fail_count, d.pass_count = d.fail_count + 1, 0
            if d.fail_count >= self.fail_threshold:
                d.active = d.stored = True
        else:
            d.pass_count, d.fail_count = d.pass_count + 1, 0
            if d.pass_count >= self.heal_threshold:
                d.active = False

    def failure_count(self, code: str) -> int:
        """Current streak of consecutive failed cycles for `code` (0 if never reported)."""
        d = self._dtcs.get(code)
        return d.fail_count if d else 0

    def active_codes(self) -> list[str]:
        return sorted(c for c, d in self._dtcs.items() if d.active)

    def stored_codes(self) -> list[str]:
        return sorted(c for c, d in self._dtcs.items() if d.stored)

    def clear_all(self) -> None:
        self._dtcs.clear()

    def export_report(self) -> str:
        lines = ["DTC REPORT", "====="]
        for code in sorted(self._dtcs):
            d = self._dtcs[code]
            lines.append(f"{code} active={d.active} stored={d.stored}")
        if len(lines) == 2:
            lines.append("(no trouble codes)")
        return "\n".join(lines) + "\n"

    # --- UDS-lite (ISO 14229 subset) -----
    @staticmethod
    def _negative(service: int, nrc: int) -> bytes:
        return bytes([0x7F, service, nrc])

    def handle_request(self, request: bytes) -> bytes:
        """Answer one UDS request (simplified subset). Always returns a well-formed response."""
        if not request:
            return self._negative(0x00, NRC_INCORRECT_LENGTH)
        sid = request[0]

```

```

if sid == 0x19: # ReadDTCInformation, sub-function 0x02 = by status mask
    if len(request) != 3:
        return self._negative(sid, NRC_INCORRECT_LENGTH)
    if request[1] != 0x02:
        return self._negative(sid, NRC_REQUEST_OUT_OF_RANGE)
    body = b"".join(c.encode() + b";" for c in self.stored_codes())
    return bytes([0x59, 0x02]) + body
if sid == 0x14: # ClearDiagnosticInformation
    self.clear_all()
    return bytes([0x54])
if sid == 0x22: # ReadDataByIdentifier
    if len(request) != 3:
        return self._negative(sid, NRC_INCORRECT_LENGTH)
    if request[1:3] != b"\xF1\x90":
        return self._negative(sid, NRC_REQUEST_OUT_OF_RANGE)
    return bytes([0x62, 0xF1, 0x90]) + self.VIN
return self._negative(sid, NRC_SERVICE_NOT_SUPPORTED)

```

ecu.py

src/autotest/ecu.py

```

"""Two cooperating ECUs: a wheel-speed sender and an instrument cluster receiver.

On the wire a wheel-speed frame (id 0x1A0) is 5 bytes:

    byte 0      byte 1      bytes 2-4
    CRC-8      alive counter  payload: speed (16 bit, 0.01 km/h) + valid flag (1 bit)
"""
from __future__ import annotations

from .bus import VirtualCANBus
from .can import CANFrame, CANMessage, E2EStatus, Signal, e2e_check, e2e_protect
from .clock import FakeClock, SystemClock
from .diagnostics import DiagnosticManager

WHEEL_SPEED_MSG = CANMessage(
    frame_id=0x1A0,
    name="WheelSpeed",
    dlc=3,
    signals=(
        Signal("speed_kmh", 0, 16, factor=0.01, minimum=0.0, maximum=300.0),
        Signal("valid", 16, 1),
    ),
)

DTC_COMM_LOST = "U0121"
DTC_E2E_FAIL = "U0100"

class WheelSpeedECU:
    def __init__(self, bus: VirtualCANBus):
        self.bus = bus
        self.counter = 0

    def send_speed(self, speed_kmh: float) -> bool:
        payload = WHEEL_SPEED_MSG.encode({"speed_kmh": speed_kmh, "valid": 1})
        data = e2e_protect(payload, self.counter)
        self.counter = (self.counter + 1) & 0x0F
        return self.bus.send(CANFrame(WHEEL_SPEED_MSG.frame_id, data))

class InstrumentCluster:
    TIMEOUT_S = 0.1

    def __init__(self, bus: VirtualCANBus, clock: FakeClock | SystemClock | None = None,
                 diag: DiagnosticManager | None = None):
        self.clock = clock or SystemClock()
        self.diag = diag or DiagnosticManager(fail_threshold=2)

```



```

self._speed: float | None = None
self._last_rx = self.clock.now()
self._expected_counter: int | None = None
bus.subscribe(self._on_frame, ids={WHEEL_SPEED_MSG.frame_id})

def _on_frame(self, frame: CANFrame) -> None:
    status, payload = e2e_check(frame.data, self._expected_counter)
    if status is not E2EStatus.OK:
        self.diag.report(DTC_E2E_FAIL, failed=True)
        if status is E2EStatus.COUNTER_ERROR: # resynchronise after a lost frame
            self._expected_counter = (frame.data[1] + 1) & 0x0F
        elif self._expected_counter is not None:
            # BUG-130: a corrupted frame still occupies one slot in the sequence, so the
            # next good frame must not be rejected as a counter error as well.
            self._expected_counter = (self._expected_counter + 1) & 0x0F
        return
    self.diag.report(DTC_E2E_FAIL, failed=False)
    signals = WHEEL_SPEED_MSG.decode(payload)
    self._speed = signals["speed_kmh"] if signals["valid"] else None
    self._last_rx = self.clock.now()
    self._expected_counter = (frame.data[1] + 1) & 0x0F

def displayed_speed(self) -> str:
    """What the driver sees: the speed, or "--" when it is unknown or stale.

    Call this once per display cycle. It also *supervises* the bus: every call checks the
    100 ms timeout and reports the result to diagnostics, so it is not a pure getter.
    """
    if self.clock.now() - self._last_rx > self.TIMEOUT_S:
        self.diag.report(DTC_COMM_LOST, failed=True)
        return "--"
    self.diag.report(DTC_COMM_LOST, failed=False)
    return "--" if self._speed is None else f"{self._speed:.0f}"

```

learn.py

src/autotest/learn.py

```

"""Helpers for the student exercises (not part of the system under test)."""

def todo(message: str = "") -> None:
    """Mark a step you have not done yet: fails the test with a readable TODO message.

    Every exercise test starts with a call to ``todo(...)``. When you begin working on that
    test, DELETE the call. The test then passes only if your own code is correct.
    """
    __tracebackhide__ = True # pytest: do not show this helper frame, only YOUR test
    raise AssertionError(f"TODO - {message}" if message else "TODO - complete this exercise")

```

sensors.py

src/autotest/sensors.py

```

"""Sensor conversions and a plausibility-checked temperature sensor."""
from __future__ import annotations

from collections import deque
from typing import Protocol

class SensorFault(Exception):
    """Raised when a raw reading is physically implausible."""

```

```

def kmh_to_ms(kmh: float) -> float:
    return kmh / 3.6

def ms_to_kmh(ms: float) -> float:
    return ms * 3.6

def celsius_to_fahrenheit(c: float) -> float:
    return c * 9 / 5 + 32

def wheel_speed_kmh(pulses: int, pulses_per_rev: int, circumference_m: float, interval_s: float) ->
float:
    """Speed from a toothed-wheel sensor: pulses counted during `interval_s` seconds -> km/h."""
    if pulses < 0 or pulses_per_rev <= 0 or circumference_m <= 0 or interval_s <= 0:
        raise ValueError(
            f"invalid wheel speed parameters: pulses={pulses}, pulses_per_rev={pulses_per_rev}, "
            f"circumference_m={circumference_m}, interval_s={interval_s} (all must be positive;
pulses may be 0)"
        )
    revs_per_s = pulses / pulses_per_rev / interval_s
    return ms_to_kmh(revs_per_s * circumference_m)

class ADC(Protocol):
    """Hardware abstraction: real boards, simulators and mocks all fit this."""

    def read(self) -> int: ...

class TemperatureSensor:
    """12-bit ADC mapped linearly to -40..150 degC. Rail values mean a wiring fault."""

    ADC_MAX = 4095
    T_MIN, T_MAX = -40.0, 150.0

    def __init__(self, adc: ADC, filter_len: int = 4):
        if filter_len < 1:
            raise ValueError(f"filter_len must be >= 1, got {filter_len}")
        self._adc = adc
        self._window: deque[float] = deque(maxlen=filter_len)

    def read(self) -> float:
        """One conversion -> degC. Raises SensorFault if the ADC sits at either rail (wiring
fault)."""
        counts = self._adc.read()
        if counts <= 0:
            raise SensorFault("short to ground")
        if counts >= self.ADC_MAX:
            raise SensorFault("open circuit / short to supply")
        return self.T_MIN + (self.T_MAX - self.T_MIN) * counts / self.ADC_MAX

    def read_filtered(self) -> float:
        """Moving average of the last `filter_len` good readings (a faulty read leaves the window
untouched)."""
        self._window.append(self.read())
        return sum(self._window) / len(self._window)

```

tpms.py

src/autotest/tpms.py

```

"""Tyre pressure monitoring - the capstone system under test.

Specification (all pressures in kPa, temperatures in degC)
S1 Pressure is normalised to 20 degC:  $p_{20} = p * 293.15 / (273.15 + temp)$ 
S2 Nominal pressure is 230 kPa.
S3  $p_{20} < 60$  % of nominal -> CRITICAL

```

```

S4 60 % <= p20 < 80 % of nominal      -> LOW
S5 80 % <= p20 <= 130 % of nominal    -> OK
S6 p20 > 130 % of nominal             -> HIGH
S7 A drop of >= 20 kPa within a 60 s window -> rapid loss (leak) alarm
S8 Sensor ids are exactly 8 hex digits (either case); anything else is rejected with
   ValueError. Valid ids are returned in UPPER case.
S9 Readings outside 0..700 kPa or -40..125 degC are rejected with ValueError.
"""
from __future__ import annotations

import enum
import re
from collections import deque

NOMINAL_KPA = 230.0

class TyreStatus(enum.Enum):
    CRITICAL = "critical"
    LOW = "low"
    OK = "ok"
    HIGH = "high"

def normalise_pressure(kpa: float, temp_c: float) -> float:
    return kpa * 293.15 / (273.15 + temp_c)

def classify(kpa: float, temp_c: float, nominal: float = NOMINAL_KPA) -> TyreStatus:
    if not 0 <= kpa <= 700 or not -40 <= temp_c <= 125:
        raise ValueError(f"reading out of sensor range: {kpa} kPa (0..700), {temp_c} degC
(-40..125)")
    p20 = normalise_pressure(kpa, temp_c)
    if p20 < 0.6 * nominal:
        return TyreStatus.CRITICAL
    if p20 < 0.8 * nominal:
        return TyreStatus.LOW
    if p20 <= 1.3 * nominal:
        return TyreStatus.OK
    return TyreStatus.HIGH

_SENSOR_ID = re.compile(r"[0-9A-Fa-f]{8}")

def validate_sensor_id(sensor_id: str) -> str:
    if not _SENSOR_ID.fullmatch(sensor_id):
        raise ValueError(f"bad sensor id {sensor_id!r}")
    return sensor_id.upper()

class LeakDetector:
    WINDOW_S = 60.0
    DROP_KPA = 20.0

    def __init__(self) -> None:
        self._samples: deque[tuple[float, float]] = deque()

    def add(self, t: float, kpa: float) -> bool:
        """Record a sample, return True when a rapid loss is detected."""
        self._samples.append((t, kpa))
        while self._samples and t - self._samples[0][0] > self.WINDOW_S:
            self._samples.popleft()
        highest = max(p for _, p in self._samples)
        return highest - kpa >= self.DROP_KPA

```

vehicle.py

src/autotest/vehicle.py

```

"""Longitudinal vehicle model (point mass + drag + rolling resistance + grade)."""
from __future__ import annotations

import math
from dataclasses import dataclass

from .cruise import CruiseController

G = 9.81

@dataclass
class Vehicle:
    speed_ms: float = 0.0
    mass: float = 1500.0
    max_drive_force: float = 2500.0 # N at full throttle
    cd_a: float = 0.66 # drag coefficient x frontal area (m^2)
    crr: float = 0.01 # rolling resistance coefficient
    rho: float = 1.2 # air density

    def step(self, throttle: float, dt: float, grade: float = 0.0, brake_force: float = 0.0) ->
float:
        """Advance by `dt` s. `grade` is rise/run (0.05 = 5 % hill). Returns the new speed in km/h.

        Newton:  $m \cdot dv/dt = \text{drive} - \text{drag} - \text{rolling} - \text{slope} - \text{brake}$ .
        """
        throttle = min(1.0, max(0.0, throttle))
        drag = 0.5 * self.rho * self.cd_a * self.speed_ms ** 2
        rolling = self.crr * self.mass * G if self.speed_ms > 0 else 0.0
        slope = self.mass * G * math.sin(math.atan(grade))
        net = throttle * self.max_drive_force - drag - rolling - slope - brake_force
        self.speed_ms = max(0.0, self.speed_ms + net / self.mass * dt)
        return self.speed_ms * 3.6

    @property
    def speed_kmh(self) -> float:
        return self.speed_ms * 3.6

def run_cruise(cc: CruiseController, start_kmh: float, duration_s: float, dt: float = 0.1,
               grade_profile=lambda t: 0.0, events=None):
    """Closed-loop simulation. `events` maps time (s) -> callable(cc, vehicle)."""
    car = Vehicle(speed_ms=start_kmh / 3.6)
    events = dict(events or {})
    trace = []
    for k in range(int(duration_s / dt)):
        t = round(k * dt, 6)
        if t in events:
            events[t](cc, car)
        throttle = cc.control(car.speed_kmh, dt)
        car.step(throttle, dt, grade_profile(t))
        trace.append((t, car.speed_kmh, throttle))
    return trace

```

Appendix E — Course tooling

Shared fixtures and options

conftest.py

```
"""Shared fixtures and a tiny requirement-traceability plugin for the whole course."""
from __future__ import annotations

from collections import defaultdict
from pathlib import Path

import pytest

from autotest.bus import VirtualCANBus
from autotest.clock import FakeClock

def pytest_addoption(parser):
    g = parser.getgroup("autotest course")
    g.addoption("--hil", action="store_true", help="run tests marked 'hil' (needs hardware)")
    g.addoption("--update-golden", action="store_true", help="rewrite golden files (Lab 9)")
    g.addoption("--req-report", action="store_true", help="print requirement traceability matrix
(Lab 13)")

def pytest_collection_modifyitems(config, items):
    if not config.getoption("--hil"):
        skip_hil = pytest.mark.skip(reason="needs hardware: run with --hil")
        for item in items:
            if "hil" in item.keywords:
                item.add_marker(skip_hil)

# ---- shared fixtures -----
@pytest.fixture
def clock() -> FakeClock:
    return FakeClock()

@pytest.fixture
def bus() -> VirtualCANBus:
    return VirtualCANBus()

@pytest.fixture
def golden(request):
    """Compare text with a stored golden file; `--update-golden` rewrites it."""
    update = request.config.getoption("--update-golden")
    base = Path(request.node.fspath).parent / "golden"

    def check(name: str, actual: str) -> None:
        path = base / name
        if update or not path.exists():
            base.mkdir(exist_ok=True)
            path.write_text(actual, encoding="utf-8")
            if not update:
                pytest.fail(f"golden file {path} did not exist - created it; re-run to verify")
            return
        assert actual == path.read_text(encoding="utf-8"), f"output differs from {path.name};
inspect, then --update-golden"

    return check

# ---- requirement traceability -----
_trace: dict[str, list[tuple[str, str]]] = defaultdict(list)
```

```

def pytest_runtest_logreport(report):
    if report.when == "call":
        for req in getattr(report, "_reqs", []):
            _trace[req].append((report.nodeid, report.outcome))

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    rep = outcome.get_result()
    rep._reqs = [m.args[0] for m in item.iter_markers("requirement") if m.args]

def pytest_terminal_summary(terminalreporter, config):
    if not config.getoption("--req-report"):
        return
    tr = terminalreporter
    tr.section("requirement traceability")
    for req in sorted(_trace):
        results = _trace[req]
        verdict = "PASS" if all(o == "passed" for _, o in results) else "FAIL"
        tr.write_line(f"{req:<14} {verdict} ({len(results)} tests)")
        for nodeid, outcome in results:
            tr.write_line(f"    {outcome:<7} {nodeid}")

```

Mutation tester (Lab 12)

tools/mini_mutate.py

```

#!/usr/bin/env python3
"""mini_mutate - a ~120 line mutation tester, small enough to read in class.

Idea: coverage tells you which lines RAN; mutation testing tells you whether your
assertions would NOTICE if the line were wrong. We make one small change (a "mutant")
to the code under test, run the tests, and expect them to FAIL ("kill" the mutant).
A mutant that survives points at a missing or weak assertion (or is an "equivalent mutant":
changed code, identical behaviour, which no test can kill).

Exit status: 0 normally (survivors are information, not an error), 1 with --strict if any survive,
2 if the unmutated code already fails the tests.

Example
python tools/mini_mutate.py --target src/autotest/bms.py --function max_charge_current \
    --tests labs/lab03_black_box/test_lab03_black_box.py
"""
from __future__ import annotations

import argparse
import ast
import copy
import os
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

SWAPS = {
    ast.Lt: ast.LtE, ast.LtE: ast.Lt, ast.Gt: ast.GtE, ast.GtE: ast.Gt,
    ast.Eq: ast.NotEq, ast.NotEq: ast.Eq,
    ast.Add: ast.Sub, ast.Sub: ast.Add, ast.Mult: ast.Div, ast.Div: ast.Mult,
    ast.And: ast.Or, ast.Or: ast.And,
}

class Mutator(ast.NodeTransformer):
    """Visits every mutable site in order; mutates only the one whose index == target."""

```

```

def __init__(self, target: int, function: str | None):
    self.target, self.function = target, function
    self.count, self.sites, self._inside = 0, [], function is None

def visit_FunctionDef(self, node):
    was = self._inside
    if self.function and node.name == self.function:
        self._inside = True
    self.generic_visit(node)
    self._inside = was
    return node

def _site(self, node, description):
    index, self.count = self.count, self.count + 1
    self.sites.append((node.lineno, description))
    return index == self.target

def visit_Compare(self, node):
    self.generic_visit(node)
    if self._inside:
        for i, op in enumerate(node.ops):
            if type(op) in SWAPS and self._site(node, f"{type(op).__name__} -> {SWAPS[type(op)].__name__}"):
                node.ops[i] = SWAPS[type(op)]()
        return node

def visit_BinOp(self, node):
    self.generic_visit(node)
    if self._inside and type(node.op) in SWAPS and self._site(node, f"{type(node.op).__name__} -> {SWAPS[type(node.op)].__name__}"):
        node.op = SWAPS[type(node.op)]()
    return node

def visit_BoolOp(self, node):
    self.generic_visit(node)
    if self._inside and self._site(node, f"{type(node.op).__name__} -> {SWAPS[type(node.op)].__name__}"):
        node.op = SWAPS[type(node.op)]()
    return node

def visit_Constant(self, node):
    if self._inside and isinstance(node.value, (int, float)) and not isinstance(node.value, bool):
        if self._site(node, f"constant {node.value!r} -> {node.value + 1!r}"):
            return ast.copy_location(ast.Constant(node.value + 1), node)
    return node

def run_tests(workdir: Path, tests: list[str], timeout: int) -> bool:
    """True when the test run PASSED."""
    # Without this, two same-sized mutants written within one second would reuse the previous
    # mutant's stale .pyc (bytecode is validated by source mtime in whole seconds + size).
    env = {**os.environ, "PYTHONDONTWRITEBYTECODE": "1"}
    try:
        r = subprocess.run([sys.executable, "-m", "pytest", "-x", "-q", "-p", "no:cacheprovider",
*tests],
                           cwd=workdir, capture_output=True, timeout=timeout, env=env)
    except subprocess.TimeoutExpired:
        return False # an infinite loop counts as detected
    return r.returncode == 0

def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--target", required=True, help="python file to mutate (relative to repo root)")
    ap.add_argument("--function", help="only mutate inside this function")
    ap.add_argument("--tests", nargs="+", required=True, help="test files / dirs to run")
    ap.add_argument("--timeout", type=int, default=60)
    ap.add_argument("--list", action="store_true", help="only list mutation sites")
    ap.add_argument("--strict", action="store_true",
                    help="exit with status 1 if any mutant survives (for CI gates; survivors can be
equivalent mutants)")

```

```

args = ap.parse_args()

root = Path(__file__).resolve().parents[1]
target = root / args.target
tree = ast.parse(target.read_text(encoding="utf-8"))
probe = Mutator(-1, args.function)
probe.visit(copy.deepcopy(tree))
print(f"{len(probe.sites)} mutants in {args.target}" + (f"::{args.function}" if args.function
else ""))

if args.list:
    for i, (line, desc) in enumerate(probe.sites):
        print(f" #{i:02d} line {line}: {desc}")
    return 0

with tempfile.TemporaryDirectory() as tmp:
    work = Path(tmp) / "proj"
    shutil.copytree(root, work, ignore=shutil.ignore_patterns(
        ".git", "__pycache__", ".pytest_cache", ".hypothesis", ".htmlcov", ".coverage"))
    mutated_file = work / args.target
    if not run_tests(work, args.tests, args.timeout):
        print("The unmutated code already FAILS the tests - fix that first.")
        return 2
    survivors = []
    for i, (line, desc) in enumerate(probe.sites):
        m = Mutator(i, args.function)
        mutated_file.write_text(ast.unparse(m.visit(copy.deepcopy(tree))), encoding="utf-8")
        killed = not run_tests(work, args.tests, args.timeout)
        print(f" #{i:02d} line {line:>3} {desc:<28} {'killed' if killed else 'SURVIVED'}")
        if not killed:
            survivors.append((i, line, desc))
    total = len(probe.sites)
    score = 100 * (total - len(survivors)) / total if total else 100.0
    print(f"\nmutation score: {total - len(survivors)}/{total} = {score:.0f} %")
    for i, line, desc in survivors:
        print(f" survivor #{i:02d} (line {line}): {desc} -> which assertion should have
failed?")

    return 1 if (survivors and args.strict) else 0

if __name__ == "__main__":
    sys.exit(main())

```

Requirements used for traceability (Lab 13)

docs/requirements.csv

```

id,component,text
REQ-CAN-001,CAN,A frame shall reject arbitration ids outside the 11-bit (standard) or 29-bit (extended)
range.
REQ-CAN-002,CAN,Signal encode followed by decode shall reproduce the physical value within one resolution
step.
REQ-CAN-003,CAN,Receivers shall reject frames whose CRC or alive counter is invalid.
REQ-BMS-001,BMS,Charging shall be inhibited below 0 degC and above 45 degC.
REQ-BMS-002,BMS,Charge current shall taper linearly to zero between 80 % and 100 % SOC.
REQ-BMS-003,BMS,Any cell above 4.20 V shall raise an overvoltage fault.
REQ-CC-001,Cruise,Cruise control shall only engage between 30 and 180 km/h.
REQ-CC-002,Cruise,Pressing the brake shall cancel cruise control immediately.
REQ-CC-003,Cruise,Throttle output shall be zero unless cruise control is active.
REQ-ABS-001,ABS,With ABS the wheel shall not lock while vehicle speed is above 5 m/s.
REQ-ABS-002,ABS,ABS shall shorten the stopping distance compared with locked-wheel braking on the reference
surface.
REQ-DIAG-001,Diagnostics,A DTC shall be confirmed only after N consecutive failed debounce cycles.
REQ-DIAG-002,Diagnostics,Unsupported diagnostic services shall be answered with negative response code 0x11.
REQ-CLU-001,Cluster,The cluster shall display "---" when no valid speed frame arrived for 100 ms.
REQ-TPMS-001,TPMS,A tyre below 60 % of nominal pressure (normalised to 20 degC) shall be classified
CRITICAL.
REQ-TPMS-002,TPMS,A pressure drop of 20 kPa or more within a 60 s window shall raise a rapid-loss alarm.

```


Appendix F — Self-check answers

Answers to the *Check your understanding* questions at the end of each chapter. Try the questions first.

Chapter 1 — Why we test: foundations

1. *Error*: the developer assumed temperatures are always real numbers. *Fault*: comparisons with NaN are all false, so neither temperature limit triggers. *Failure*: full charge current is permitted at an unknown temperature.
2. Verification — it checks the code against a stated rule. Validation would ask whether "100 A at 25 °C and 50 % SOC" is actually what the cell supplier and safety case require, i.e. whether the rule itself is right.
3. Tests show presence, not absence, of defects. Ten thousand tests that all probe the same easy cases reveal nothing about boundaries, rare combinations or hostile inputs, and the suite itself may have weak assertions (Chapter 15).
4. Long-lived platforms with many variants, frequent software updates, and safety evidence that must be re-established after every change make manual re-testing economically and practically impossible.

Chapter 2 — The testing landscape

1. (a) Unit; (b) integration (two ECUs and a bus, with timing); (c) system-level closed-loop (SIL).
2. It is an "ice-cream cone": slow, brittle, hard to localise failures, expensive to run, so it is run rarely and trusted little. A broad base of fast unit tests should carry most of the weight.
3. SIL runs the production code on a PC against a simulated plant; HIL runs the real ECU against a real-time simulator. SIL is far faster, cheaper, parallelisable and available before hardware exists; it catches most logic defects early, leaving HIL for hardware/timing effects.
4. It has no independent oracle — both sides come from the code, so no bug can make it fail. The expected value must come from the specification (50.0).

Chapter 3 — Automotive primer: the domain you will test

1. A CRC only proves the *bits* are intact. A frame that is perfectly valid but old — repeated by a stuck sender, replayed, or arriving after intermediate frames were lost — still passes the CRC. The counter shows whether the sequence is continuous.
2. $130 \times 0.25 - 40 = -7.5$. Resolution is the factor: 0.25.
3. No: the failures were never three *consecutive* ones (the pass in the middle resets the streak). Debouncing avoids storing codes for transient glitches such as a single noisy sample.
4. Beyond the peak slip, friction *falls*: the locked wheel decelerates the car less and cannot steer. ABS deliberately releases pressure to stay near the peak.

Chapter 4 — unittest: the standard library's test framework

1. The charge changes the shared object; any test running afterwards sees a different SOC, so results depend on execution order — hidden coupling. Build the object in `setUp` instead.
2. With `try/except ... pass`, a function that raises nothing still passes. `assertRaises` fails if no exception is raised, so the test genuinely verifies the behaviour.

3. A failure is an assertion that was violated (the code disagreed with the oracle). An error is any other exception — the test or the code crashed in an unanticipated way.
4. `expectedFailure` keeps *running* the test and reports an unexpected success when the bug is fixed, forcing cleanup. `skip` hides it entirely and the test can rot unnoticed.

Chapter 5 — pytest: concise, powerful tests

1. It is the teardown. It runs after the test that used the fixture, even if that test failed or errored.
2. $3 \times 4 = 12$ (the cartesian product).
3. Unregistered, a misspelled marker is silently accepted and the tests are never selected; with `--strict-markers` it is an immediate error.
4. A non-strict `xfail` quietly reports "XPASS" and the run stays green, so the marker lingers forever. Strict `xfail` makes the unexpected pass a *failure*, forcing the marker to be removed.

Chapter 6 — Designing tests from a specification

1. Lower boundary 0: -1, 0, 1. Upper boundary 100: 99, 100, 101 → -1 and 101 are invalid (expect rejection), the rest valid.
2. It forces you to enumerate *combinations* and so exposes interactions (e.g. cold *and* tapering) and specification gaps that single-condition tests never reach.
3. $4 \times 3 \times 2 = 24$ exhaustive combinations. Pairwise only needs every *pair* of values to appear once; the largest pair is $4 \times 3 = 12$, so roughly a dozen tests suffice.
4. A value copied from the code's own output merely freezes the current behaviour — including any bug. The test could then never fail for a wrong result.

Chapter 7 — Isolating the unit: testability and test doubles

1. (a) stub; (b) mock; (c) fake; (d) spy.
2. `ecu.py` imported the name with `from .can import e2e_protect`, so it holds its own reference. Patching the original module changes a different binding; patch `autotest.ecu.e2e_protect` instead.
3. When the *call itself* is the externally required behaviour — e.g. "a DTC must be reported" and there is no other observable state — or when the collaborator is a boundary you must not actually invoke (sending a real message).
4. The fake clock advances instantly (no real waiting) and exactly (101 ms is precisely 101 ms), whereas `sleep` waits real time and its duration is subject to scheduler jitter on loaded machines.

Chapter 8 — Integration testing: where components meet

1. For instance: mismatched byte order or scaling between sender and receiver; incompatible alive-counter wrap; inconsistent timeouts; receiver ignoring a validity flag.
2. The round trip uses the *same* definition on both sides, so a consistent wrong change still round-trips. Fixed bytes taken from the written specification fail as soon as either side drifts.
3. An integration test with a fake clock that advances by the sender's cycle (20 ms) and checks the receiver still shows data — it fails because $20\text{ ms} > 15\text{ ms}$. The clock is simulated, so no real waiting is needed.
4. Add a unit test (or lowest-level reproduction) for the defect so it is guarded where it is cheapest and most precise; keep the integration test as the system-level check.

Chapter 9 — State-based and system-level testing

1. $5 \times 6 = 30$ cells. Covering only valid arrows misses events that should be *ignored or refused* in states where they are meaningless, which is where unintended behaviour hides.
2. 343+ sequences make hand-written expected states impractical and error-prone; invariants (e.g. "no throttle outside ACTIVE") are short, always true, and detect hazards in every reachable state.
3. "Then cruise control is STANDBY, the target is cleared, the saved target keeps the previous set speed, and the throttle request is 0."
4. Anti-windup stops the integral term accumulating while the actuator is saturated. Without it the controller overshoots badly and takes long to settle after a long saturation (e.g. after a hill or a big set-speed step).

Chapter 10 — Property-based testing and fuzzing

1. A metamorphic property (a known relation between two runs of the system).
2. It re-implements the code in the test, so any bug in the logic is duplicated and the test cannot fail for the right reason. A property should state something *independent* — an invariant, a round trip, a simple reference model of the specification.
3. `assume()` or `.filter()` rejected nearly all generated inputs. Generate valid inputs directly (build the structure you need, e.g. base + small delta) instead of filtering.
4. "The server must survive and answer with a well-formed response (positive or negative) — it must never crash, hang or return garbage."

Chapter 11 — Robustness and fault injection

1. Both are fail-safe (rejecting the input, or allowing 0 A). Returning the *full* current for an unknown temperature is not acceptable.
2. The oracle is a safety property: whatever the cluster shows must be a value that was actually sent (or `--`). Exact values cannot be predicted because which frames survive is random.
3. If `f(x)` raises nothing, the test still passes; and catching `ValueError` broadly hides whether the *right* error occurred. Use `pytest.raises(ValueError, match=...)`.
4. That the fault is *detected* (and reported), that the system moves to the *safe state / reaction*, and that after the fault is removed it *recovers* (and its history, such as stored DTCs, is retained appropriately).

Chapter 12 — Regression testing, golden files and replay

1. Test first (it must fail, red), then the fix (green). Also verify the test fails without the fix — otherwise it guards nothing.
2. Either the refactoring really changed behaviour (find and fix it), or the golden file is outdated (regenerate it *after reviewing the diff*). Blindly regenerating to make it green is almost always wrong.
3. E.g. real time → fake clock; unseeded randomness → seed and log it; shared state/order dependence → isolated fixtures; real I/O → fakes or `tmp_path`.
4. Real traces contain situations nobody wrote a requirement for — odd timing, corrupted frames, sequences of events — so they exercise combinations that specification-based tests omit.

Chapter 13 — Performance, timing and resource testing

1. $4/10 = 40\%$. Measure the execution time repeatedly (warm-up first) and assert that the **maximum** (and a high percentile with headroom) is below the 10 ms deadline — on the target hardware for a genuine WCET.
2. The mean hides the slow tail; a few long executions can miss deadlines while the average looks fine.
3. Measure at two input sizes and compare the ratio of times with the ratio of sizes: a $16\times$ larger input costing roughly $16\times$ (or more) more per item is a red flag.
4. It exposes algorithmic and relative performance regressions, but the compiler, CPU, cache and RTOS of the target determine the real WCET, which must be measured/analysed there.

Chapter 14 — Simulation, closed-loop testing and acceptance criteria

1. Windup shows up as a consequence over *time* — after long saturation the speed overshoots badly. A single call returns just one number; only a loop with a plant reveals the accumulated effect.
2. Not as stated (overshoot of what?). For example: "after a 20 km/h set-speed step at constant speed on a flat road, the peak speed shall not exceed the new target by more than 20 % of the step (4 km/h)".
3. It proves the scenario is hazardous (the wheel really locks without ABS), so the ABS result is meaningful rather than vacuous.
4. The exact distance depends on the model, step size and tuning; it freezes one run. A relative criterion ($\geq 10\%$ shorter than locked-wheel braking) captures the intent and survives modelling changes.

Chapter 15 — Test adequacy: coverage and mutation testing

1. Statement coverage 100 % (every line ran). Branch coverage 50 % — the `False` outcome of the `if` was never taken.
2. At least $n + 1 = 5$.
3. No test failed when the code was altered. Either a test is missing or too weak to notice that behaviour, or the mutant is *equivalent* (it does not change observable behaviour, so no test could kill it).
4. Coverage counts executed lines, not verified behaviour. A test with no assertions can reach 95 %; mutation testing (or review) is needed to show the assertions would actually detect faults.

Chapter 16 — Requirements, traceability, standards and continuous integration

1. For example: "When no valid wheel-speed frame has been received for 100 ms, the instrument cluster shall display `--` (REQ-CLU-001)." — unambiguous, measurable, identified.
2. Requirements with no verifying test (gaps) and tests that cite non-existent requirements (typos, stale references / orphan tests).
3. Fast checks give the quickest feedback and fail early without spending time on slow stages; expensive stages (stable timing, hours-long HIL) run only when the cheap ones pass or on a schedule.
4. The fixture supplies a different back-end per parameter while the test body is unchanged; the `hil` variant is skipped unless hardware is available (or `--hil` is passed), so it only runs on the bench.

Chapter 17 — Putting it all together: building a test strategy

These questions have no single right answer; compare your reasoning with the recipe in §17.1 and the techniques in Chapters 6, 7, 10, 11 and 15. A strong answer names concrete boundary values and hostile inputs (NaN, empty, wrap-around), identifies a seam (clock, hardware, bus), states at least one invariant and one safety property, and pairs coverage (what is untested) with mutation score (what is not checked) while acknowledging that neither proves correctness.